

Rechnernetze & Verteilte Systeme

Ludwig-Maximilians-Universität München

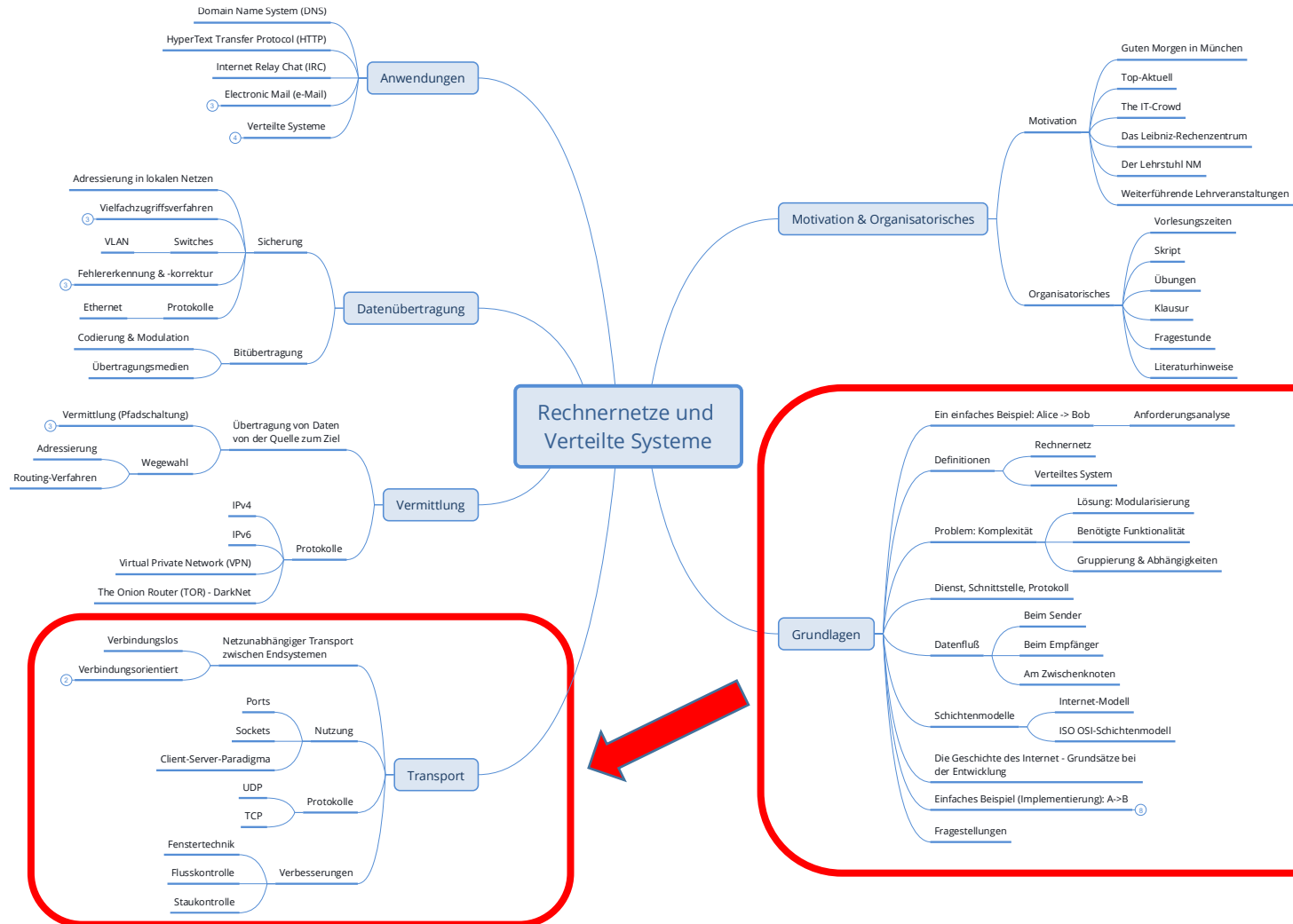
Sommersemester 2026

Prof. Dr. D. Kranzlmüller

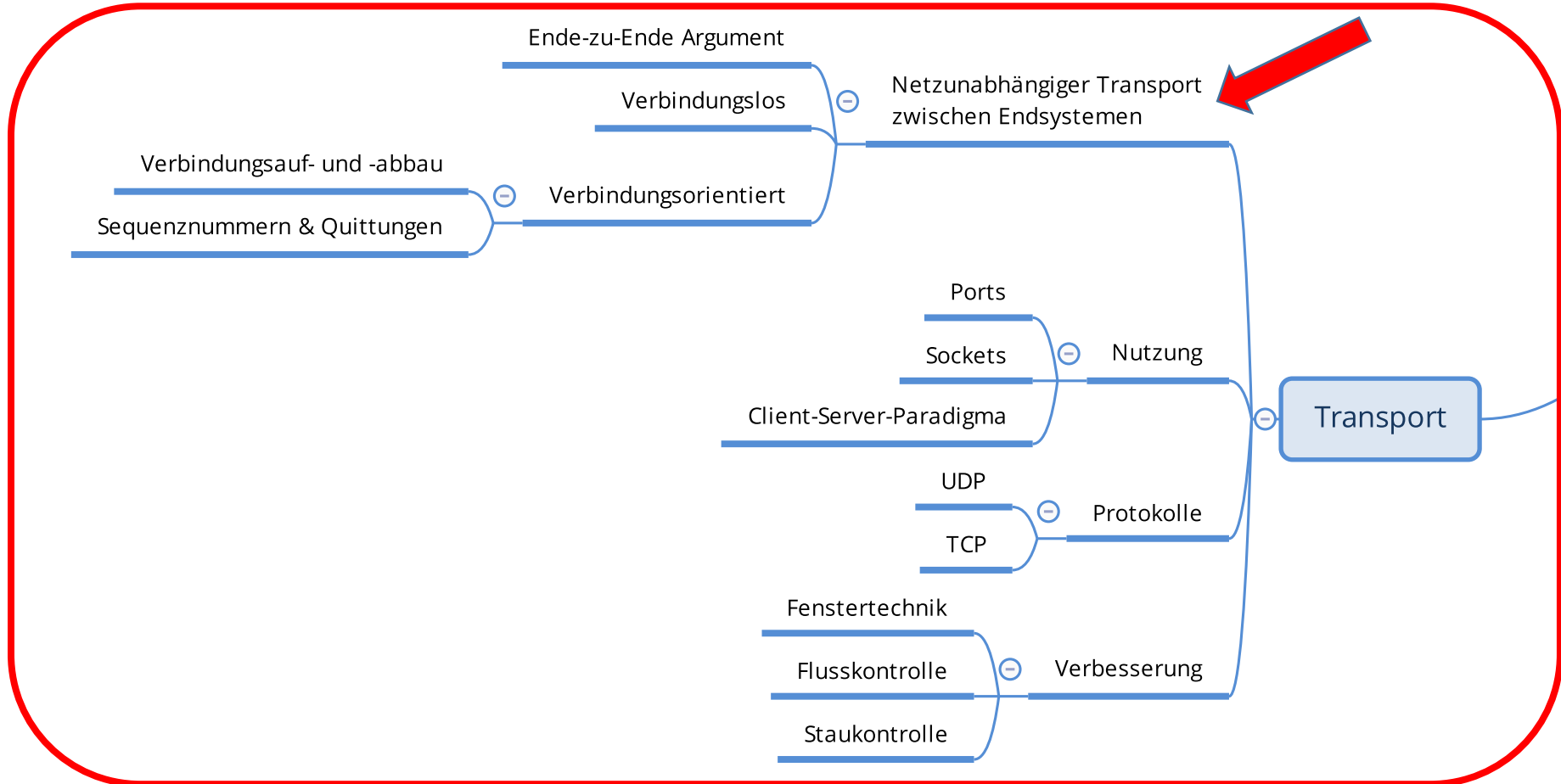
<https://www.ifi.lmu.de/rnvs>



RNVS MindMap

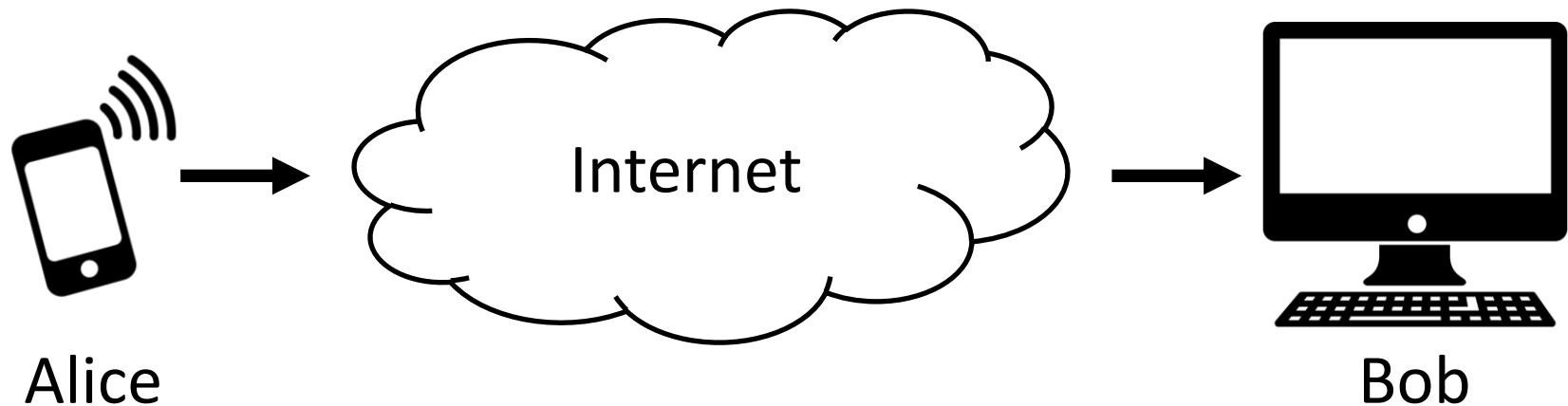


RNVS – Kapitel 3



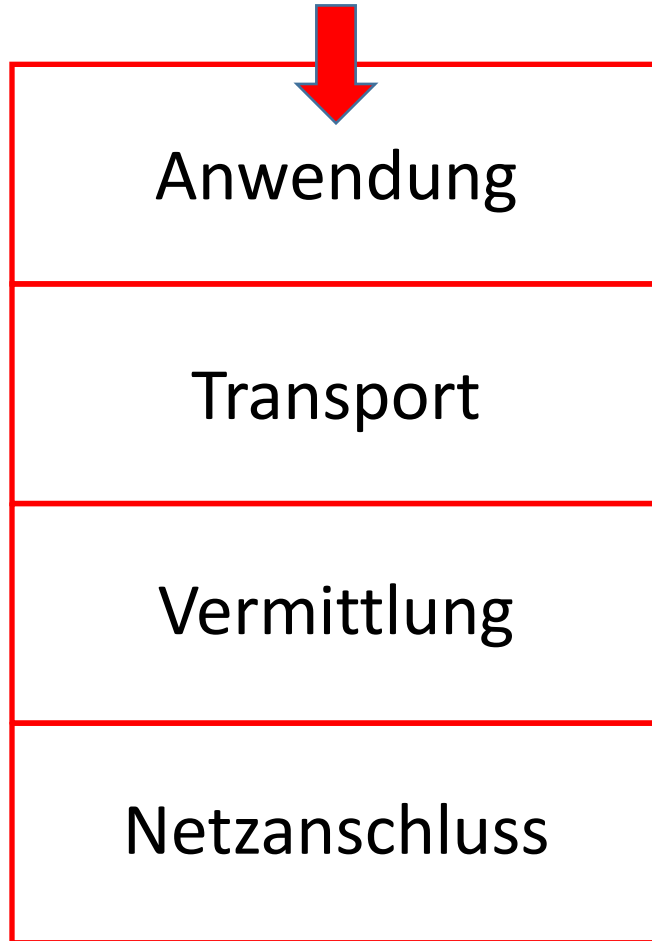
Einfaches Beispiel

- Alice sendet eine Nachricht an Bob



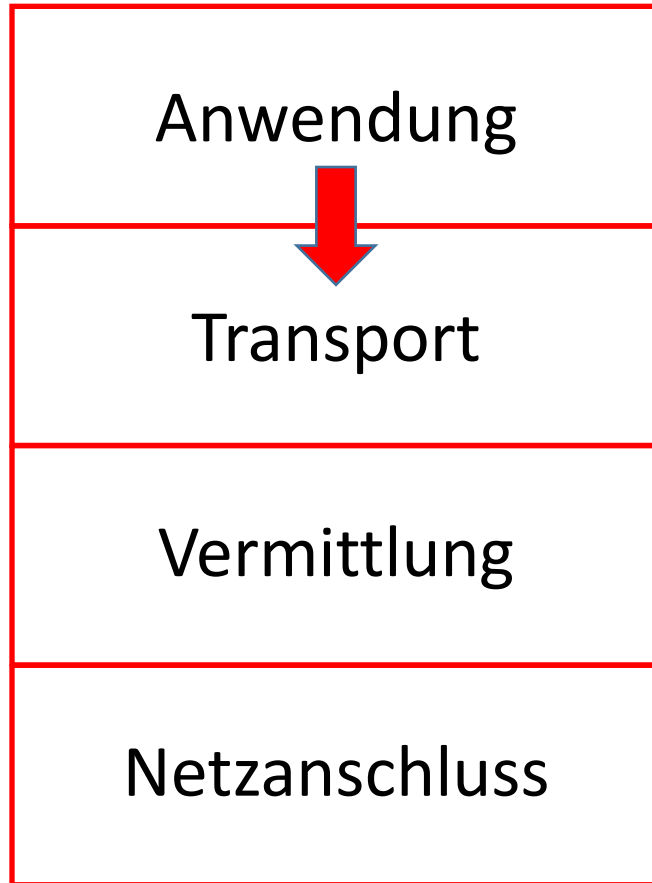
Computer by Patrick Morrison from the Noun Project
Smartphone Transmission by Roberto Chiaveri from the Noun Project

Anwendung des Internet-Schichtenmodells



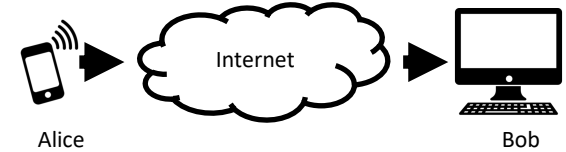
```
send_http_request  
(host, message)
```

Anwendung des Internet-Schichtenmodells



$$\left. \begin{array}{l} \text{ICI} = \{ \text{host, port} \} \\ \text{ID} = \{ \text{request} \} \end{array} \right\} \text{IDU}$$

Datenfluss beim Sender

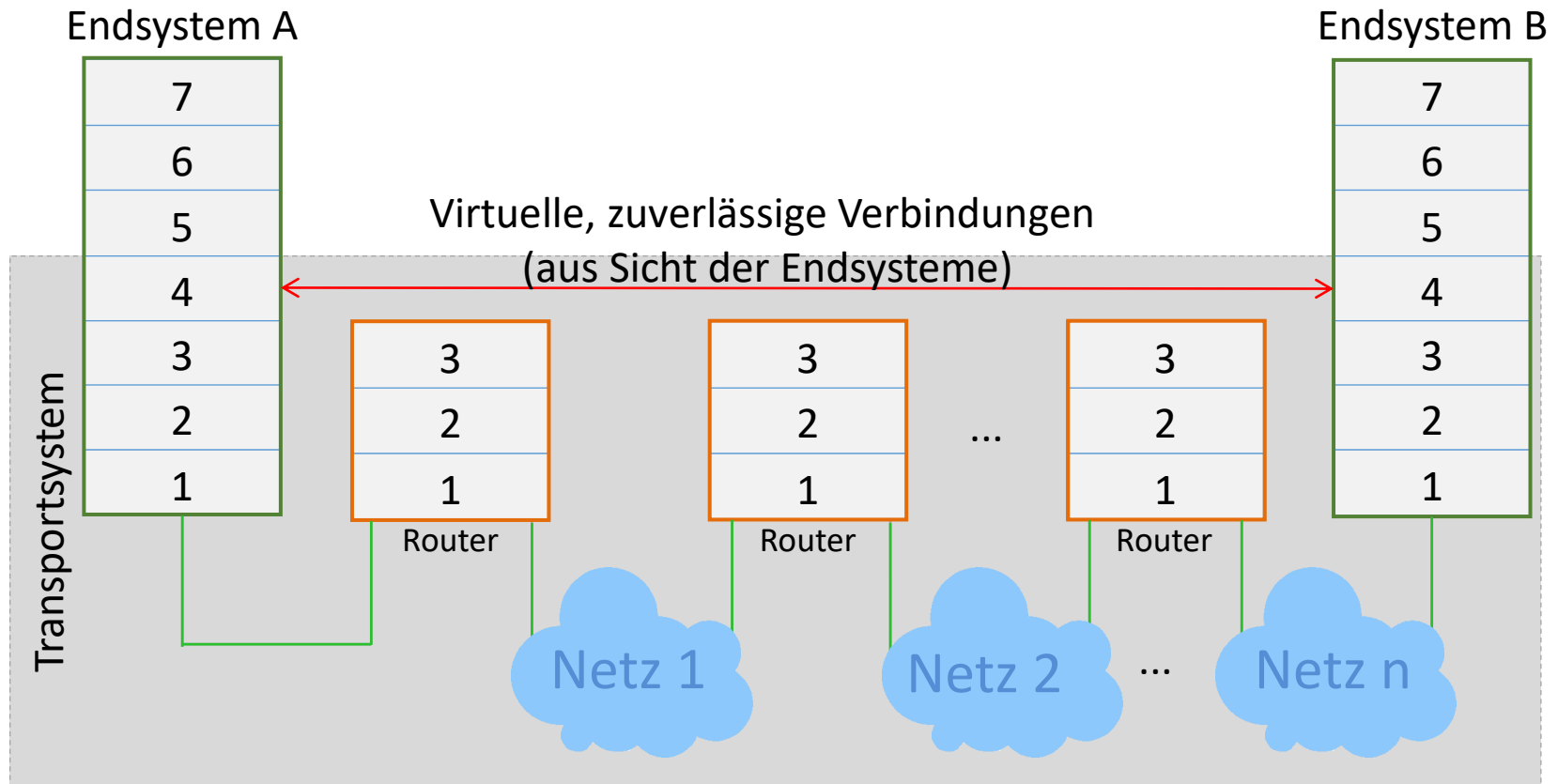


↓ Nachricht,
Ziel

↓ Daten,
Verb.Info

- Transport der Nachricht
 - Aufbau und Abbau der Verbindung
 - Mit/ohne Zustellgarantie
 - Über unsichere Kanäle
 - Mit Verkehrsmanagement (Stau, ...)
- Weg durch das Netz
 - Vermittlung/Pfadschaltung
 - Wegewahl im lokalen Netz / im Internet
- Datenübertragung
 - Sicherung
 - Bitübertragung

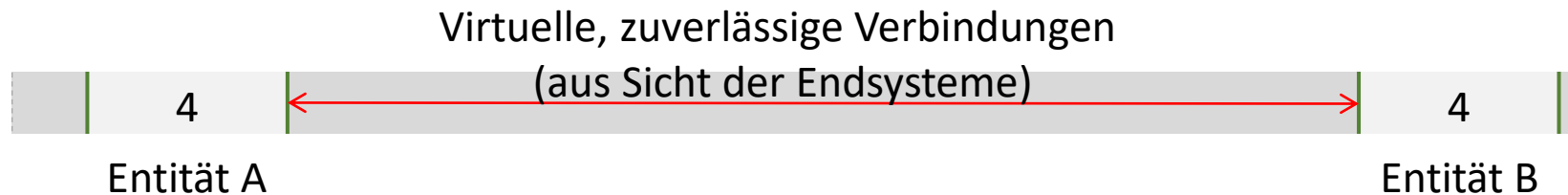
Transportschicht im ISO/OSI Referenzmodell



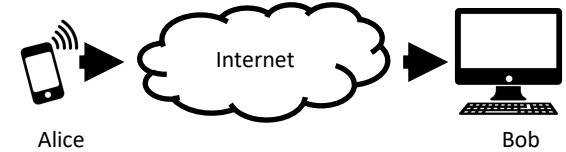
Ende-zu-Ende Betrachtung im ISO/OSI Referenzmodell

Endsystem A

Endsystem B



Datenfluss beim Sender



-
- Transport der Nachricht
 - Aufbau und Abbau der Verbindung
 - Mit/ohne Zustellgarantie
 - Über unsichere Kanäle
 - Mit Verkehrsmanagement (Stau, ...)
-

 Nachricht,
Ziel

 Daten,
Verb.Info

Anforderung an Transportschicht

- Dienste der darüberliegenden Anwendungsschicht erwarten
 - Transport der Nachricht zwischen den Endsystemen
 - mit oder ohne Zuverlässigkeitsgarantien
- Dienste der darunterliegenden Vermittlungsschicht bieten
 - Übertragung der Nachrichten über Transitsysteme
 - ohne Zuverlässigkeitsgarantien
(nach dem „*Best Effort*“-Prinzip)

Beispiel: Bei Überlastung können Pakete in der Vermittlungsschicht verworfen werden.

Mögliche Probleme bei der Nachrichtenzustellung (in einem unzuverlässigen Netz)

- Verlust von Nachrichten
- Verdoppelung von Nachrichten
- Verfälschung der Inhalte von Nachrichten
- Vertauschung der Reihenfolge mehrere Nachrichten
- Verzögerung bei der Übertragung der Nachrichten

Ende-zu-Ende Argument

- Grundproblem aus Sicht der Nutzer von Endsystemen:
 - Transitsysteme sind allgemein außerhalb der Kontrolle der Anwendung und oft unzuverlässig
 - Die Anwendung hat gewisse Anforderungen an die Transitsysteme, die nur die Anwendung kennt.
- Ende-zu-Ende Argument:
 - Unzuverlässigkeit der Transitsysteme wird akzeptiert
 - In den Endsystemen wird in den höheren Schichten eine umfangreiche Fehlerbehandlung durchgeführt

Ende-zu-Ende Argument

Zuverlässigkeitsgarantien:

- Auslieferungsgarantien
- Unterdrückung von Nachrichtenverdopplung
- Sichere Datenübertragung
- Garantierte Zustellungsreihenfolge

Zielkonflikt:

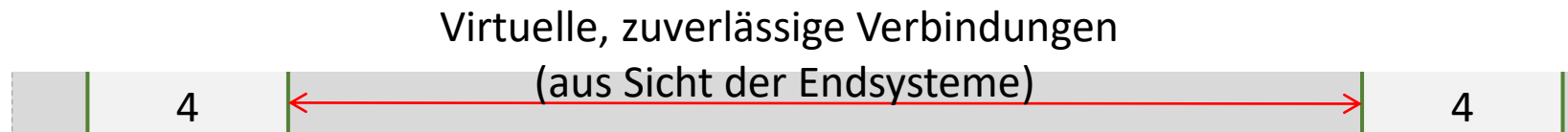
Zuverlässigkeitsgarantien $\Leftrightarrow$ Kommunikationsleistung

➔ Trade-Off (Kosten-Nutzen-Abwägung) erforderlich

Ende-zu-Ende Betrachtung im ISO/OSI Referenzmodell

Endsystem A

Endsystem B



Realisierung des Transportdienstes



- **Verbindungslos:** keine Verbindung zwischen Entitäten
 - Zustand der Entitäten irrelevant (keine zeitliche Abhängigkeit)
 - Geringe Möglichkeiten zur Fehlererkennung/-korrektur
 - Hohe Übertragungsleistung (aufgrund des geringen Synchronisations-/Verarbeitungsaufwands)
- **Verbindungsorientiert:**
 - Abhängigkeit zwischen den Entitäten (Kontext relevant)
 - Umfangreiche Fehlerbehandlung (**Zuverlässigkeit**)
 - Höherer Verarbeitungsaufwand (notwendig)

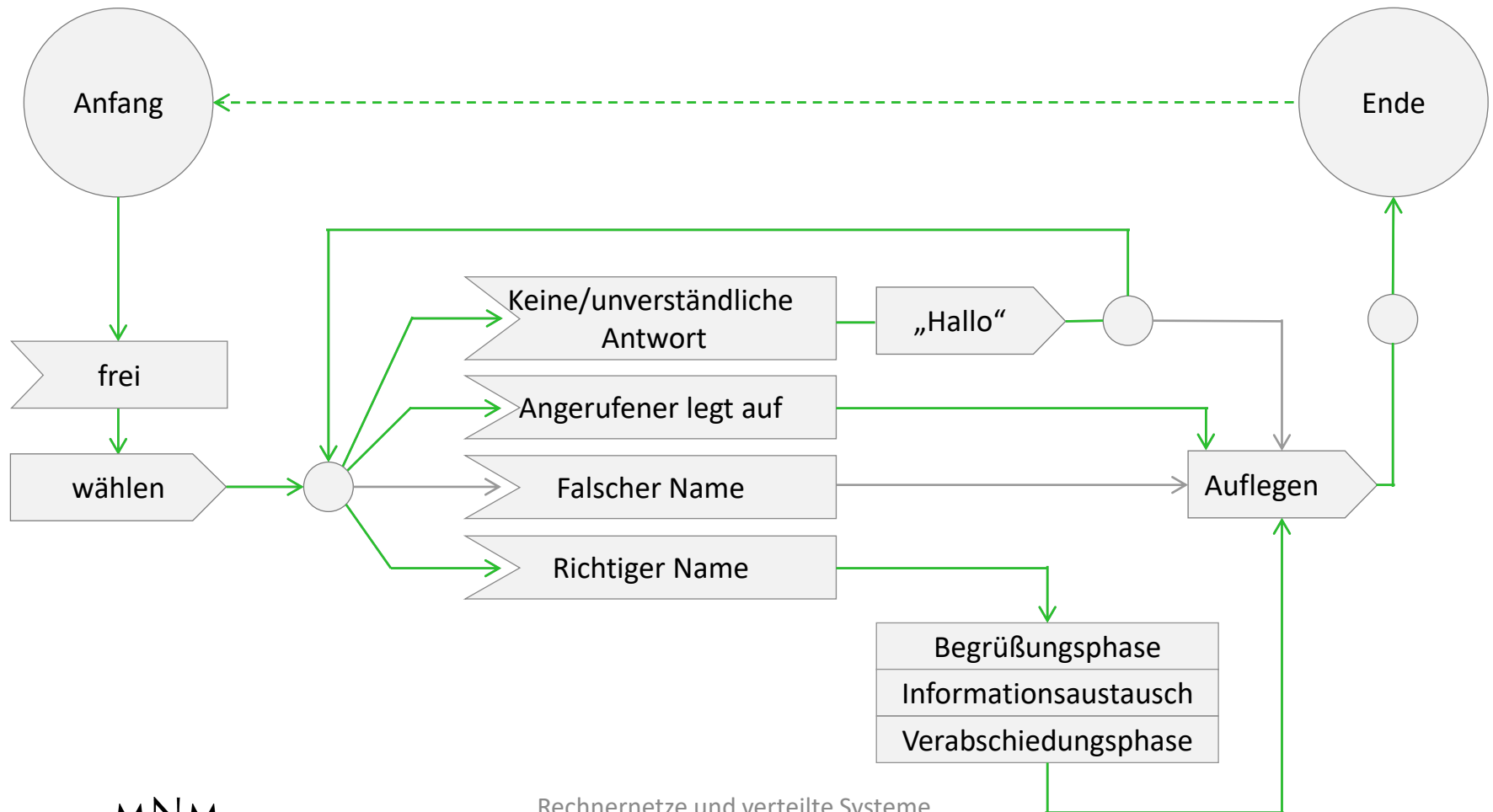
Verbindungsloser Dienst (connectionless service)

- Keine Verbindung zwischen den Entitäten
- Keine Empfangsbestätigung (Quittierung)
- Sender hat keine Information darüber, ob die Nachricht das Ziel erreicht hat
- ICI enthält alle Informationen zum Ziel
- Nachricht sucht selbständig Weg durch das Netz
- Geringer Verwaltungsaufwand → Hohe Leistung (im Vergleich zur verbindungsorientierten Kommunikation)

Fragen

- Ist das „Beispiel:Telefongespräch“ aus Kapitel 2 verbindungslos oder verbindungsorientiert?

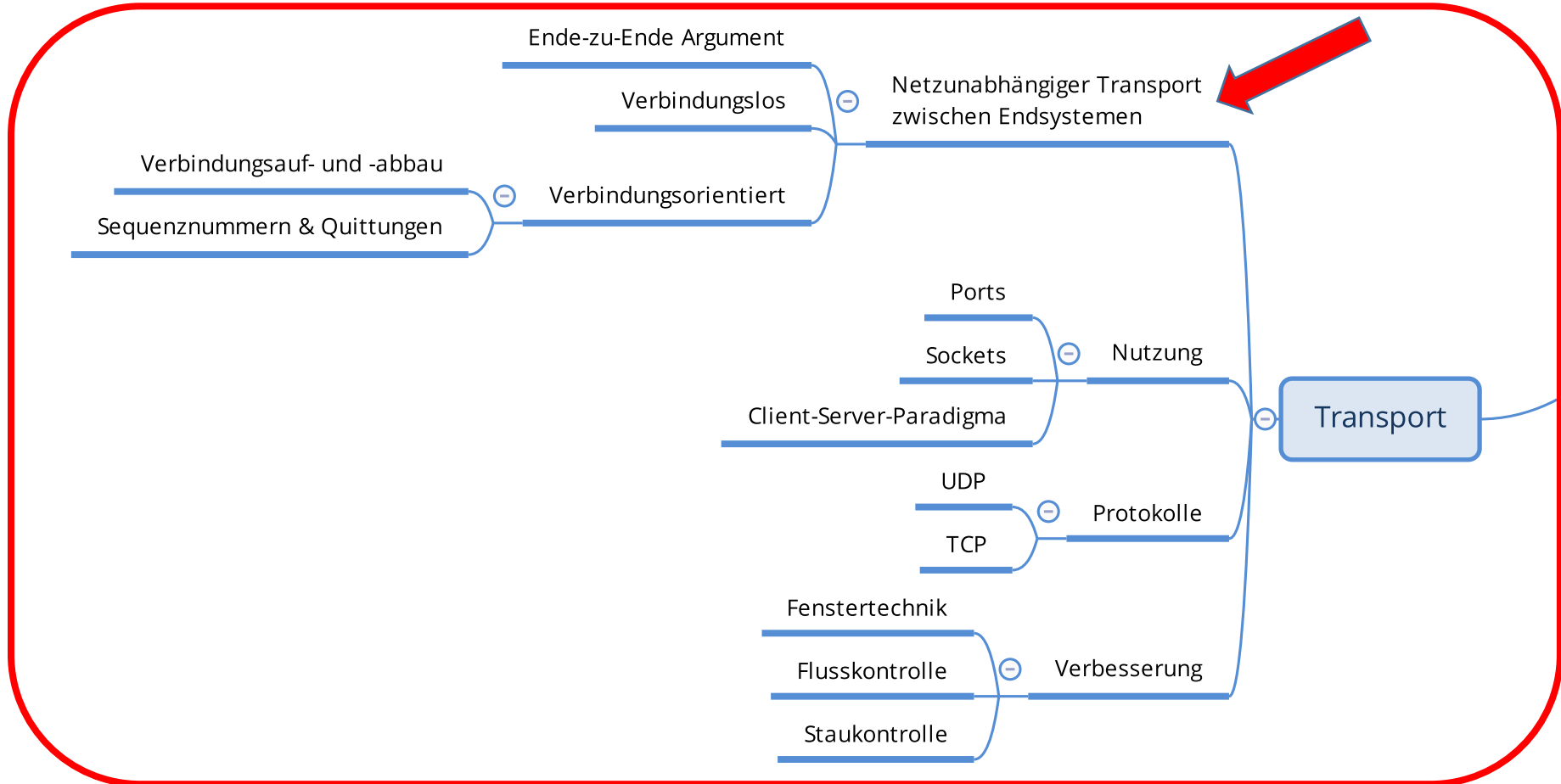
Beispiel: Telefongespräch



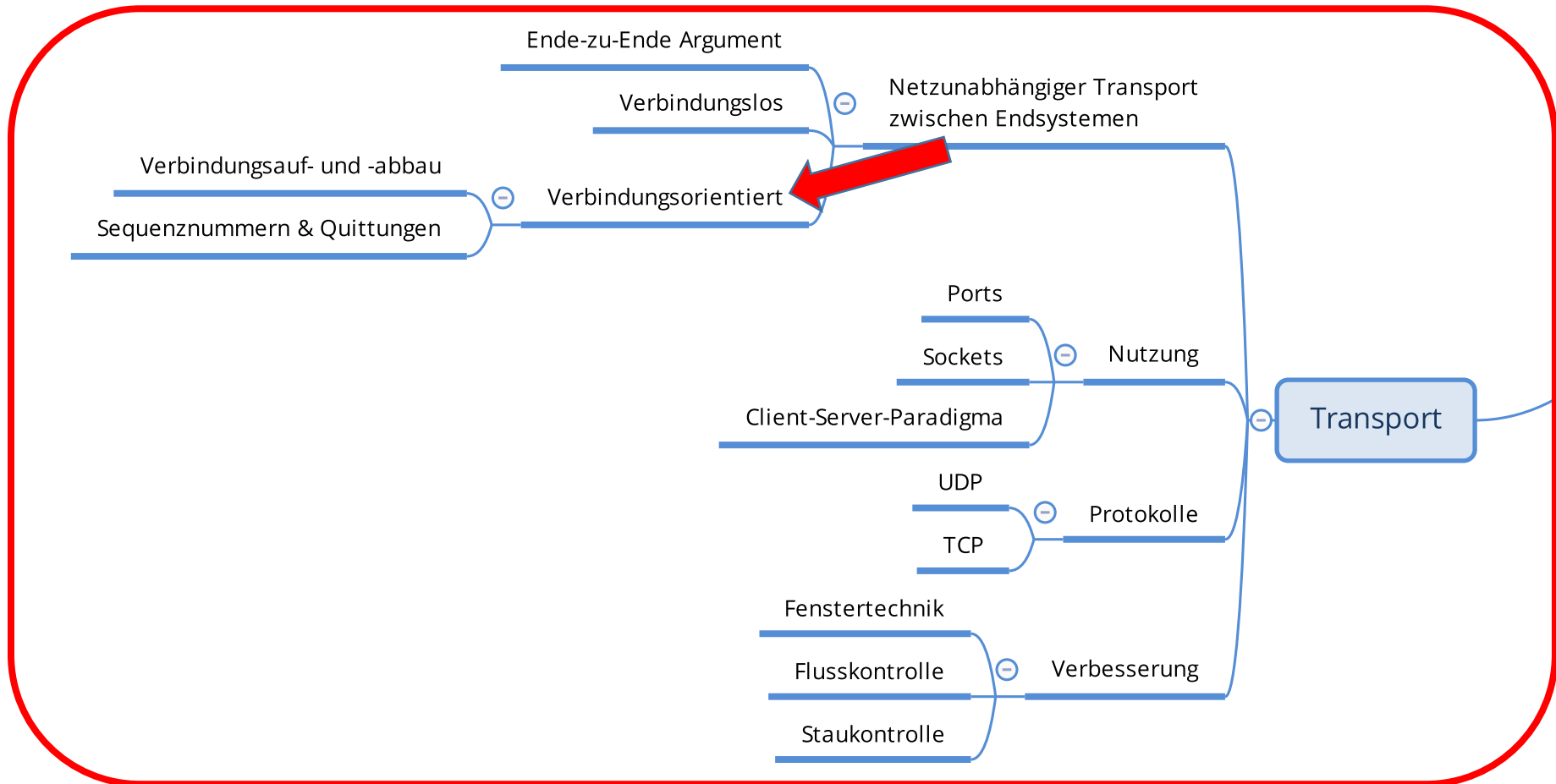
Fragen

- Ist das „Beispiel:Telefongespräch“ aus Kapitel 2 verbindungslos oder verbindungsorientiert?
- Welche Arten von Fehlern werden im „Beispiel:Telefongespräch“ vom Protokoll behandelt?
- Nennen Sie ein Beispiel für eine verbindungslose Ende-zu-Ende Kommunikation

RNVS – Kapitel 3



RNVS – Kapitel 3



Verbindungsorientierter Dienst (connection-oriented service)

- Aufbau einer logischen Verbindung (Sitzung)
- Strategien zur Fehlerbehandlung:
 - Kein Verlust von Nachrichten
 - Keine Verdoppelung von Nachrichten
 - Keine Verfälschung der Inhalte von Nachrichten
 - Keine Vertauschung der Reihenfolge mehrere Nachrichten
- Verbindungsmanagement durch Flusssteuerung und Staukontrolle zur Handhabung von Verzögerungen bei der Nachrichtenübertragung

Verbindungsorientierter Dienst

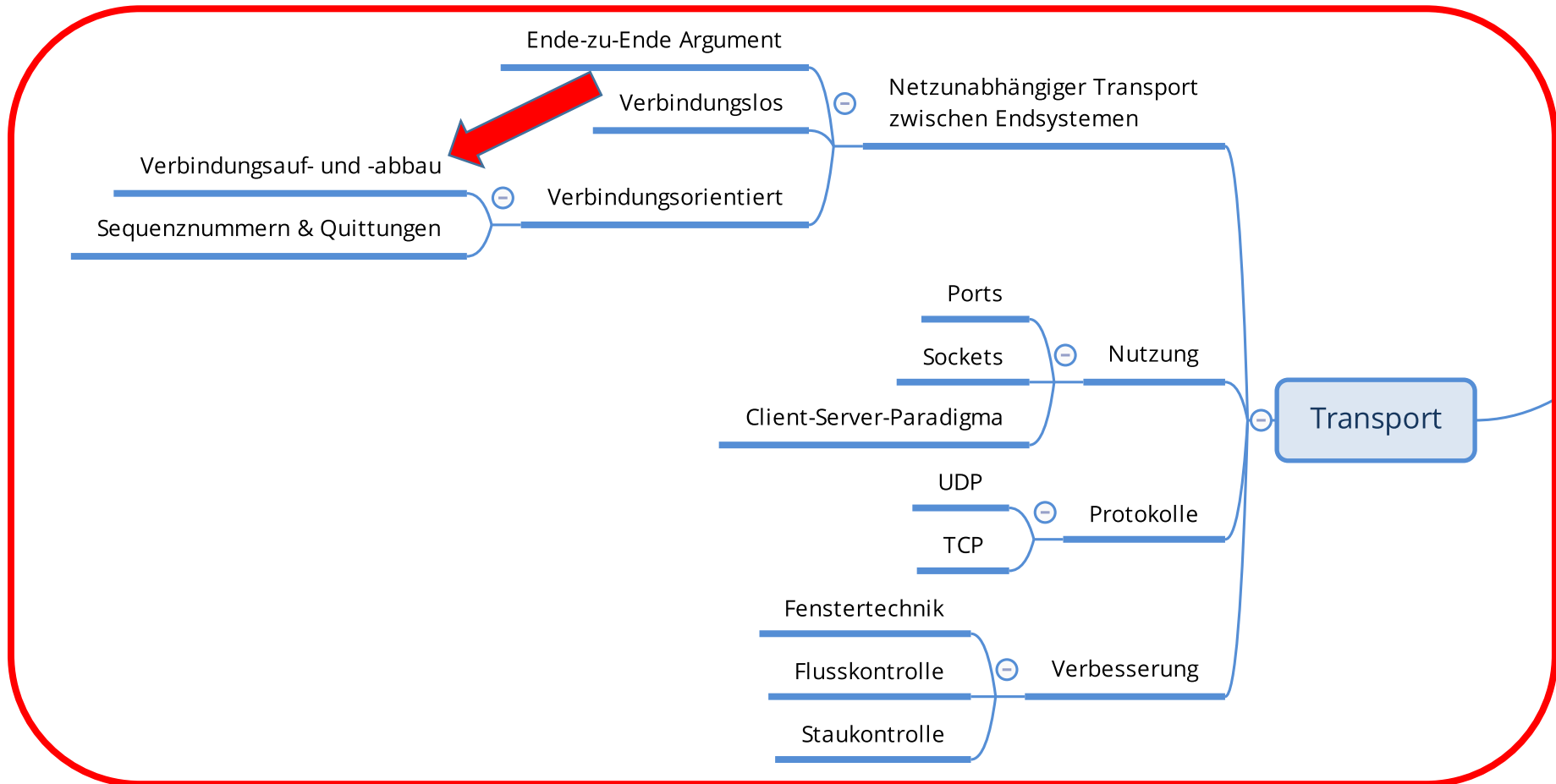
Techniken:

- Verbindungsauf-/abbau
- Sequenznummern und Quittungen
- Flusssteuerung und Staukontrolle

Vorgehensweise:

1. Theoretische Überlegungen
2. Technische Umsetzung
3. Implementierung (TCP / UDP)

RNVS – Kapitel 3



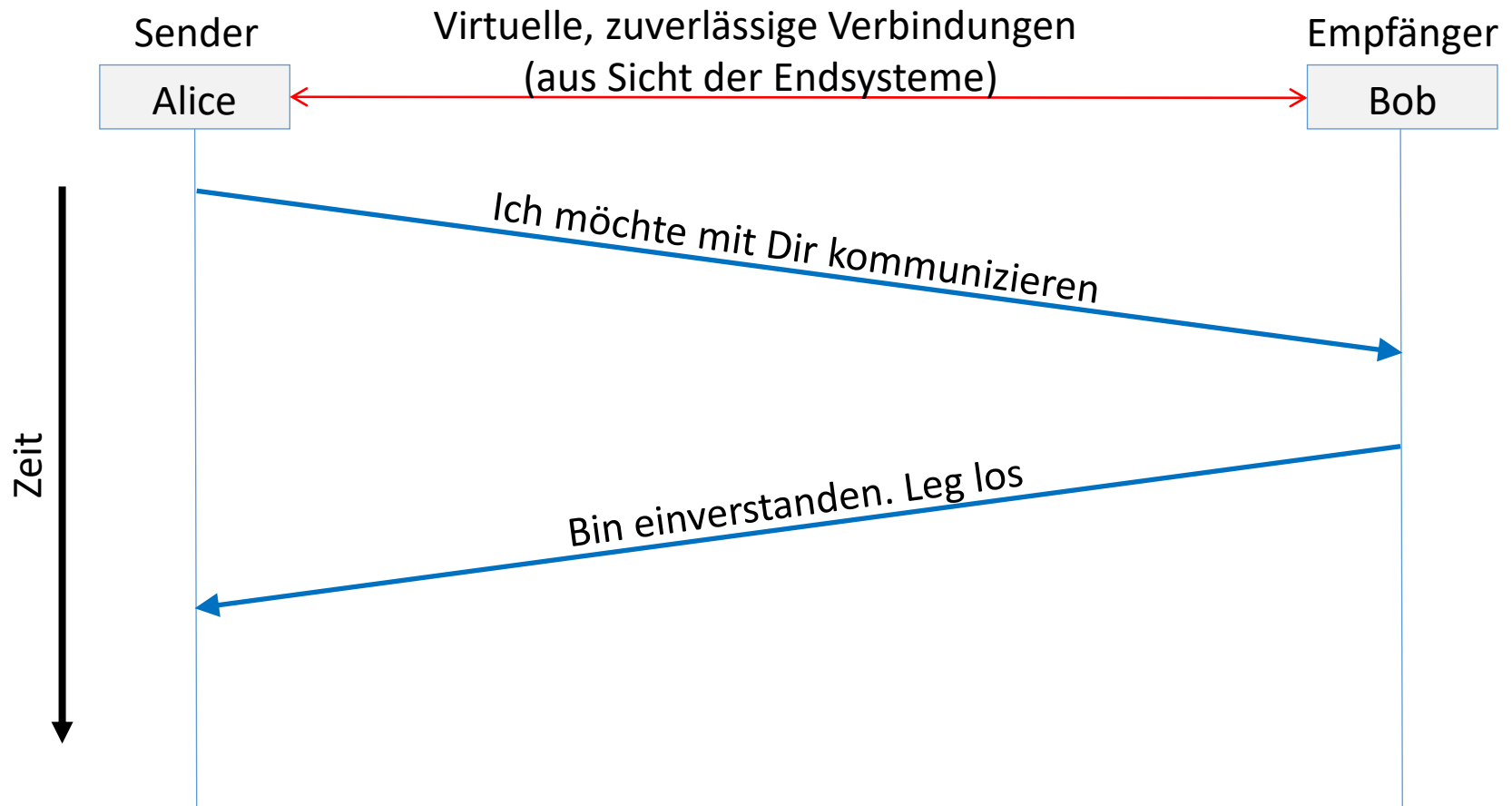
Verbindungsaufbau

- Anforderung: Am Beginn einer Kommunikation zum Aufbau einer logischen Verbindung
- Fragestellungen:
 - Wie wird dem Kommunikationspartner der Wunsch zum Aufbau einer neuen Verbindung mitgeteilt?
 - Wie kann der Empfänger einen neuen Verbindungswunsch von früheren Aufbauwünschen unterscheiden?
- Lösungskonzept: Handschlag-Protokoll

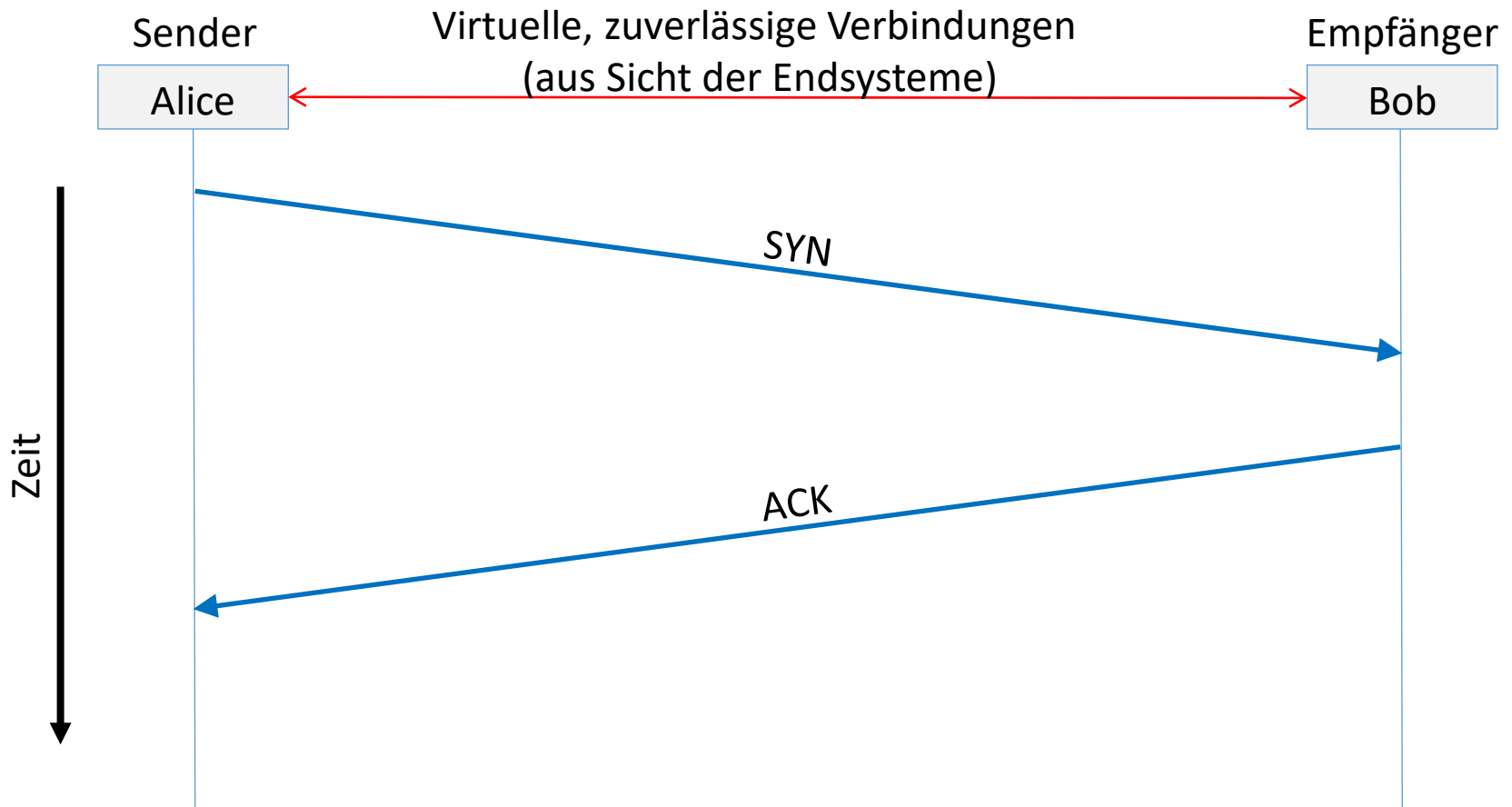
Handschlag-Protokoll - Vorbereitung

- Festlegung spezieller Nachrichtentypen für das Verbindungsmanagement als Teil der Steuerinformation (PCI):
 - SYN = Bitte zum Aufbau einer Verbindung (oder zur Synchronisation)
 - CLS = Bitte zum Abbau der Verbindung (“close”)
 - ACK = Positive Bestätigung („acknowledge“)
 - NAK = Negative Bestätigung
- Teil der Protokollbeschreibung

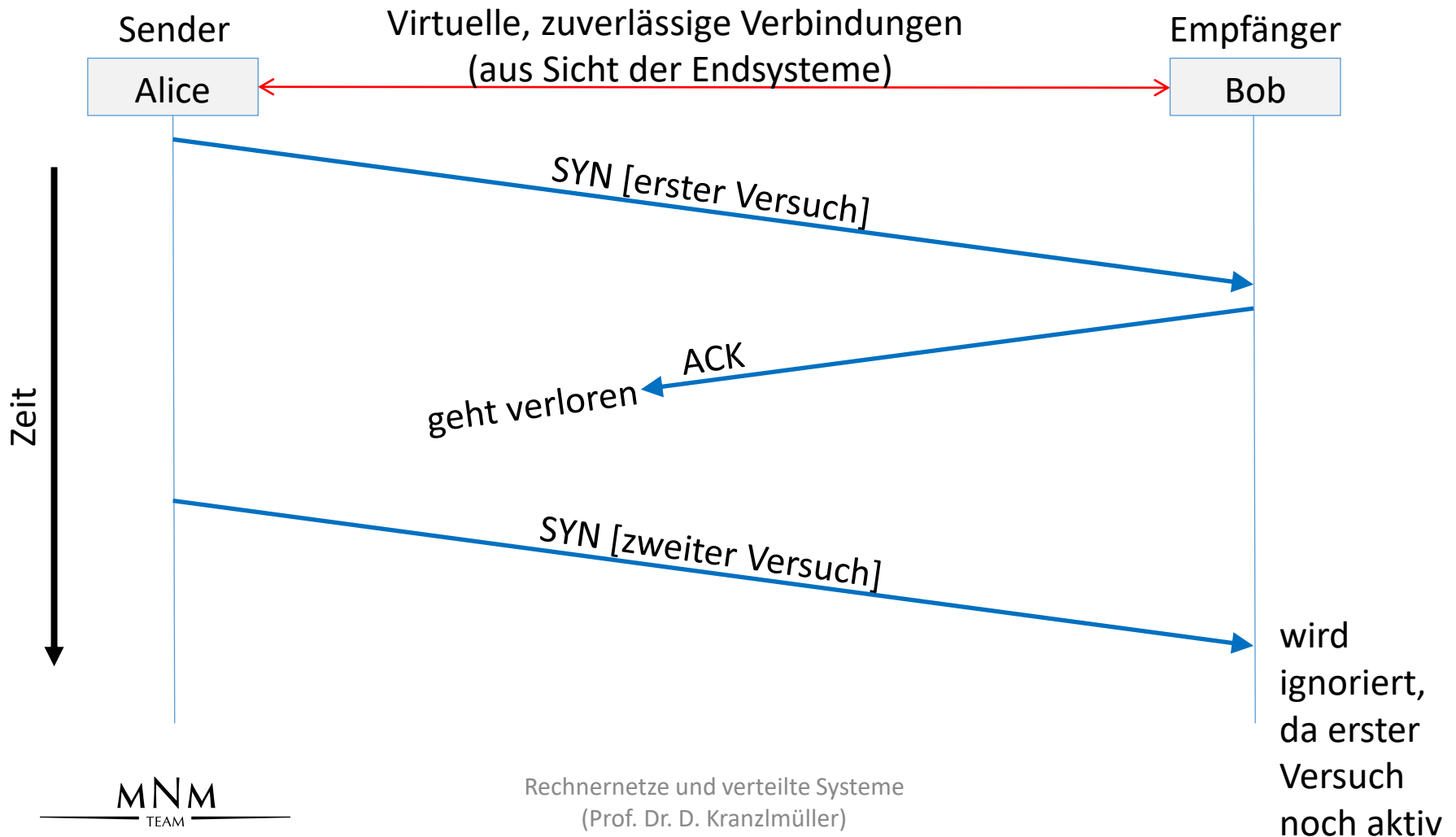
Handschlag-Protokoll



2-Wege-Handschlag (2-Way Handshake)



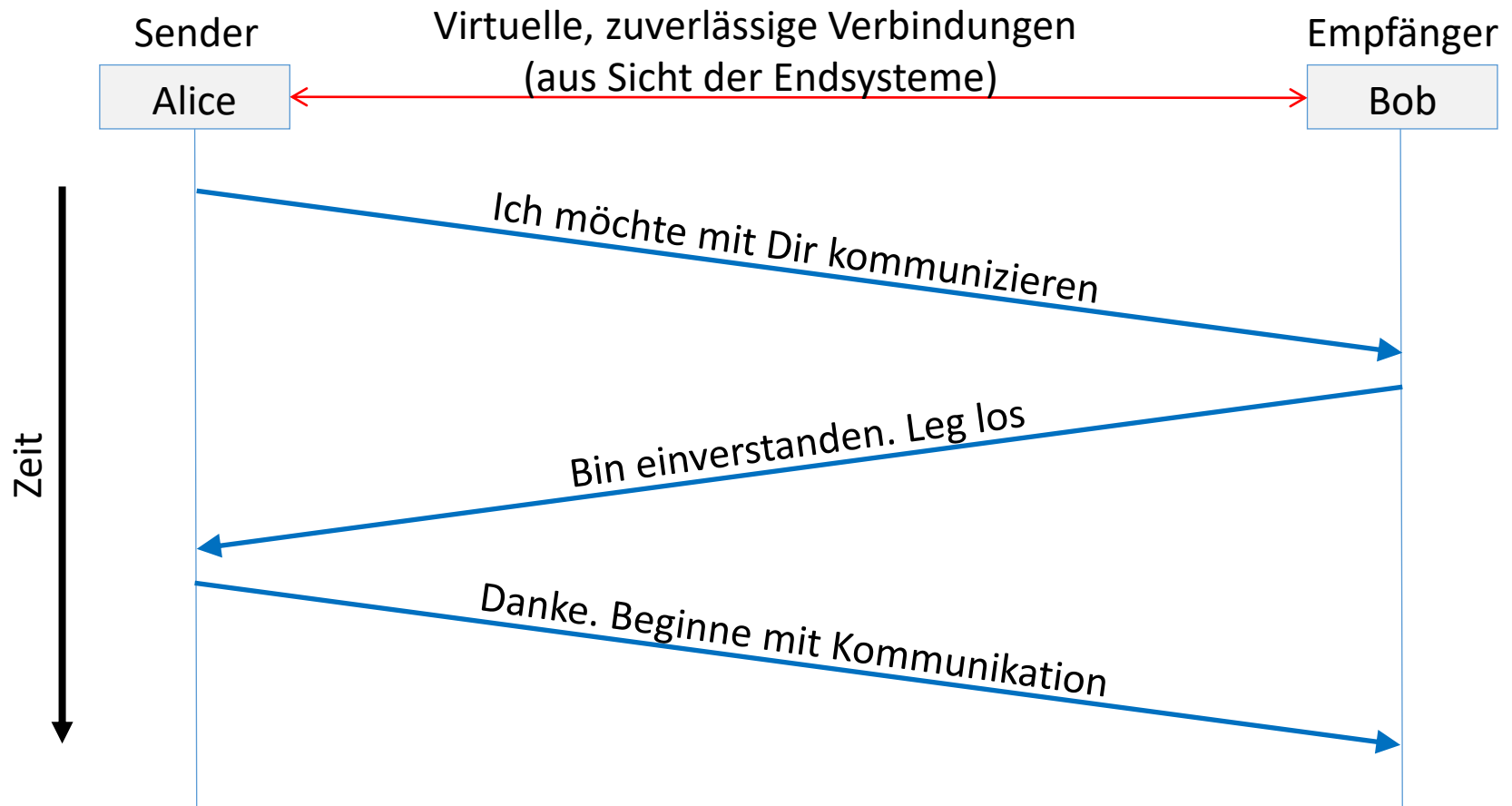
Problem beim 2-Wege-Handschlag



Problem beim 2-Wege-Handschlag

- Es gibt eine Reihe von Möglichkeiten, wie Kommunikationsfehler einen Verbindungsaufbau verhindern können.
- **Übung:** Finden Sie ein anderes Beispiel, bei dem der 2-Wege-Handschlag nicht zum Verbindungsaufbau führt.
- Frage: Wie kann man das Problem lösen?

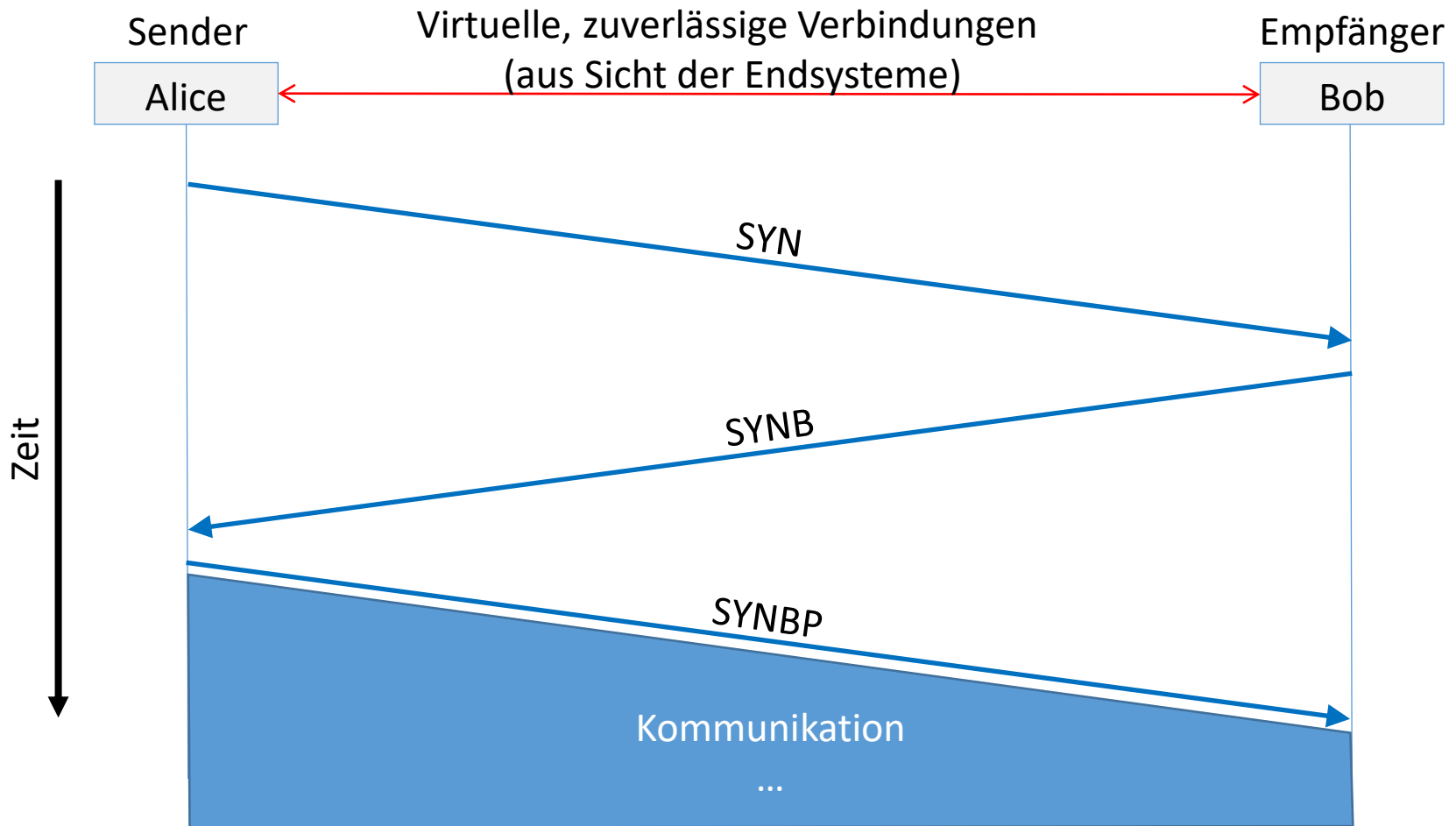
Handschlag-Protokoll



Handschlag-Protokoll - Protokollerweiterung

- Festlegung spezieller Nachrichtentypen für das Verbindungsmanagement als Teil der Steuerinformation (PCI):
 - SYN = Bitte zum Aufbau einer Verbindung (oder zur Synchronisation)
 - SYNB = Bestätigung des Aufbauwunsches
 - SYNBP bzw. SYNBN = positive bzw. negative Bestätigung der Bestätigung
 - CLS = Bitte zum Abbau der Verbindung (“close”)
 - ACK = Positive Bestätigung („acknowledge”)
 - NAK = Negative Bestätigung

3-Wege-Handschlag (3-Way Handshake)



Verbindungsabbau

- Anforderung: Am Ende einer Kommunikation zum Beenden/Schließen einer logischen Verbindung
- Fragestellungen:
 - Wie wird dem Kommunikationspartner der Wunsch zum Abbau einer neuen Verbindung mitgeteilt?
 - Wie können beide Kommunikationspartner die Kommunikation beenden?
 - Müssen beide Kommunikationspartner gleichzeitig aufhören?

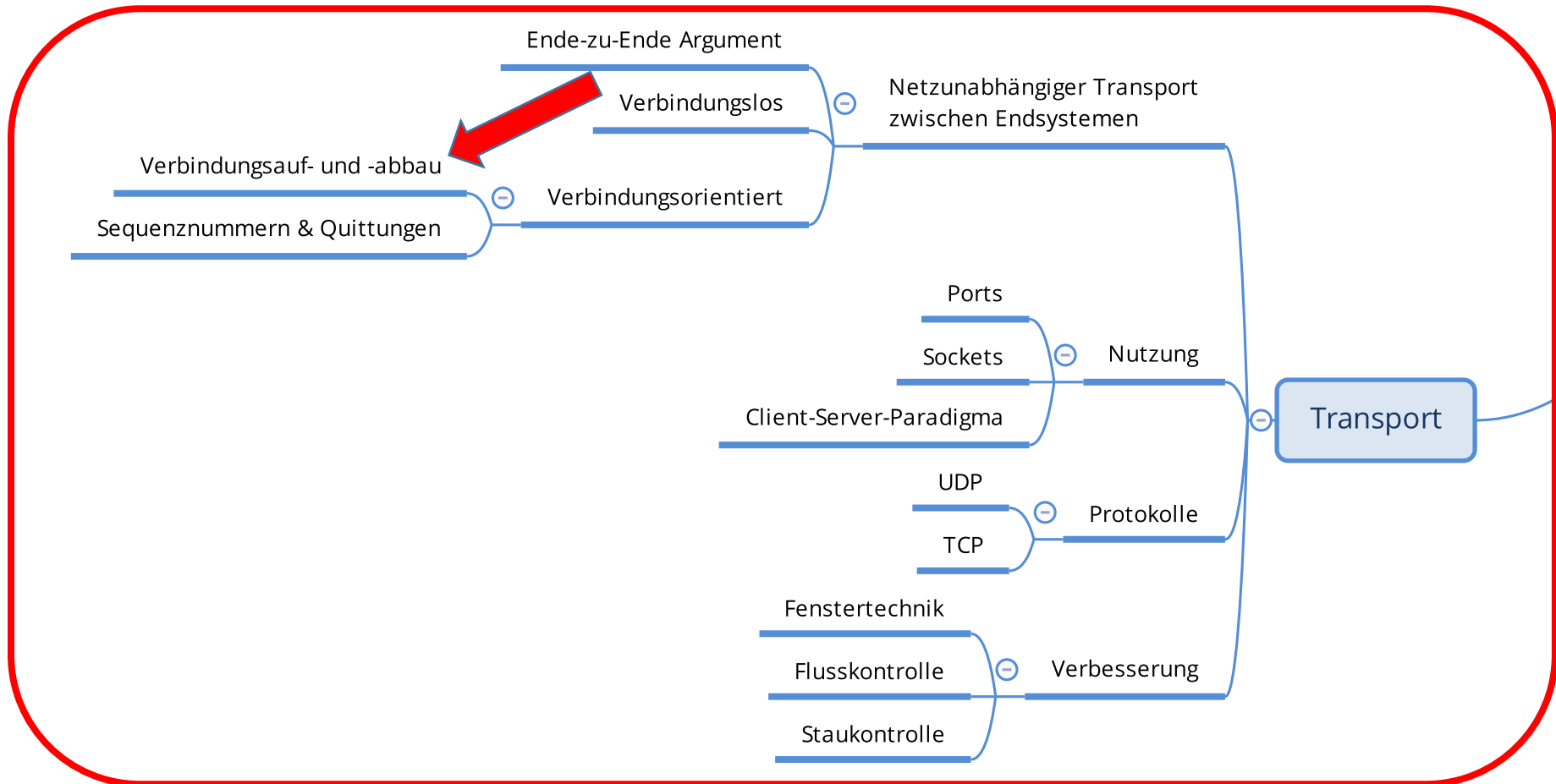
Handschlag-Protokoll - Erweiterung Verbindungsabbau

- Festlegung spezieller Nachrichtentypen für das Verbindungsmanagement als Teil der Steuerinformation (PCI):
 - CLS = Bitte zum Abbau der Verbindung (“close”)
 - CLSB = Bestätigung des Abbauwunsches
- Der Verbindungsabbau muss von beiden Kommunikationspartner initiiert werden → beide „Halbverbindungen“ werden explizit geschlossen
- **Übung:** Entwerfen Sie ein Zustandsdiagramm

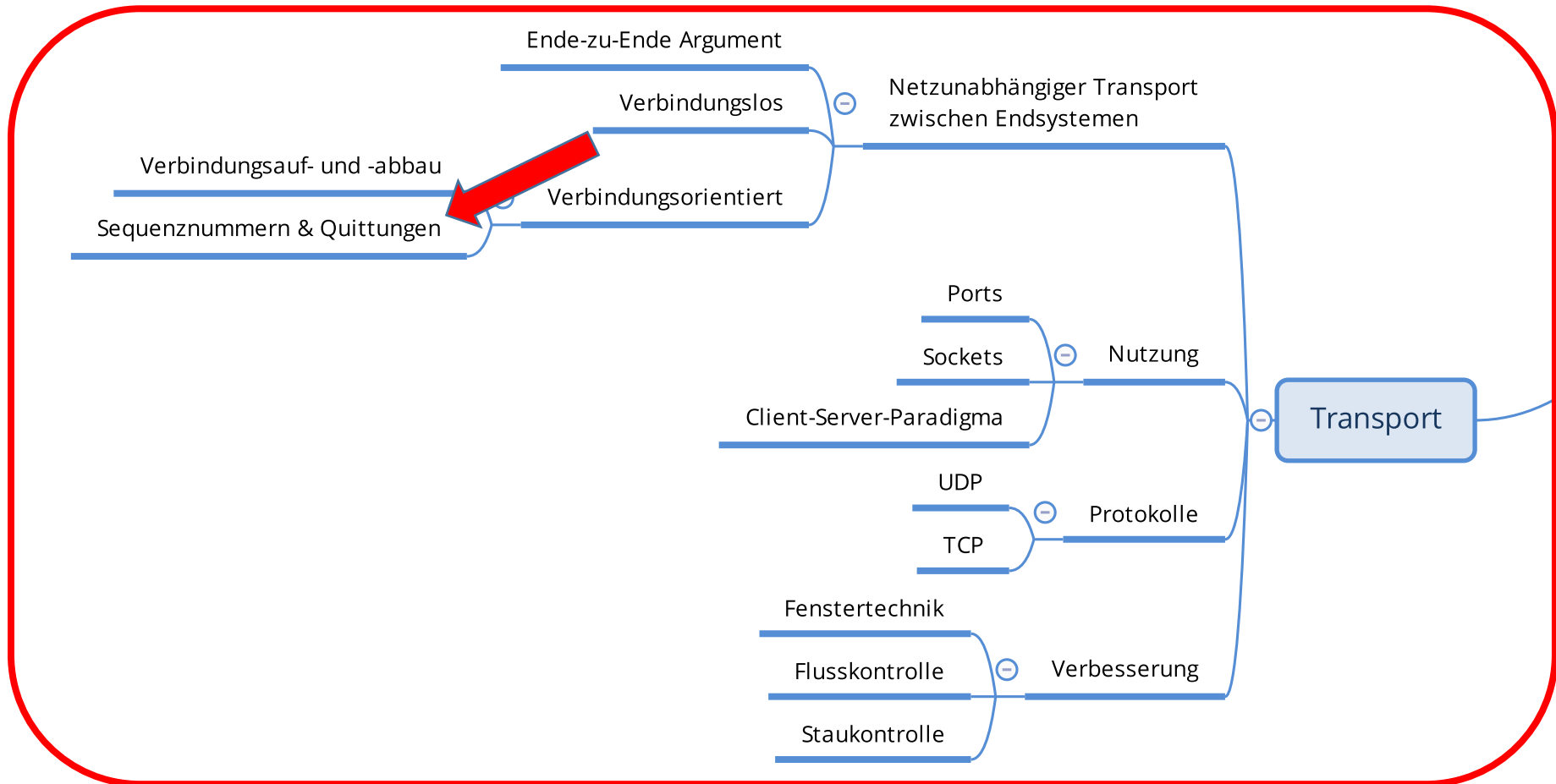
Zusammenfassung

- Verbindungsauf- und -abbau sind Funktionen eines verbindungsorientierten Dienstes
- Für den Aufbau einer Verbindung verwenden wir den 3-Wege-Handschlag
- Der Abbau einer Verbindung wird durch die symmetrische Verbindungsfreigabe (=Schließen beider Halbverbindungen) durchgeführt
- Für beide Funktionen werden spezielle Nachrichtentypen als Teil des PCI festgelegt

RNVS – Kapitel 3



RNVS – Kapitel 3



Zuverlässige Kommunikation

- **Zur Erinnerung:**

- Ein verbindungsorientierter Dienst realisiert eine zuverlässige Kommunikation trotz unzuverlässiger Transitsysteme

- **Zuverlässigkeitsgarantien:**

- Auslieferungsgarantien
- Unterdrückung von Nachrichtenverdopplung
- Sichere Datenübertragung
- Garantierte Zustellungsreihenfolge

- **Frage:** Wie könnte man das Problem lösen?

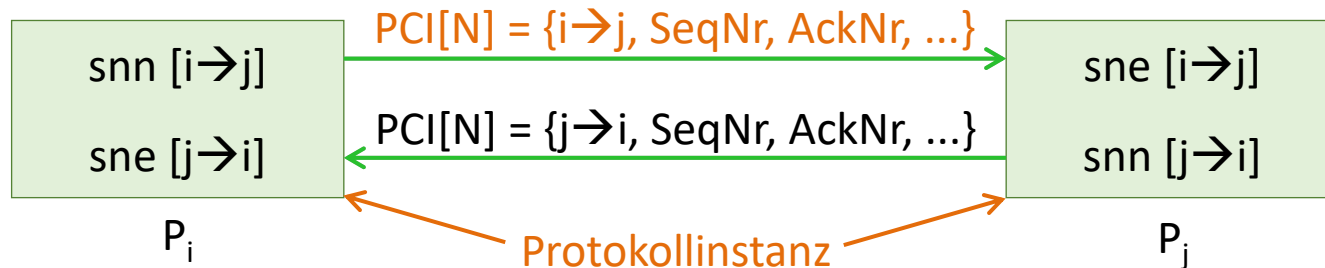
Sequenznummern

- Zähler für Nachrichtenübermittlung: Jede übertragene Nachricht (PDU) wird durchnummeriert
 - Auslieferungsgarantie durch Bestätigung angekommener Nachrichten (➔ Quittung)
 - Nachrichtenverdopplung wird anhand identischer Nummern erkannt
 - Fehlende Nachrichten erkennt man durch Lücken in der Nummerierung
 - Zustellungsreihenfolge kann durch Sortierung erreicht werden.
- Standardlösung verbindungsorientierter Protokolle

Quittungen

- Empfangsbestätigung: Empfänger quittiert den erfolgreichen Empfang einer Nachricht (PDU)
- Mögliche Realisierung:
 - Positiv/Negativ: PDU mit Sequenznummer N ist angekommen/nicht angekommen
 - Sammelquittung für mehrere aufeinander folgende PDUs: PDUs bis Sequenznummer N sind angekommen
 - Selektive Quittung für eine Untermenge der PDUs
- Erweiterung durch Timer (Zeitschalter)
 - Ankunft innerhalb festgelegten Zeitfensters erwartet

Notation: Sequenznummern



- $\text{snn } [i \rightarrow j]$: SeqNr der nächsten zu sendenden PDU
- $\text{sne } [i \rightarrow j]$: SeqNr der nächsten erwarteten PDU
- $\text{PCI}[N] = \{i \rightarrow j, \text{SeqNr}, \text{AckNr}, \dots\} =$
Steuerinformationen
 - $i \rightarrow j$: Verbindungs- und Richtungskennung
 - SeqNr: SeqNr der Nachricht
 - AckNr: SeqNr der nächsten erwarteten Nachricht (= sne)

Beispiel: Sequenznummern

Nachricht

The quick brown fox jumps over the lazy frog.



PDU (Protocol Data Unit):

- UD = { je ein Buchstabe der Nachricht }
- PCI = { Sequenznummer: 1 Ziffer pro PDU }

Beispiel: Sequenznummern

Nachricht

The quick brown fox jumps over the lazy frog.

0	T
1	h
2	e
3	␣
4	q
5	u
6	i

⋮



5?

0,1,2,3,4,5,6

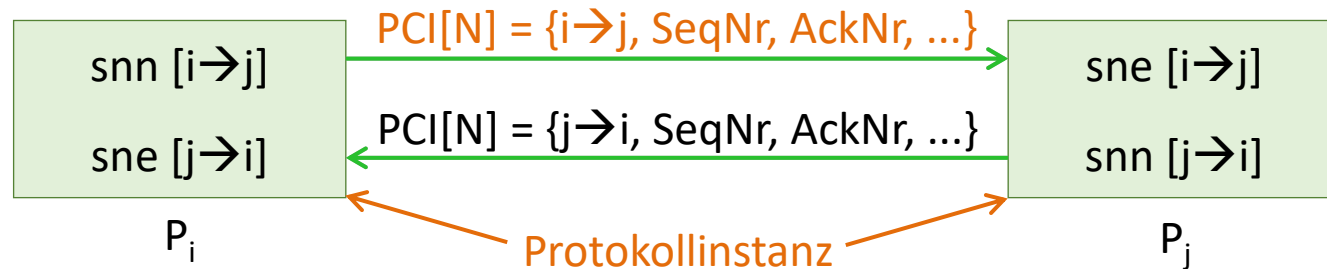


6



Quittung

Trace: Sequenznummern



Zeitschritt	Protokoll- instanz	Komm.- richtung	UD	snn $[i \rightarrow j]$	PCI[N] {SeqNr}	PCI[N] {AckNr}
t = 1	P_i	$i \rightarrow j$	T	0	0	7*
t = 2	P_j	$j \rightarrow i$			7	1
t = 3	P_i	$i \rightarrow j$	h	1	1	8
t = 4	P_j	$j \rightarrow i$			8	2
t = 5	P_i	$i \rightarrow j$	e	2	2	9

* beliebig

Optimierungen für höhere Leistung

Im Beispiel:

- Jedes Zeichen wird einzeln versendet und bestätigt

Verbesserungen:

- Mehr Daten in den UD (im Beispiel nur 1 Zeichen)
- Senden von mehreren PDUs während gleichzeitig auf den Empfang der Quittungen gewartet wird
- Sammelquittungen oder selektive Quittungen für eine Untermenge der PDUs

Weitere Fragestellungen

- Wie viele Sequenznummern gibt es eigentlich?
Ist die Anzahl der Sequenznummern begrenzt?
 - ➔ Bestimmung der Größe des Sequenznummernraumes
- Wie lange können Nachrichten im Netz unterwegs sein? Wie lange „leben“ Nachrichten?
 - ➔ Maximale Lebensdauer von Nachrichten

Größe des Sequenznummernraumes (1/2)

- Sequenznummernräume sind **endlich** (und haben meist eine feste Größe – festgelegt im Protokoll).
- Im Beispiel:
 - 10 unterschiedliche Sequenznummern (0,1,...,9).
 - Nachricht 11 verwendet eine Sequenznummer (=0), die bereits früher einmal verwendet wurde.
 - **Gefahr: verzögerte Nachricht** mit derselben Sequenznummer (=0) ist noch im Netz unterwegs
 - Würde die verzögerte Nachricht vor Nachricht 11 ankommen:
 - Empfänger interpretiert die falsche Nachricht als erwartete Nachricht 11 und
 - verwirft danach ankommende Nachricht 11 als Duplikat

Größe des Sequenznummernraumes (2/2)

- *Wie lange kann eine Nachricht im Netz unterwegs sein?*
- **Gesucht:** Mechanismus, um die Dauer der maximalen Verzögerung einer Nachricht zu bestimmen
- Wähle Sequenznummernraum so, dass innerhalb der maximalen Verzögerung nie dieselbe Sequenznummer doppelt verwendet werden muss
- **Möglichkeit:** Sequenznummernraum $>$ Senderate $\times$ RTD (engl.: Round Trip Delay).
- Beispiel:
 - Sei RTD = 15ms, Senderate 200/ms
 - Benötigt: 12 Bits zur Codierung der Sequenznummern

Maximale Lebensdauer von Nachrichten

- Begrenzung der Lebensdauer verzögerter Nachrichten auf ein Maximum:
 - alle Nachrichten erhalten einen **Zeitstempel** und eine **Lebensdauer**
 - zu alte Nachrichten werden einfach verworfen
- Erfordert aufwändige Schätzung oder bestenfalls synchronisierte Uhren (=> Leslie Lamport)
- Andere Möglichkeit: Zählung der Hops (“Sprünge“)

Verbindungsorientierter Dienst

Techniken:

- Verbindungsauf-/abbau ✓
- Sequenznummern und Quittungen ✓
- Flusssteuerung und Staukontrolle

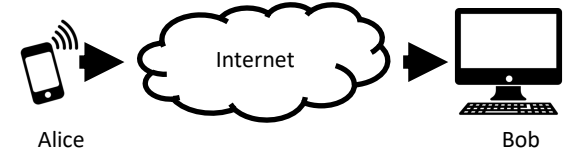
Verbindungsorientierter Dienst (connection-oriented service)

- Aufbau einer logischen Verbindung (Sitzung) ✓
- Strategien zur Fehlerbehandlung:
 - Kein Verlust von Nachrichten ✓
 - Keine Verdoppelung von Nachrichten ✓
 - Keine Verfälschung der Inhalte von Nachrichten
 - Keine Vertauschung der Reihenfolge mehrere Nachrichten ✓
- Verbindungsmanagement durch Flusssteuerung und Staukontrolle zur Handhabung von Verzögerungen bei der Nachrichtenübertragung

Zusammenfassung

- Verbindungsorientierte Protokolle werden oft mit Sequenznummern & Quittungen realisiert
- Gesendete Nachrichten werden durchnummeriert
 - Verloren gegangene Nachrichten werden neu angefordert
 - Duplikate werden verworfen
 - Nachrichten in falscher Reihenfolge werden neu angeordnet

Datenfluss beim Sender

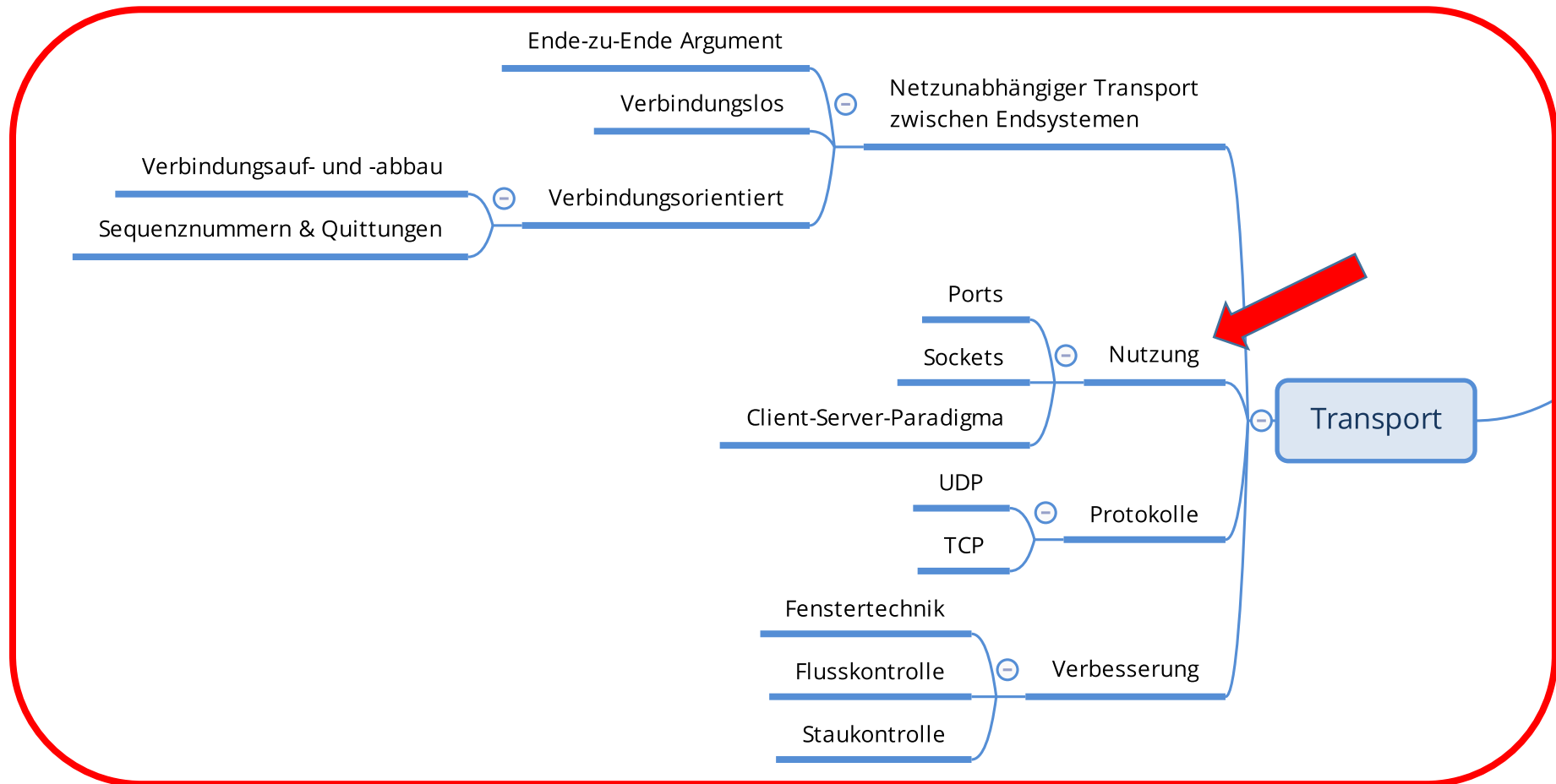


-
- Transport der Nachricht
 - Aufbau und Abbau der Verbindung
 - Mit/ohne Zustellgarantie
 - Über unsichere Kanäle
 - Mit Verkehrsmanagement (Stau, ...)
-

 Nachricht,
Ziel

 Daten,
Verb.Info

RNVS – Kapitel 3



Nutzung des Transportdienstes



Grundlagen der Programmierung:

- Adressierung des Kommunikationsendpunkts
- Schnittstellenbeschreibung
- Programmiermodell

Adressierung des Kommunikationsendpunkts



Transportschicht liefert aus Sicht der Endsysteme eine virtuelle, zuverlässige Ende-zu-Ende Verbindung

Fragen:

- Gibt es am Endsystem B (=Empfänger) mehr als einen Dienst?
- Wenn ja, wie wählt man den passenden Dienst am Endsystem B aus?



Ports zur Adressierung des Kommunikationsendpunkts

Ports:

- TSAP – engl. Transport Service Access Point
- Adressen von Kommunikationsendpunkten einer Protokollinstanz der Transportschicht auf einem gegebenen Host

Ziel:

- Identifikation von Diensten bzw. Protokollen
- Unterscheidung mehrerer Verbindungen zwischen demselben Paar an Endpunkten

Eigenschaften von Ports

- Flacher Adressraum (16-Bit): 0-65535 ($=2^{16}$)
- Portnummern von der IANA (engl.: Internet Assigned Numbers Authority) verwaltet.

Zuordnung lt. RFC 6335:

- *Well Known* Ports (0-1023)
- *Registered* Ports (1024-49151): meist herstellerspezifisch
- *Dynamic and/or Private* Ports (49152-65535): frei nutzbar

Beispiele für Well Known Ports

21	FTP
22	Secure Shell (SSH)
23	telnet
25	SMTP
80	HTTP
143	IMAP

Beispiele für Registered Ports

2628	DICT
2481	Oracle GIOP
6000	X Windows



Nutzung des Transportdienstes



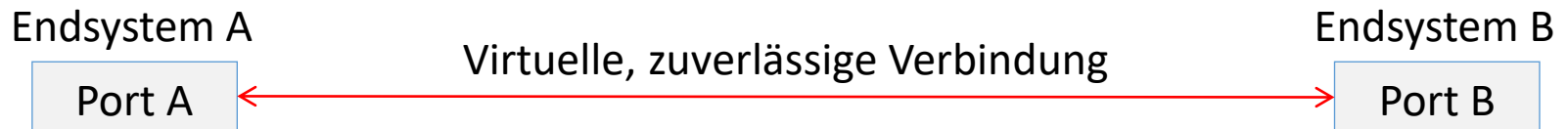
Grundlagen der Programmierung:

- Adressierung des Komm.-endpunkts - Ports ✓
- Schnittstellenbeschreibung
- Programmiermodell

Schnittstellen von Transportdiensten

Sockets:

- plattformunabhängige, standardisierte Schnittstelle (API) für netzglobalen Kommunikationsendpunkt
- für Zugriff auf Transportdienste

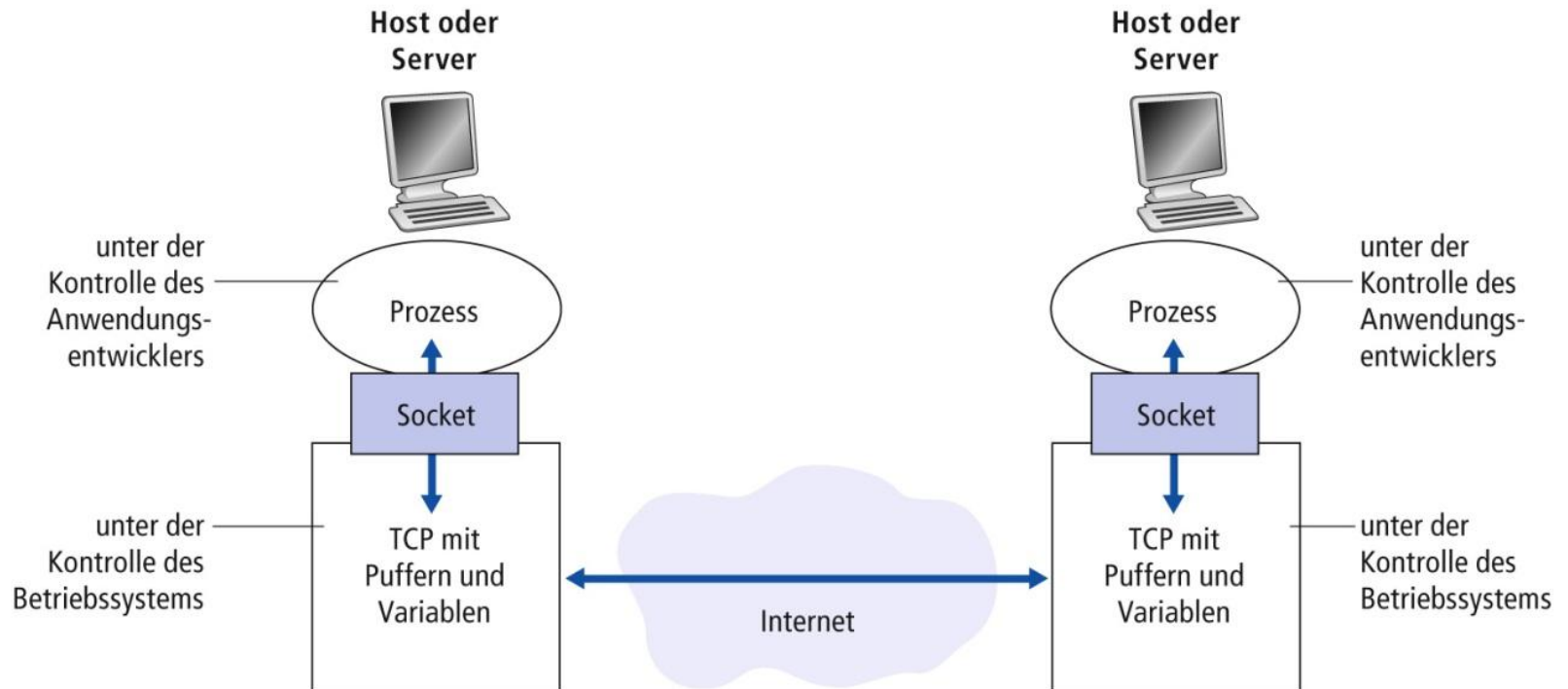


- Verbindung zwischen zwei Sockets

Eigenschaften von Sockets

- vom Betriebssystem verwaltet
- Spezifikation in BSD 4.1 UNIX (1981)
- Umgang mit Sockets ähnlich wie mit einer Datei
 - *„Everything is a file in UNIX“*
https://de.wikipedia.org/wiki/Everything_is_a_file
 - **Open:** Socket wird erzeugt / geöffnet
 - **Read/Write:** Es wird vom Socket gelesen / zum Socket geschrieben
 - **Close:** Socket wird geschlossen / zerstört

Sockets



*Analog für UDP

Eigenschaften von Sockets

- Bidirektional – können senden und empfangen
- Arten von Internet-Sockets:
 - Datagramm Sockets: Kommunikation mit Datagrammen
 - Stream Sockets: Kommunikation mit Zeichenströmen

- Tupel aus Verbindungsendpunkten

[Quelle — Senke]

- Verbindungsendpunkt: Tupel aus IP-Adresse und Port:

[(IP-Adresse, Port) — (IP-Adresse, Port)]

Vermittlungsschicht ↑

↑ Transportschicht

- Verbindung immer zwischen zwei Sockets

Nutzung des Transportdienstes



Grundlagen der Programmierung:

- Adressierung des Komm.-endpunkts - Ports ✓
- Schnittstellenbeschreibung - Sockets ✓
- Programmiermodell

Client-Server-Paradigma

- Server Socket:
 - wartet auf Verbindungsanfragen von Clients
- Normaler Socket:
 - wird nach dem Verbindungsaufbau von Client- sowie Serverprozessen zur Kommunikation genutzt.
- Server/Daemon:
 - erzeugt neue Threads/Prozesse zur Abwicklung der Kommunikation nach dem Verbindungsaufbau.

Nutzung des Transportdienstes

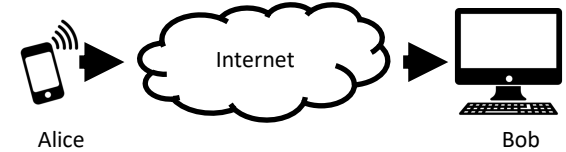


Grundlagen der Programmierung:

- Adressierung des Komm.-endpunkts - Ports ✓
- Schnittstellenbeschreibung - Sockets ✓
- Programmiermodell – Client-Server ✓

Datenfluss beim Sender

- Transport der Nachricht
 - Aufbau und Abbau der Verbindung
 - Mit/ohne Zustellgarantie
 - Über unsichere Kanäle
 - Mit Verkehrsmanagement (Stau, ...)



↓ Nachricht,
Ziel

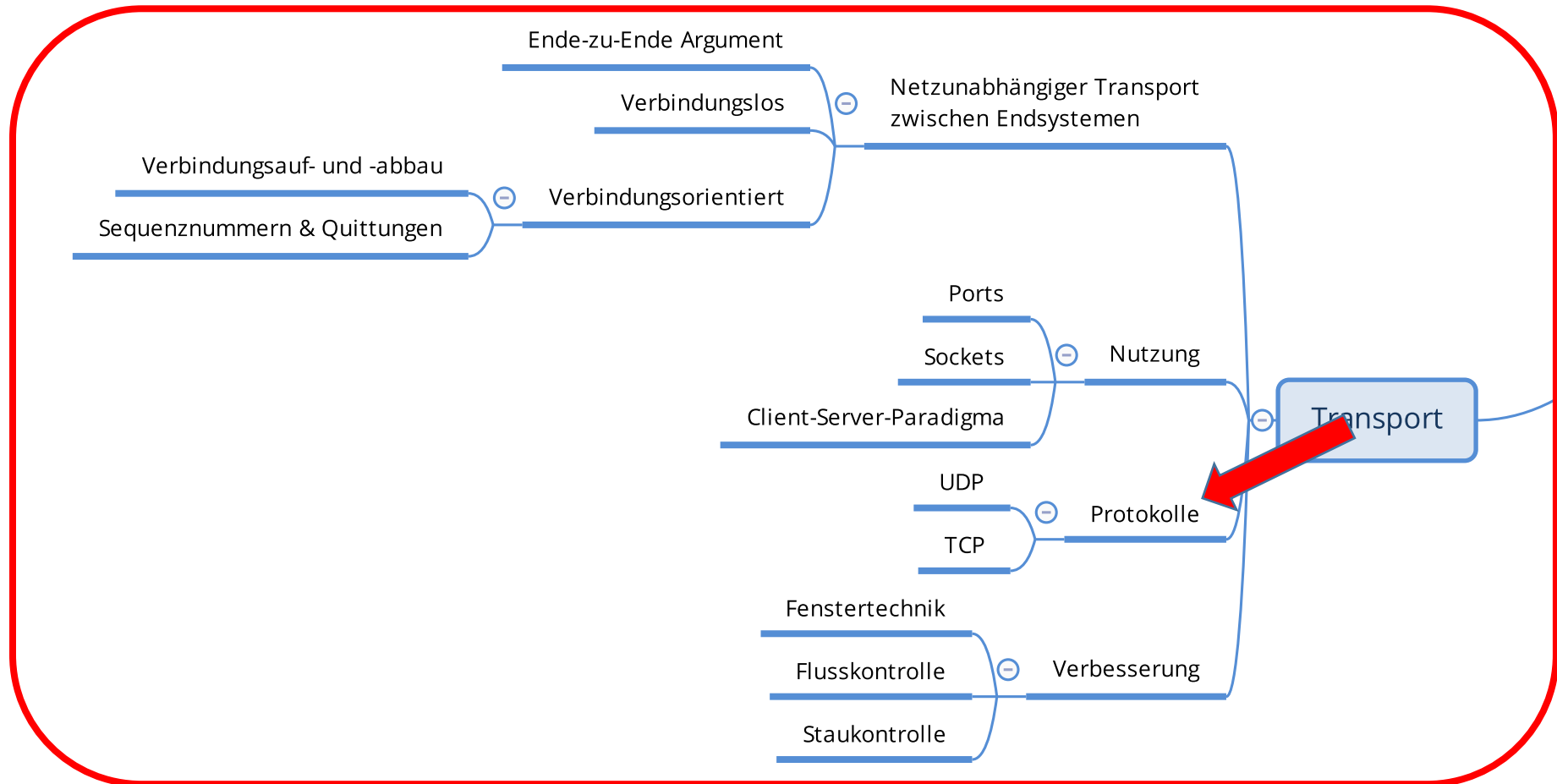
↓ Daten,
Verb.Info
=
Socket

Zusammenfassung

Nutzung des Transportdienstes:

- **Port:** Adressierung des Kommunikationsendpunkts
 - Well-known / Registered - für festgelegte Dienste
 - Dynamic / Private – frei nutzbar
- **Socket:** Schnittstelle zur Programmierung
 - Zugriff wie Dateizugriff (open, read, write, close)
- **Client-Server:** Programmiermodell

RNVS – Kapitel 3



Transportschicht

Verbindungsloser Dienst:

- UDP – User Datagram Protocol
 - Unzuverlässig aber schnell

Verbindungsorientierter Dienst:

- TCP – Transmission Control Protocol
 - Zuverlässig aber langsam

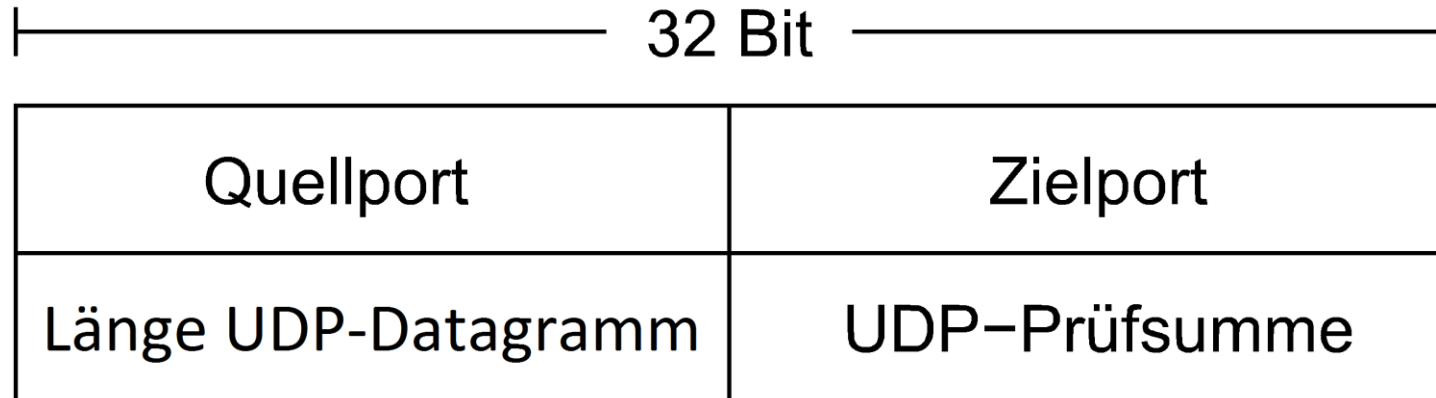
UDP – User Datagram Protocol

Grundlagen von UDP

Verbindungsloses (unzuverlässiges) Internet Transportprotokoll (RFC 768)

- kein Verbindungsaufbau, -abbau, oder -status
- keine Sequenznummern
 - ➔ keine Reihenfolgesicherung, Duplikats- oder Verlusterkennung
- kein Verkehrsmanagement - keine Regulierung der Senderate

UDP Header (PCI)



- **Quellport/Zielpport** (je 16 Bit): zur Adressierung
- **Datagrammlänge** (16 Bit): Maximallänge von 65515 Bytes (bei Verwendung von IPv4)
- **Prüfsumme** (16 Bit): (optionale) Prüfsumme über IP-Pseudoheader und UDP-Datagramm

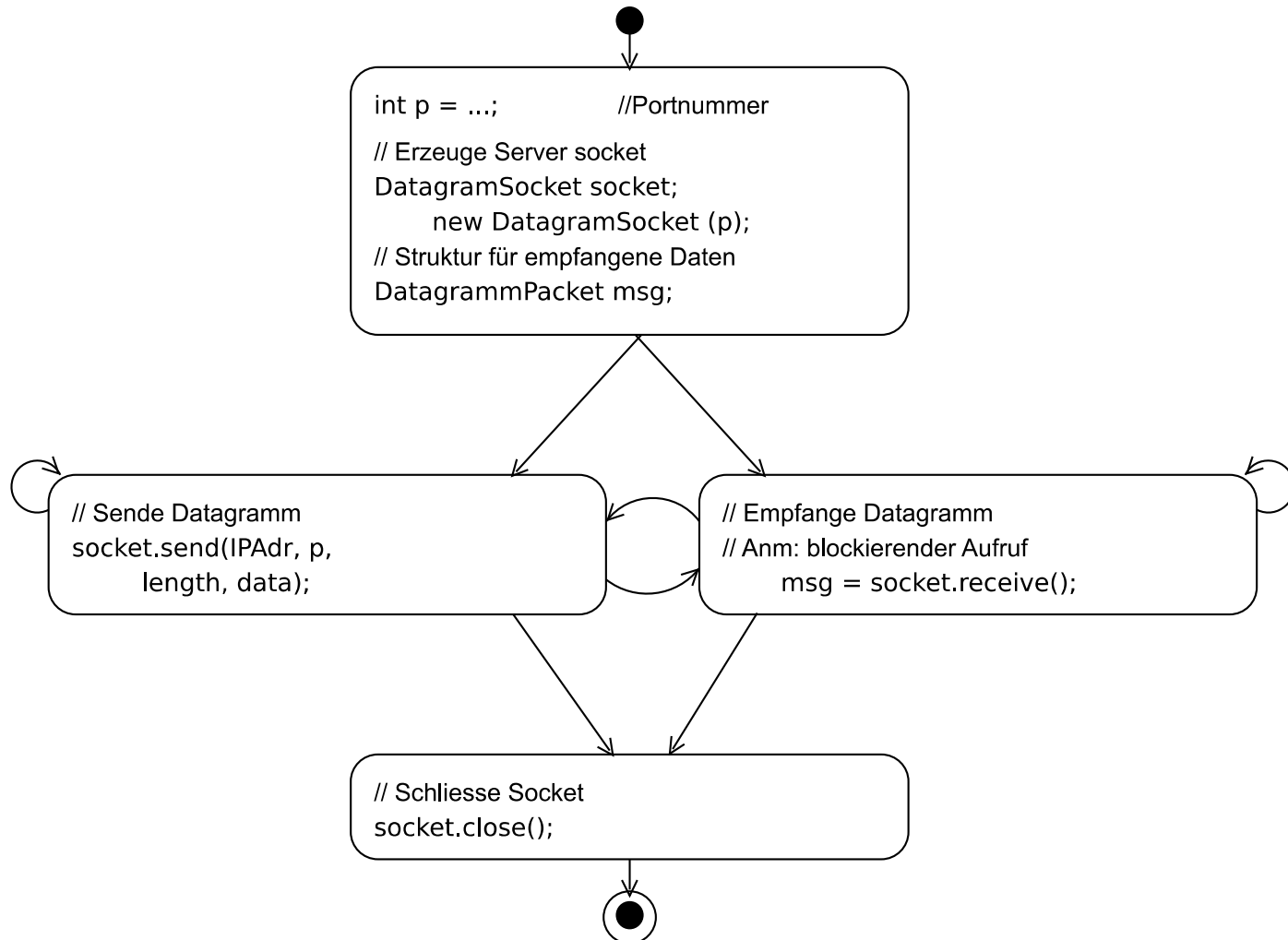
UDP Eigenschaften

- $2 \times 32 \text{ Bit} = 64 \text{ Bit} = 8 \text{ Byte Header}$
(vgl. TCP mit mindestens 20 Byte)
- UDP-Ports: Unterstützten Multiplexen von Anwendungen
- UDP-Sockets: zur Adressierung von Anwendungen
- Protokolle die UDP verwenden:
 - TFTP (Trivial File Transfer Protocol)
 - DNS (Domain Name System)
 - RPC (Remote Procedure Call)
 - SNMP (Simple Network Management Protocol)

Verwendung von UDP

- Programmierung mit UDP Sockets
 - Senden/Empfangen isolierter Datagramme
 - Sicht auf lokales Socket
 - Senden/Empfangen zu/von mehreren Hosts (IP-Adresse und Port des Empfängers werden i.d.R. beim Senden jedes einzelnen Datagramms angegeben)
- Reihenfolgesicherung, Verlust- und Duplikatsbehandlung, sowie Steuerung der Senderate obliegen Anwendungsprogramm.

UDP-Socket-Programm



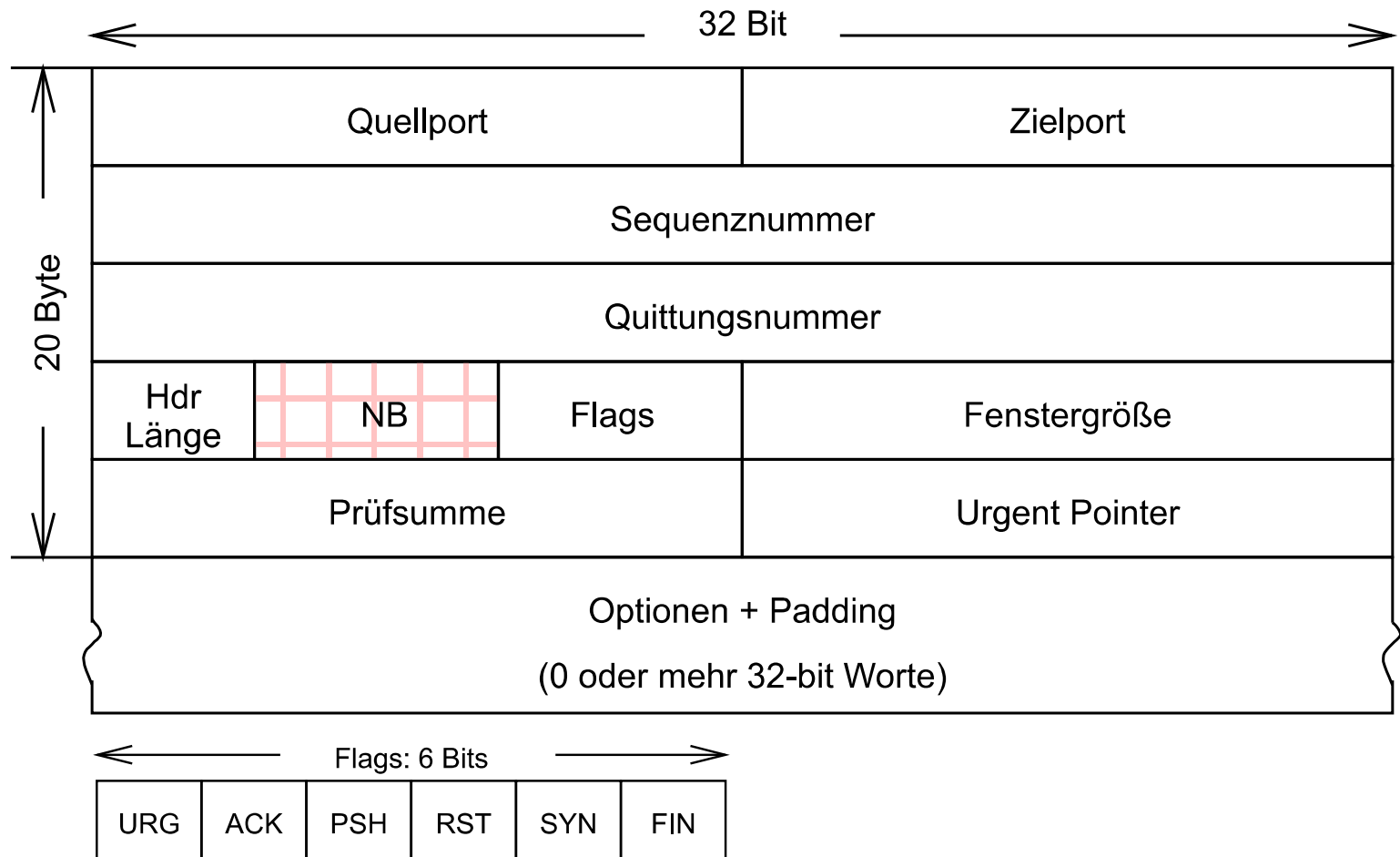
TCP – Transmission Control Protocol

Grundlagen TCP

- Verbindungsorientiertes (zuverlässiges) Internet Transportprotokoll (RFC 793)
- unterstützt Ende-zu-Ende-Transportverbindungen
 - Vollduplex (gleichzeitig Senden/Empfangen in beide Richtungen)
 - mit Fehlerbehandlung (Verlust, Duplikate, usw.)
 - mit Flusssteuerung und Staukontrolle
- Kommunikationsaufbau per Drei-Wege-Handschlag
- Byte-Strom-orientiert: Sequenz- und Quittungsnummern beziehen sich auf Bytes.

TCP Header (PCI)

(1/4)



TCP Header (PCI)

(2/4)

- **Quellport/Zielpport** (je 16 Bit): Endpunkte der Verbindung
- **Sequenznummer** (32 Bit): Bytestromnummer des ersten Nutzdaten-Bytes des Segments.
- **Quittungsnummer** (32 Bit): Bytestromnummer des nächsten erwarteten Nutzdaten-Bytes – nur lückenlos fehlerfrei empfangene Daten werden quittiert.
- **Header Länge** (4 Bit): Anzahl der 32 Bit Wörter im Header.

TCP Header (PCI)

(3/4)

- **NB** (6 Bit): werden nicht benutzt.
- **Flags** (je 1 Bit):
 - URG: *Urgent Pointer* verwendet (Interrupt Data)
 - ACK: *Acknowledge* (Quittungsnummer gültig)
 - PSH: *Push Data* (nicht puffern, sondern sofort zustellen)
 - RST: *Reset* (abgestürzt, Aufbauwunsch abgelehnt, ...)
 - SYN: *Synchronize* (Verbindungsaufbau)
 - FIN: *Finalize* (Abbauwunsch/keine weiteren Daten)
- **Fenstergröße** (16 Bit): Anzahl der Bytes, die ab letzter Quittung gesendet werden dürfen.

TCP Header (PCI)

(4/4)

- **Prüfsumme** (16 Bit): Einerkomplement der Summe aller 16-Bit-Worte über Pseudoheader und TCP-Segment (Der Pseudoheader enthält die folgenden Felder aus dem IP-Header: Quell- und Zieladresse, Protokoll, und Länge des TCP-Segments).
- **Urgent Pointer** (16 Bit): Zeigt auf erstes Byte nach einer Kette von *dringenden* Daten (engl.: out-of-band data).
- **Optionen** ($n * 32$ Bit): Wählbare Eigenschaften, wie z.B. maximale Segmentgröße und Timestamp-Option.

Handschlag-Protokoll – Realisierung mit TCP

Bisher \ TCP	URG	ACK	PSH	RST	SYN	FIN
SYN					X	
SYNB		X			X	
SYNBP		X				
SYNBN				X		
CLS						X
CLSB		X				
ACK		X				
NAK		X*				

*Wiederholen einer alten Quittung: Quittungsduplikate

Verbindungsauf-/abbau mit Sequenznummern

- **Frage:** Wie realisiert TCP den Drei-Wege-Handschlag mit Sequenznummern?
- **Im Speziellen:** Synchronisation am Beginn einer Kommunikation:

Wie wird einem Kommunikationspartner bei einer neuen Verbindung die erste verwendete Sequenznummer mitgeteilt?

- **Problem:** auf Empfängerseite müssen verzögerte Duplikate alter Aufbauwünsche von aktuellen Aufbauwünschen unterschieden werden

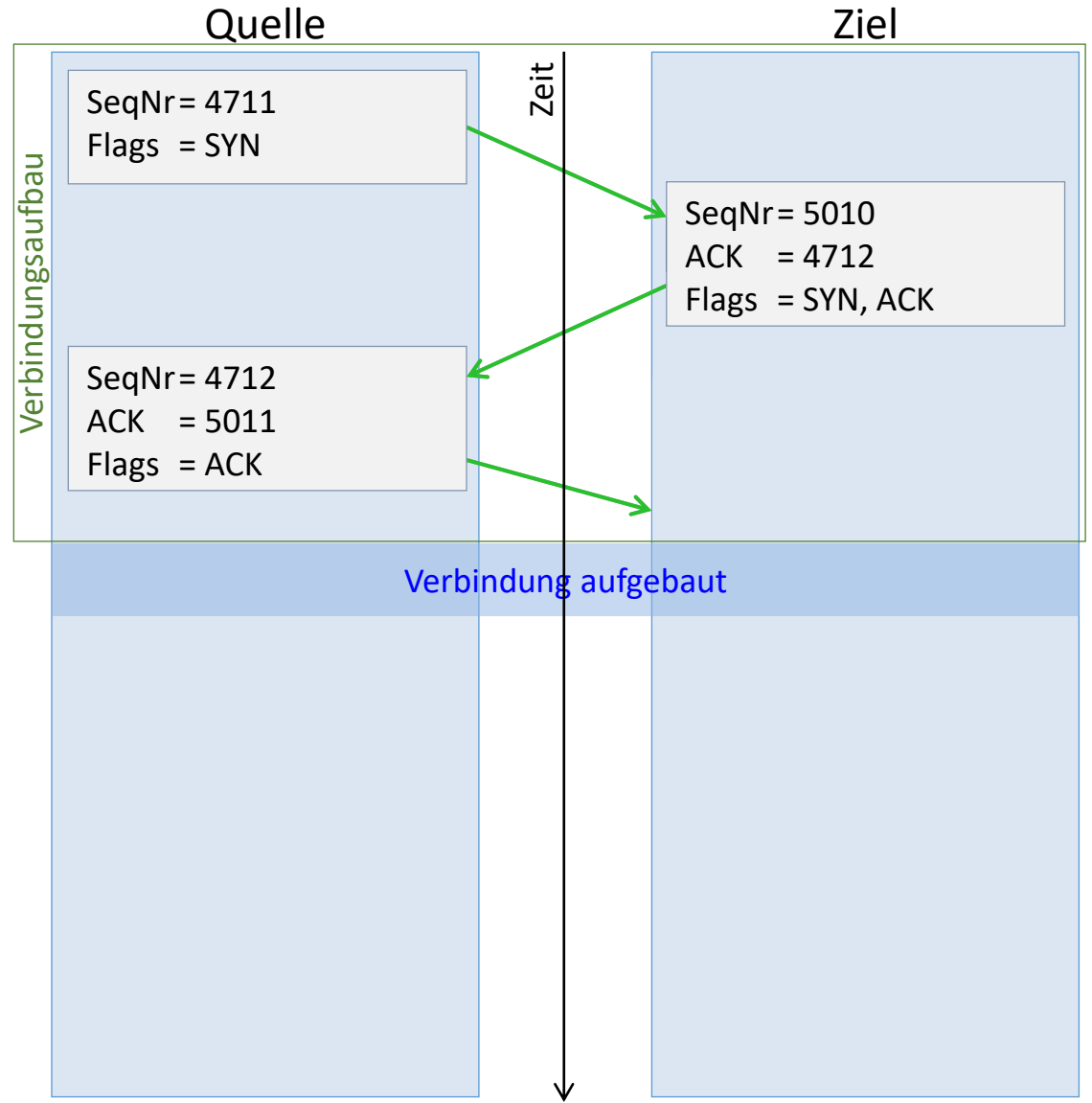
Verbindungsaufbau: Drei-Wege-Handschlag

Initiale Sequenznummern

- gewählt aufgrund Zeitgeber
- Eindeutigkeit bei oft auf-/abgebauten Verbindungen

Sequenznummern werden erhöht aufgrund von

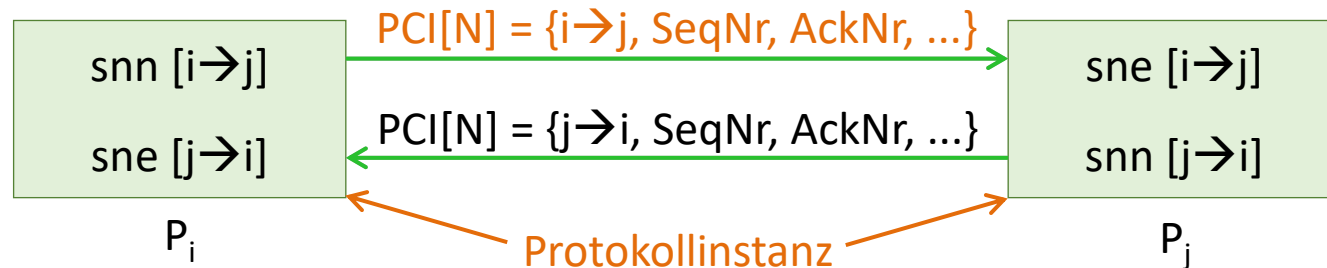
- SYN und FIN (Schutz des Verbindungsmanagement, jeweils +1)



Drei-Wege-Handschlag mit Sequenznummern

- Erste Nachricht (SYN) enthält erste Sequenznummer für Kommunikation von $P_i \rightarrow P_j$
- Antwort (SYN,ACK) bestätigt Sequenznummer und enthält Retoursequenznummer für den Verkehr von $P_j \rightarrow P_i$.
- Bestätigung (ACK) bestätigt Retoursequenznummer (und kann je nach Implementierung auch bereits erste Daten enthalten).
- Verzögerte Duplikate alter SYN Nachrichten können anhand der verwendeten Sequenznummern erkannt werden

Notation: Sequenznummern



- $snn [i \rightarrow j]$: SeqNr des nächsten zu sendenden **Bytes**
- $sne [i \rightarrow j]$: SeqNr des nächsten erwarteten **Bytes**
- $PCI[N] = \{i \rightarrow j, SeqNr, AckNr, \dots\} =$
Steuerinformationen
 - $i \rightarrow j$: Verbindungs- und Richtungskennung
 - $SeqNr$: SeqNr der Nachricht (erstes Byte der Nachricht)
 - $AckNr$: SeqNr der nächsten erwarteten Nachricht (= sne)

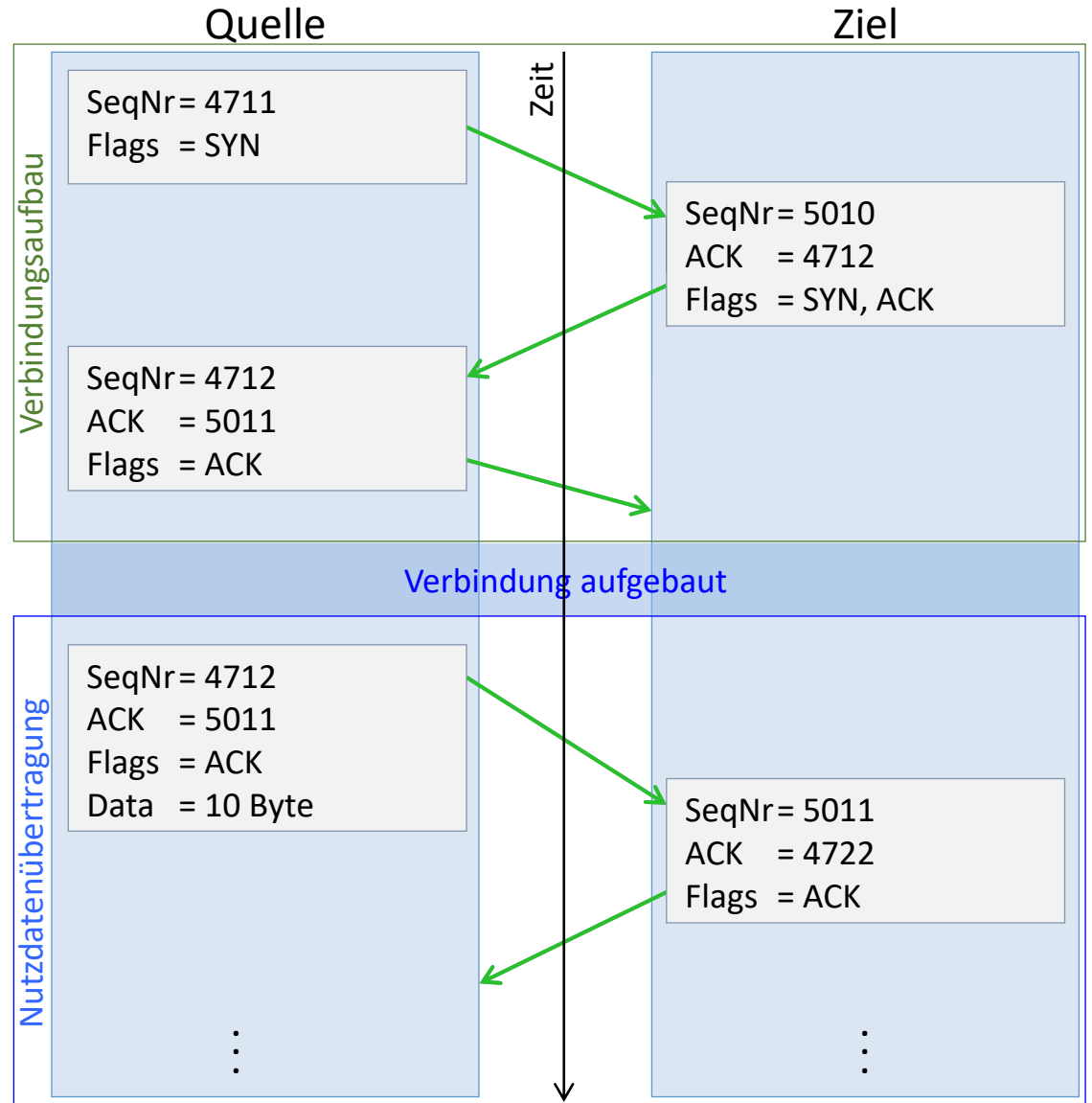
Verbindungsaufbau: Drei-Wege-Handschlag

Initiale Sequenznummern

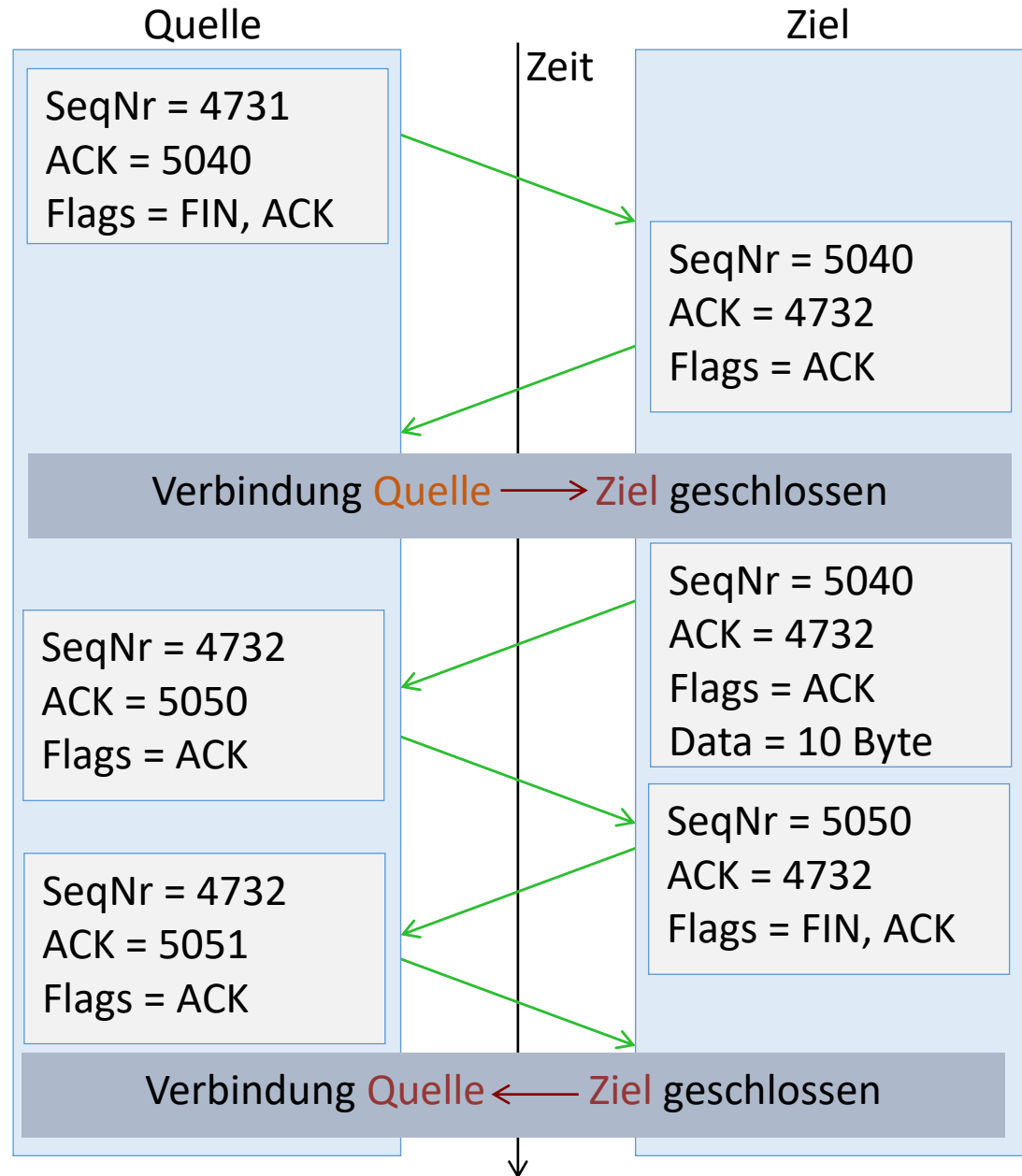
- gewählt aufgrund Zeitgeber
- Eindeutigkeit bei oft auf-/abgebauten Verbindungen

Sequenznummern werden erhöht aufgrund von

- SYN und FIN (Schutz des Verbindungsmanagement, jeweils +1)
- Segmenten mit Nutzdaten (um 1 pro Byte Nutzdaten)
- sonst: „leere“ Segmente -> „alte“ Seq. Nr. (z.B. Quittungen)



Beidseitiger Verbindungsabbau



TCP Eigenschaften

- TCP-PDU wird „**Segment**“ genannt
- TCP-Ports: Erlaubt Multiplexen von Anwendungen
- TCP-Sockets: zur Adressierung von Anwendungen
- Puffern: Dienstdaten werden zwischengespeichert
- Protokolle die TCP verwenden:
 - HTTP (Hypertext Transfer Protocol)
 - SMTP (Simple Mail Transfer Protocol)
 - IMAP (Internet Message Access Protocol)
 - FTP (File Transfer Protocol)

TCP-Socket-Programm

Clientprozess auf Host A

```
int p = ...; //bekannte Portnr.
```

```
//Erzeuge Socket und verbinde  
//es mit Host B auf Port p  
clientSocket = new Socket(B,p);
```

```
//Sende Anfrage an Server  
clientSocket.write();
```

```
//Lese Antwort von Server  
clientSocket.read();
```

```
clientSocket.close();
```

Serverprozess auf Host B

```
int p = ...; //Portnummer  
ServerSocket welcomeSocket;  
Socket connectionSocket;  
// Erzeuge Server socket  
welcomeSocket =  
    new ServerSocket (p);
```

```
// Warte auf Verbindung von  
Client  
// Anm: blockierender Aufruf  
connectionSocket =  
welcomeSocket.accept();
```

```
// Lese Anfrage von Client  
connectionSocket.read();
```

```
// Sende Antwort  
connectionSocket.write();
```

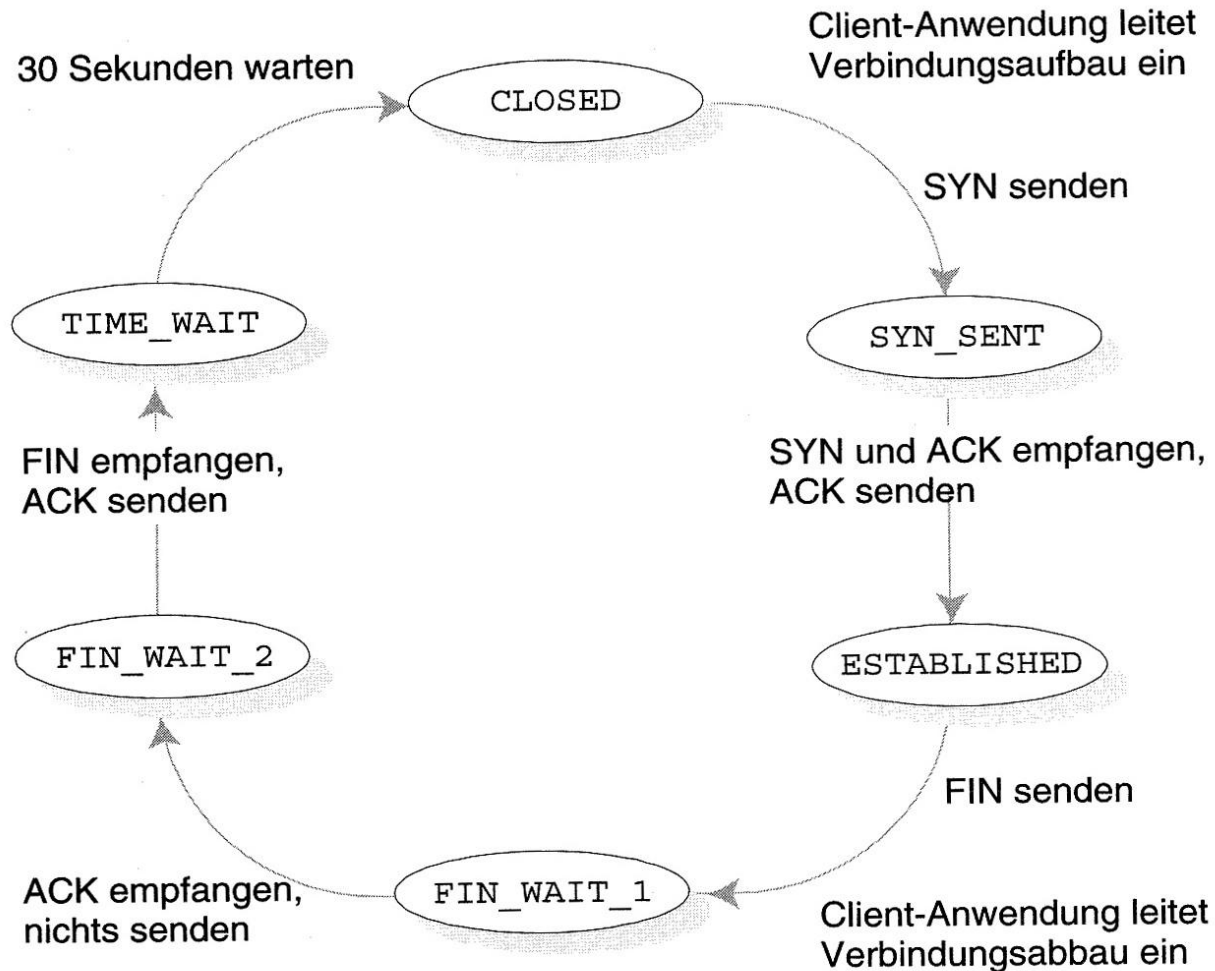
```
// Schliesse Verbindung  
connectionSocket.close();
```

Verbindungsaufbau

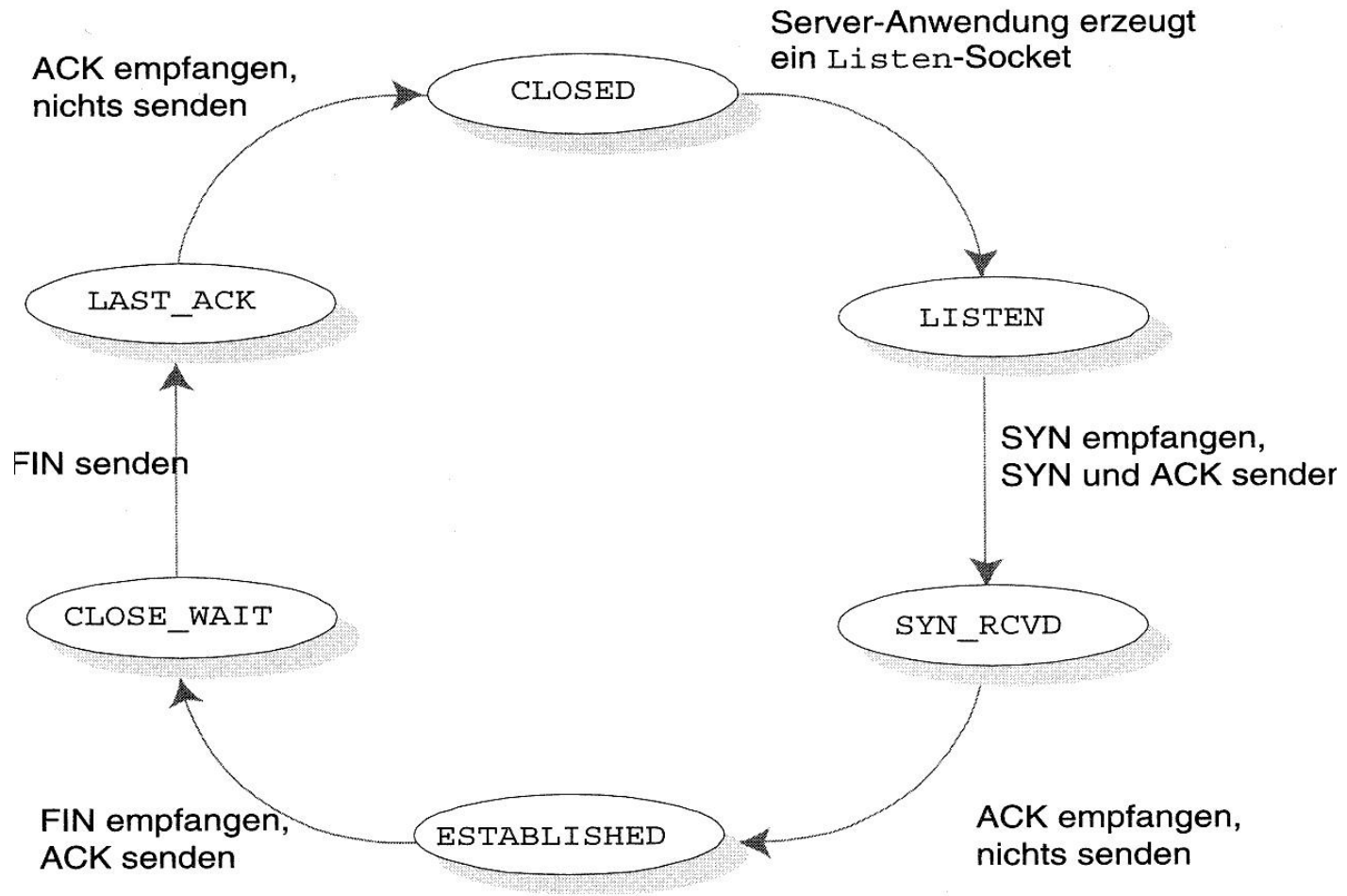
Daten

Daten

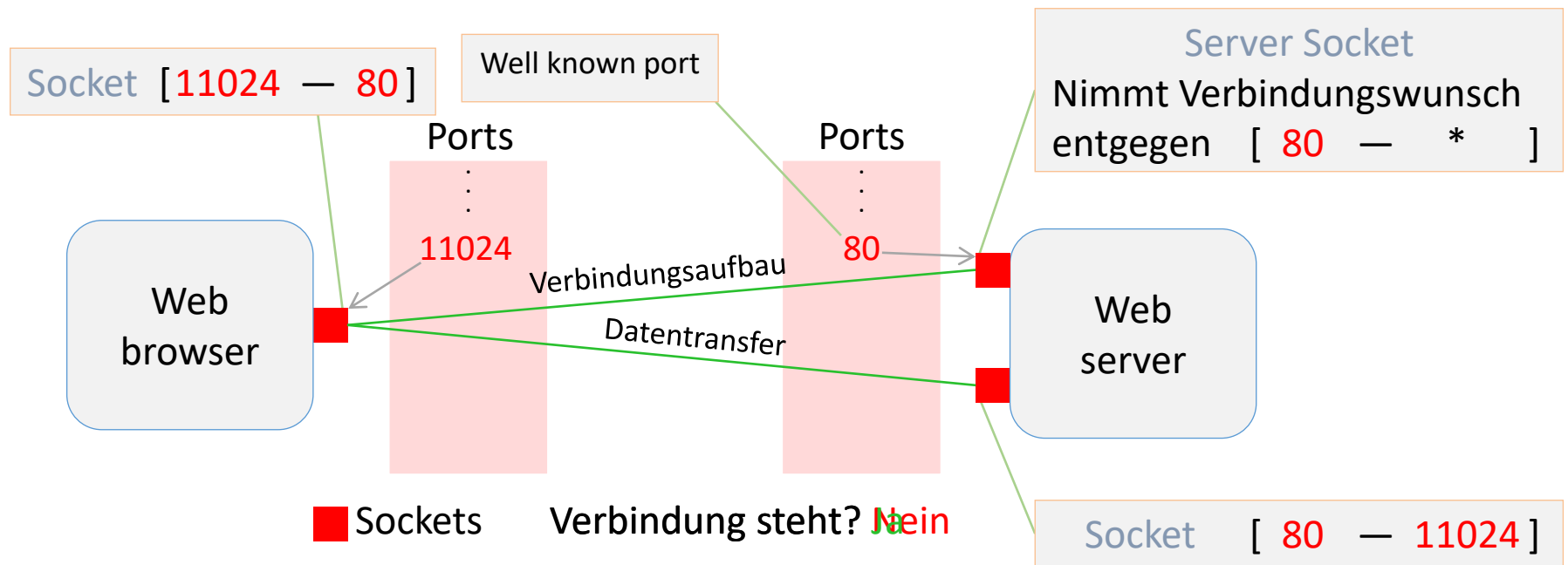
Typische TCP-Zustände im Client



Typische TCP-Zustände im Server



Beispiel: Verbindungssocket



Nutzung von Sockets

- Ende-zu-Ende-Verbindung an Dienst bzw. Anwendung gebunden
- Socket: Endpunkt für Kommunikation
 - Tupel aus Verbindungsendpunkten (jeweils IP-Adresse und Portnummer)
 - erzeugt von / „gehört“ einem Prozess
- Verbindung gegeben durch ein Socket-Paar

```
webserver:~$ netstat -tulpn
```

Active Internet connections (only servers)

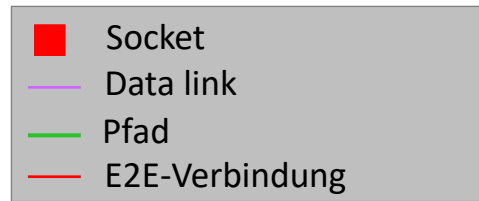
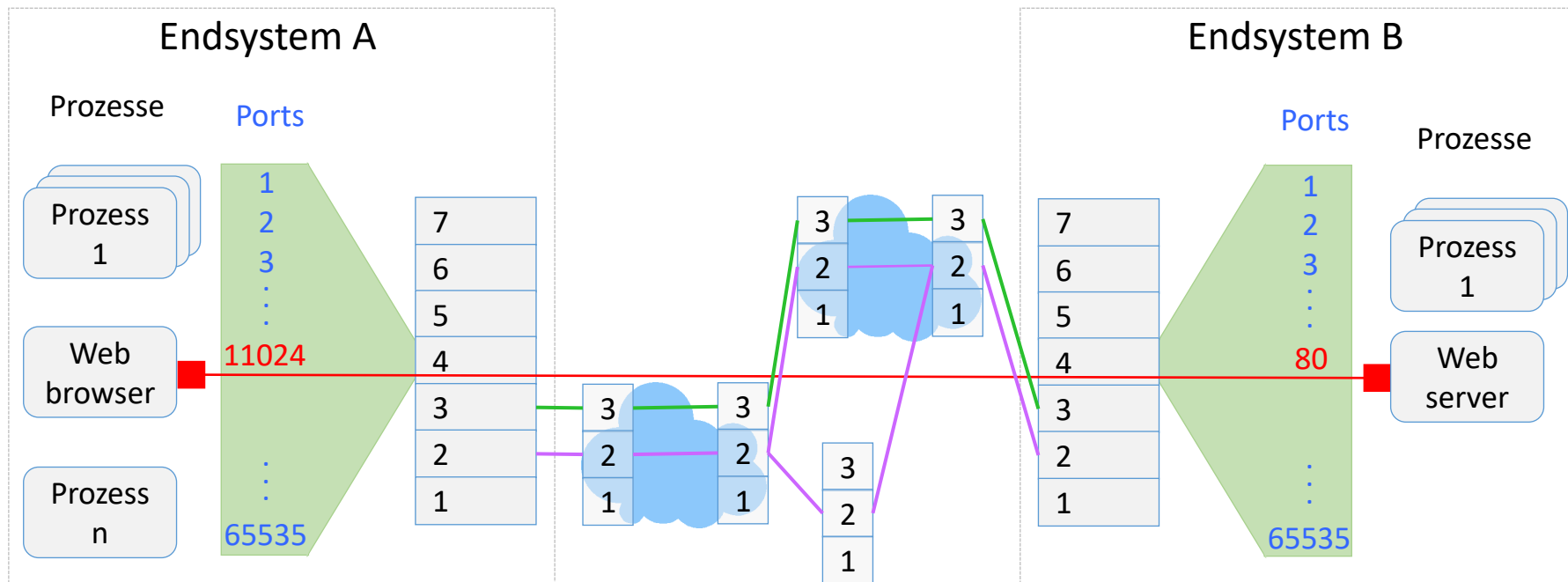
Proto	Local Address	Foreign Address	State	PID/Program name
tcp	127.0.0.53:53	0.0.0.0:*	LISTEN	303/systemd-resolve
tcp	0.0.0.0:34541	0.0.0.0:*	LISTEN	555/rpc.statd
tcp	0.0.0.0:22	0.0.0.0:*	LISTEN	552/sshd: /usr/sbin
tcp	0.0.0.0:80	0.0.0.0:*	LISTEN	628/apache2
tcp	0.0.0.0:443	0.0.0.0:*	LISTEN	628/apache2
tcp	0.0.0.0:5355	0.0.0.0:*	LISTEN	303/systemd-resolve
tcp	0.0.0.0:6556	0.0.0.0:*	LISTEN	627/xinetd
udp	127.0.0.1:731	0.0.0.0:*		555/rpc.statd
udp	127.0.0.53:53	0.0.0.0:*		303/systemd-resolve
udp	0.0.0.0:45154	0.0.0.0:*		555/rpc.statd
udp	0.0.0.0:5355	0.0.0.0:*		303/systemd-resolve

```
webserver:~$ netstat -tupn
```

Active Internet connections (w/o servers)

Proto	Local Address	Foreign Address	State	PID/Program name
tcp	10.163.95.35:685	10.163.95.10:731	TIME_WAIT	-
tcp	10.163.95.35:22	10.163.96.50:60647	ESTABLISHED	46734/sshd: user
tcp	10.163.95.35:1012	10.163.95.32:2049	ESTABLISHED	-
tcp	10.163.95.35:686	10.163.95.10:615	TIME_WAIT	-
tcp	10.163.95.35:443	10.163.96.50:60664	ESTABLISHED	2369/apache2

Multiplexen von Anwendungen



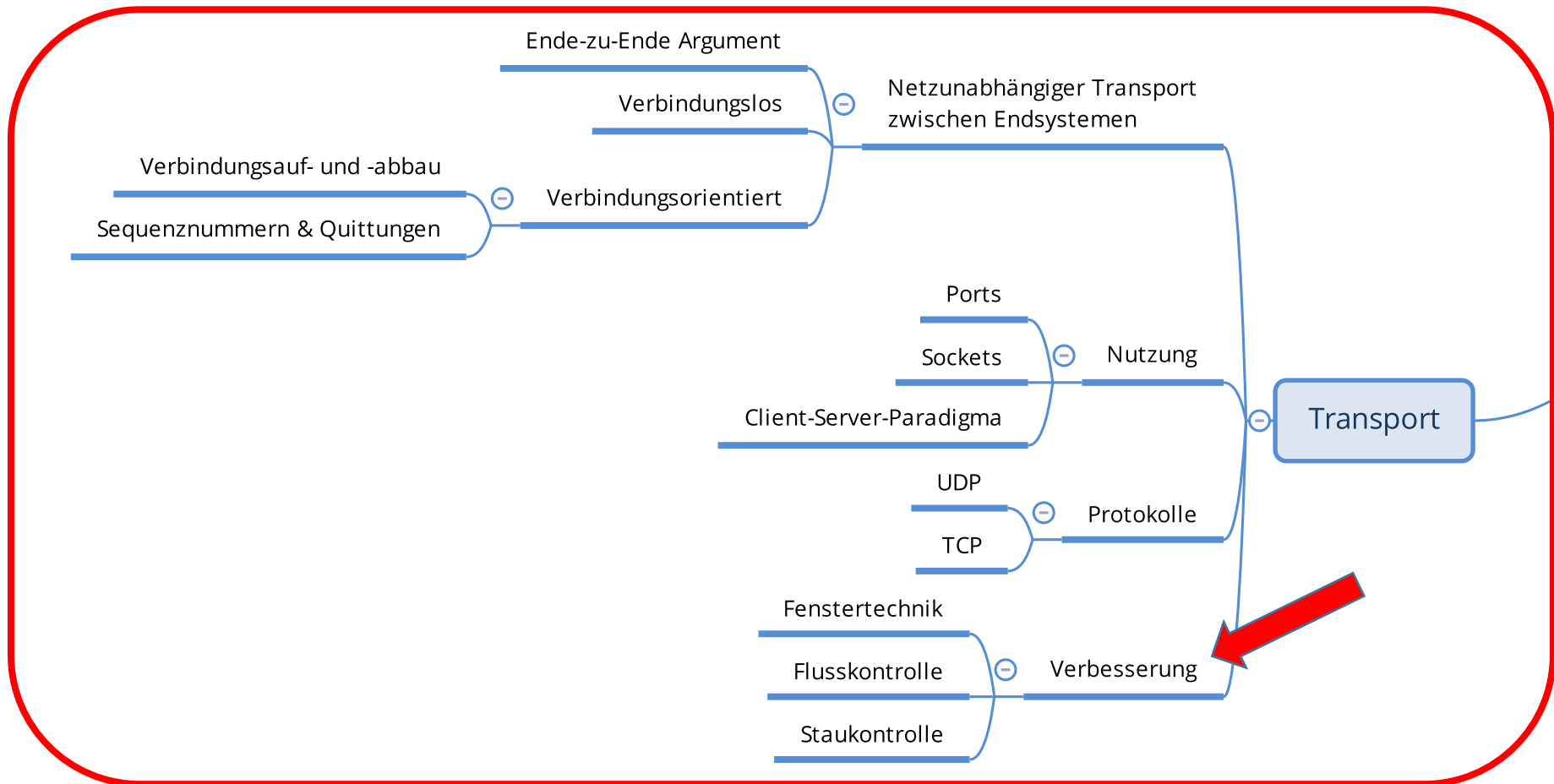
Zusammenfassung

- UDP und TCP sind Protokolle der Transportschicht
- Beide bieten Ende-zu-Ende Kommunikation
- UDP ist verbindungslos, daher unzuverlässig, aber schnell
- TCP ist verbindungsorientiert, daher zuverlässig, aber langsam
- Programmierung in beiden Fällen über Sockets
- Ports erlauben Multiplexen von Anwendungen/Diensten

Ausblick

- UDP mit hoher Kommunikationsleistung, aber unzuverlässig
 - TCP mit Zuverlässigkeitsgarantien und weniger Kommunikationsleistung
 - UDP kann genutzt werden, um (Teile von) TCP nachzubilden
 - *Overhead* kann verbessert werden, wenn nicht alle Features von TCP benötigt werden
- ⇒ QUIC: UDP-Based Multiplexed and Secure Transport
- Designed 2012 von Jim Roskind (Google)
 - Inzwischen standardisiert von der IETF als RFCs (8999-9002, 2021)

RNVS – Kapitel 3



Verbesserungen

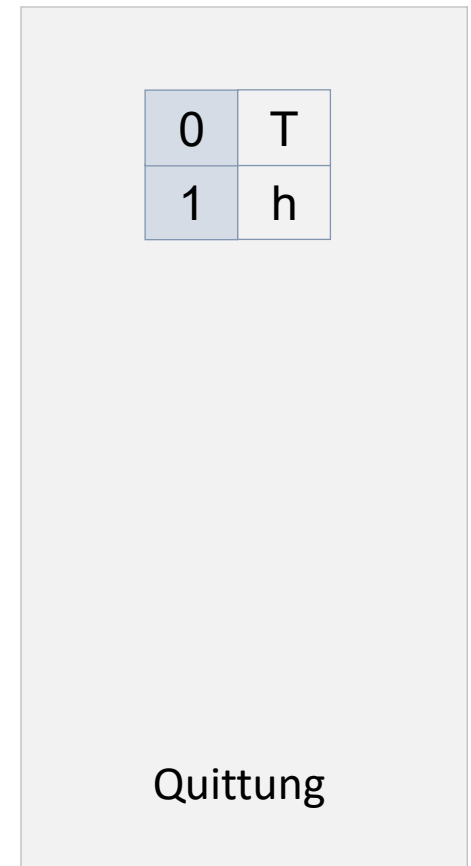
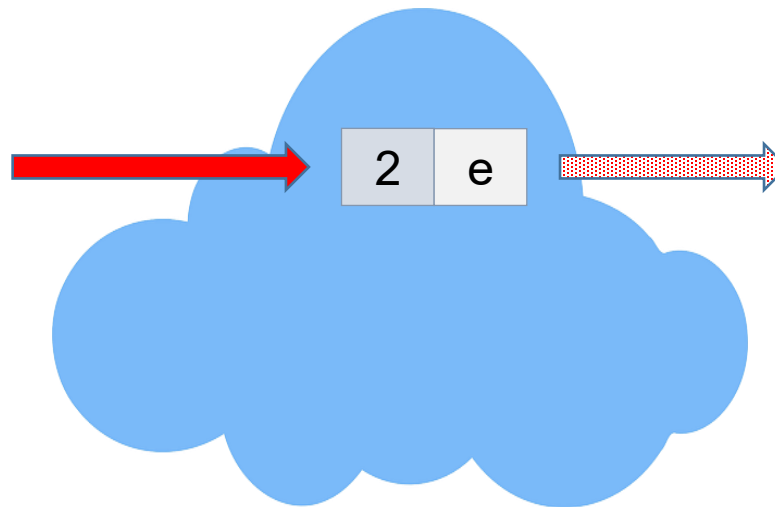
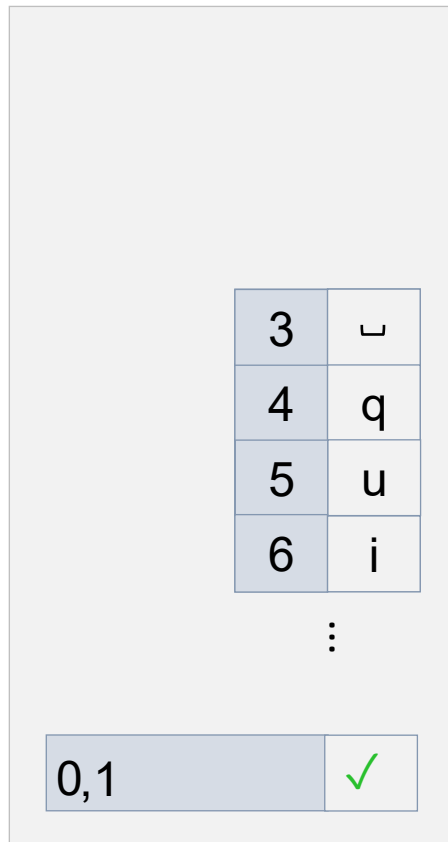
- Kommunikationsleistung
 - Höherer Durchsatz
 - Bessere Auslastung
- Zuverlässigkeit
 - Flusskontrolle
 - Staukontrolle / Überlastkontrolle

Ziel: Höhere Leistung bei geringerer Fehlerrate

Beispiel: Sequenznummern

Nachricht

The quick brown fox jumps over the lazy frog.



Zur Erinnerung

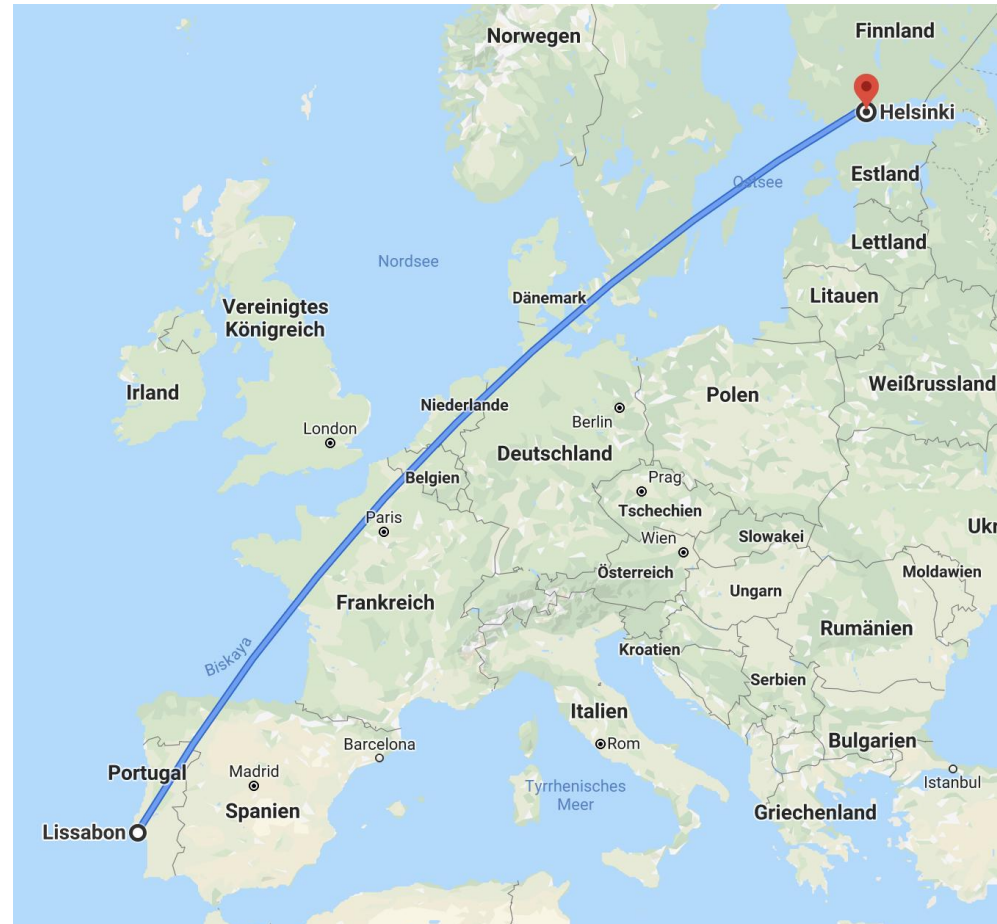
Im Beispiel:

- Es wird jeweils ein Zeichen als Nutzlast übertragen
 - Der Header (=Overhead) ist ein Zeichen (Seq.Nr)
 - Ansatz wird als „Stop-and-Wait“ bezeichnet.
-
- Verbesserung:
 - Mehr als ein Zeichen je Segment
 - Verhältnis Header / Nutzlast dadurch besser

Beispiel: Erreichbarer Durchsatz bei Stop-and-Wait (1/3)

Vgl. S. 242, J. Kurose, K. Ross,
„Computernetzwerke - Der Top-Down-Approach“
2. Auflage, Pearson Deutschland GmbH, 2014

- Nachricht von Lissabon → Helsinki
- Ca. 4400 km Leitung
- Signalausbreitung mit Lichtgeschwindigkeit
- Roundtripdelay (RTD): ca. 30 ms
- 1 Gbit/s Übertragungsgeschwindigkeit



Beispiel: Erreichbarer Durchsatz bei Stop-and-Wait (2/3)

- Nachricht mit ca. 1000 Byte (= 8000 Bit)
- $d_{trans} = \frac{8000 \text{ Bit/Nachricht}}{10^9 \text{ Bit/s}} = 8 \mu\text{s/Nachricht}$
- Wenn die Nachricht bei $t=0$ verschickt wird:
 - $t=8 \mu\text{s}$: das letzte Bit verlässt den Sender
 - $t=15,008 \text{ ms}$: das letzte Bit erreicht den Empfänger
- Annahme: ACK sehr klein, daher ignorieren wir die Übertragungszeit für die Nachricht
- $t=30,008 \text{ ms}$: Bestätigung des Erhalts beim Sender

Beispiel: Erreichbarer Durchsatz bei Stop-and-Wait (3/3)

Problem:

- In 30,008 ms wurde eine Nachricht mit der Länge von 1000 Bytes übertragen. Der Kanal war dabei 0,008 ms lang belegt.
- Auslastung: $U_{Sender} = \frac{0,008}{30,008} = 0,000267$
- Effektiver Durchsatz: 267 kbit/s
- **Ursache:** Stop-and-Wait Protokoll

Photo by [Charles Deluvio](#) on [Unsplash](#)



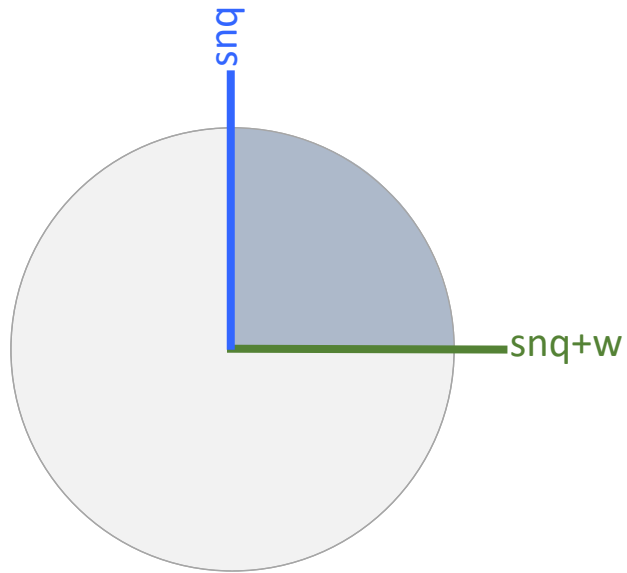
Verbesserungsmöglichkeiten

- Anstelle von Stop-and-Wait:
 - Absender darf mehrere Nachrichten senden, ohne auf die jeweilige Bestätigung warten zu müssen
- Konsequenzen:
 - Wertebereich der Sequenznummern muss angepasst werden
 - Sender und Empfänger müssen mehr als eine Nachricht zwischenspeichern können → Sende/Empfangspuffer
- **Im Beispiel:**
3 Nachrichten anstatt 1 → 3-facher Durchsatz

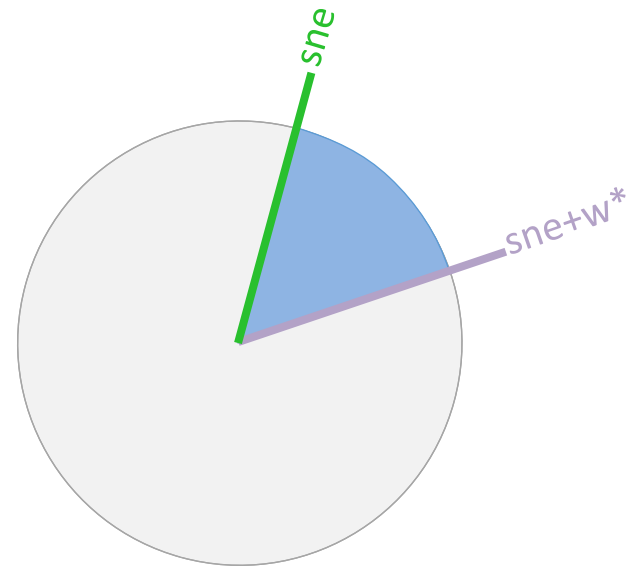
Fensterertechnik („Sliding Window“)

- Festlegung eines Teilbereiches des SeqNr-Raumes als
 - Sendefenster (Sendepuffer)
 - Größe w
 - Beginn Sendefenster: zuletzt (lückenlos) quittierte SeqNr snq
 - Sender sendet nur Nachrichten mit Seq innerhalb Sendefensters
 - Empfangsfenster (Empfangspuffer)
 - Größe w^*
 - Beginn Empfangsfenster: nächste erwartete SeqNr (sne)
 - Nachrichten mit SeqNr außerhalb Empfangsfenster werden verworfen
- Fenster werden verschoben, wenn Nachrichten quittiert oder empfangen werden

Fenster Technik (Animation)



— snq
— snq + w



— sne
— snq + w*

-
- Lage Sendefenster und Empfangsfenster i.A. verschieden
 - $snq \leq sne \leq snq + w$
 - w und w^* können differieren
 - Fenster Technik unterstützt Fluss-/Staukontrolle

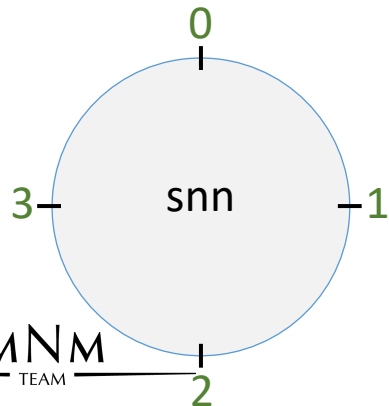
Nachricht

The quick brown fox jumps over the lazy frog.

Quelle/Sender

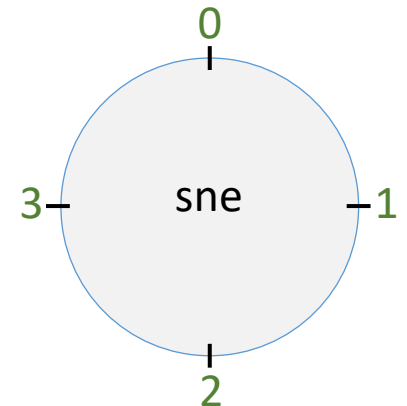
0	T
1	h
2	e
3	┐
0	q
1	u

⋮



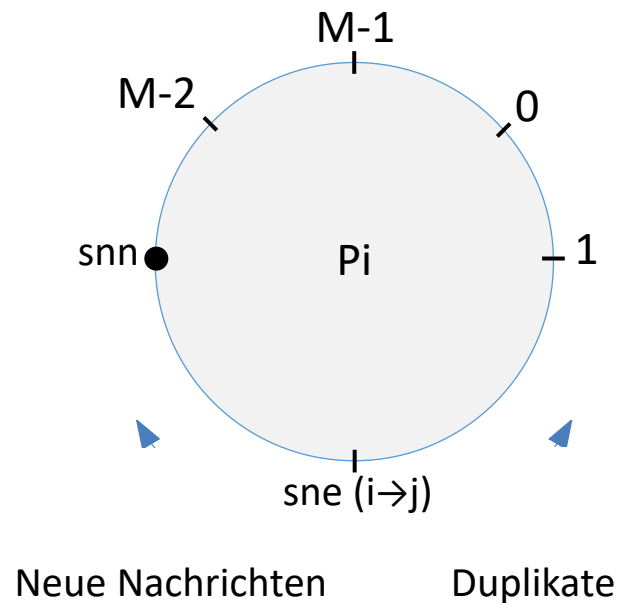
Senke/Empfänger

⋮



Prüfung der Sequenznummern

- Problem: Ist SeqNr neu oder Duplikat?
- Fenstertechnik:



Nachricht

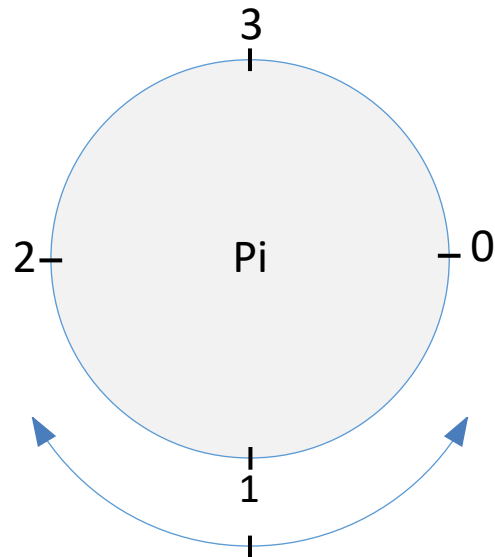
The quick brown fox jumps over the lazy frog.

Quelle

1	u
---	---

Senke

0	T
1	h
2	e
3	└
0	q



Neue Nachrichten

Duplikate

Vorteile der Fenstertechnik

- Allgemein einsetzbar, auch außerhalb RNVS
- Flexibel an Bedürfnisse konkreter Protokolle anpassbar
- Fenstergröße = 1: „Stop-and-Wait“ Protokoll.
- Je nach Protokoll: dynamische Anpassung von Fenstergrößen, u.a. zur Flusskontrolle
- Grundlegende Verfahren:
 - Go-Back N
 - Selective Repeat

Go-Back N (GBN)

Einfaches Pipelining:

Sicht des Senders:

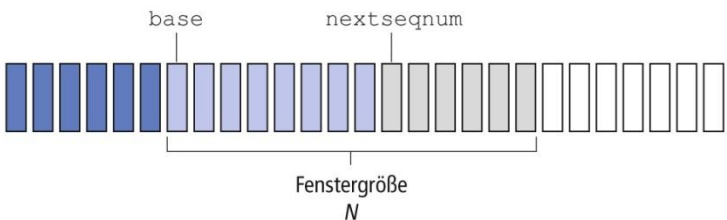


Abbildung 3.19: Sequenznummern aus Sicht des Senders bei Go-Back-N.

- Legende:
- bereits bestätigt
 - gesendet, noch nicht bestätigt
 - nutzbar, noch nicht gesendet
 - nicht benutzbar

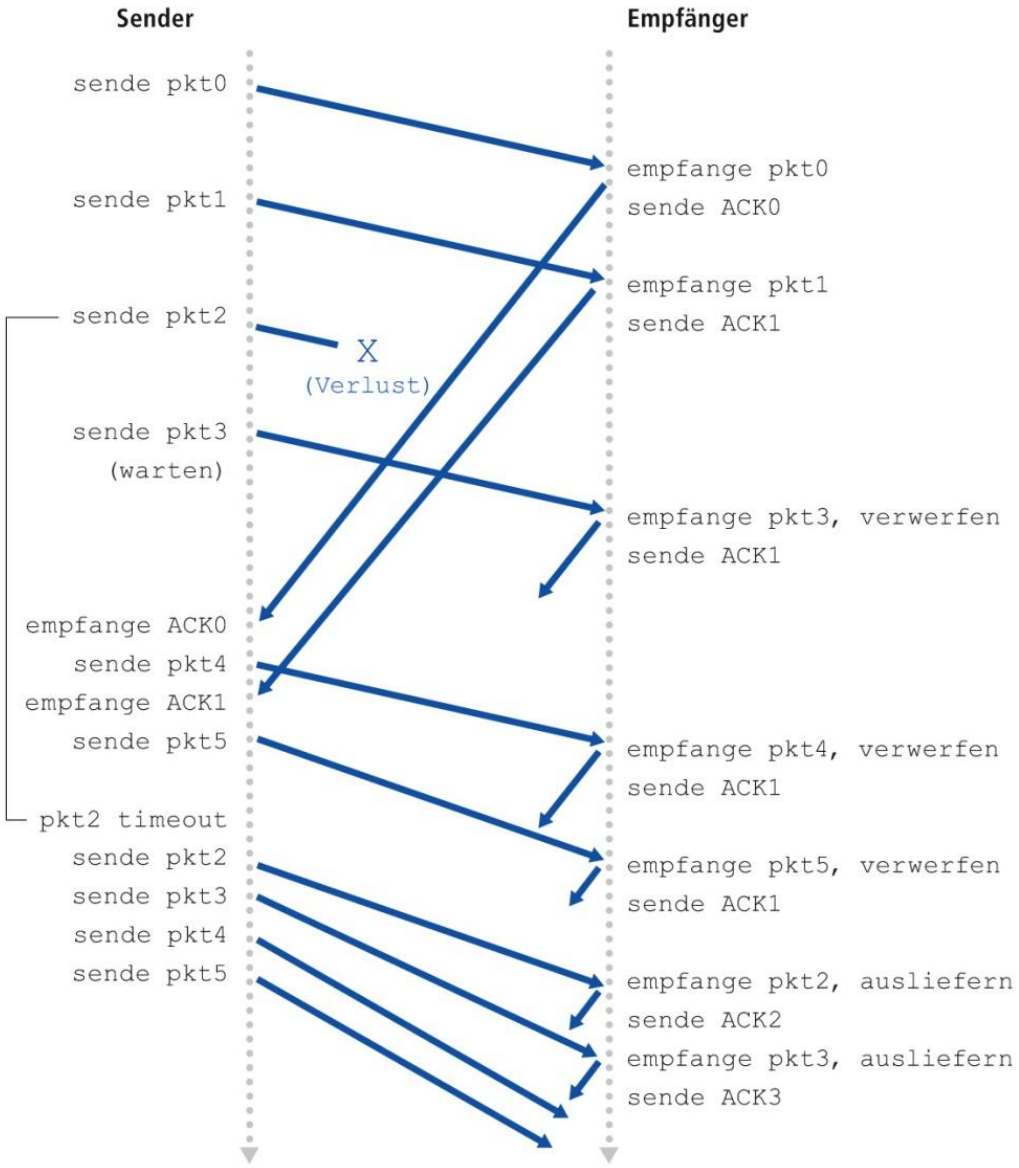


Abbildung 3.22: Arbeitsweise von Go-Back-N.

Selective Repeat (SR)

Kurose/Ross, „Computernetzwerke – Der Top-Down-Ansatz“
2. Auflage, Pearson Deutschland, 2014, S. 249ff

- Bereits bestätigte Segmente innerhalb des Sendefensters werden nicht erneut versandt.

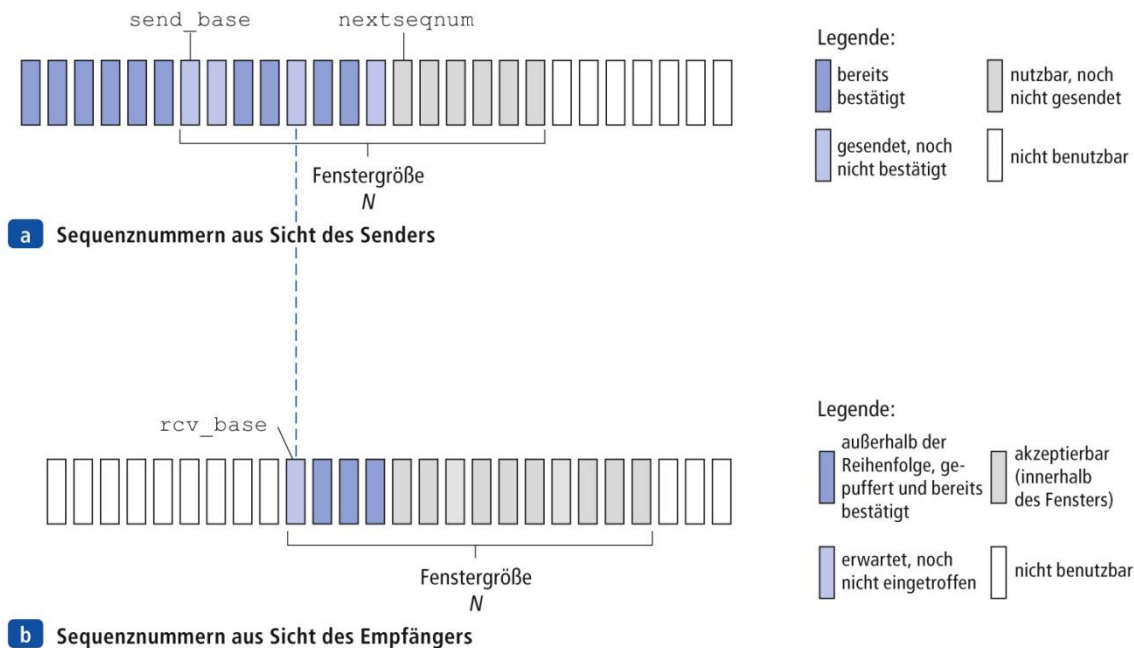


Abbildung 3.23: Sender- und Empfängersicht des Bereichs der Sequenznummern bei Selective Repeat (SR).

Selective Repeat (SR) Ablauf

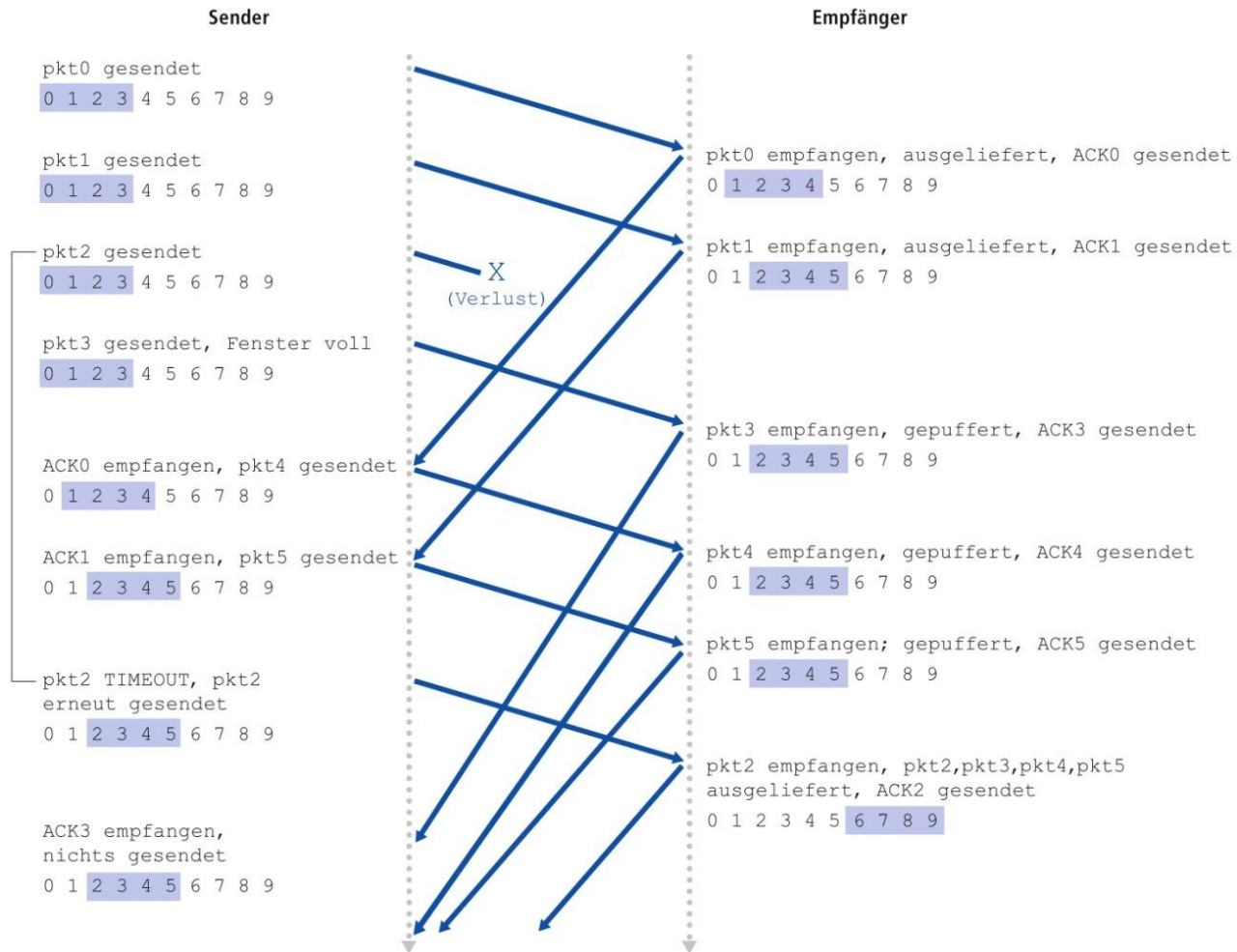


Abbildung 3.26: SR-Arbeitsweise.

Zusammenfassung

- Sequenznummer: fortlaufende Nummerierung (von Bytes) in Segmenten.
- Acknowledgement (ACK): Quittungen über erfolgreich erhaltene Pakete.
- Timer: Wird verwendet, um ggf. verloren gegangene Segmente erneut zu übertragen.
- Fenstertechnik (Pipelining): maximale Anzahl unbestätigter Segmente auf dem Sendekanal.

Verbesserungen

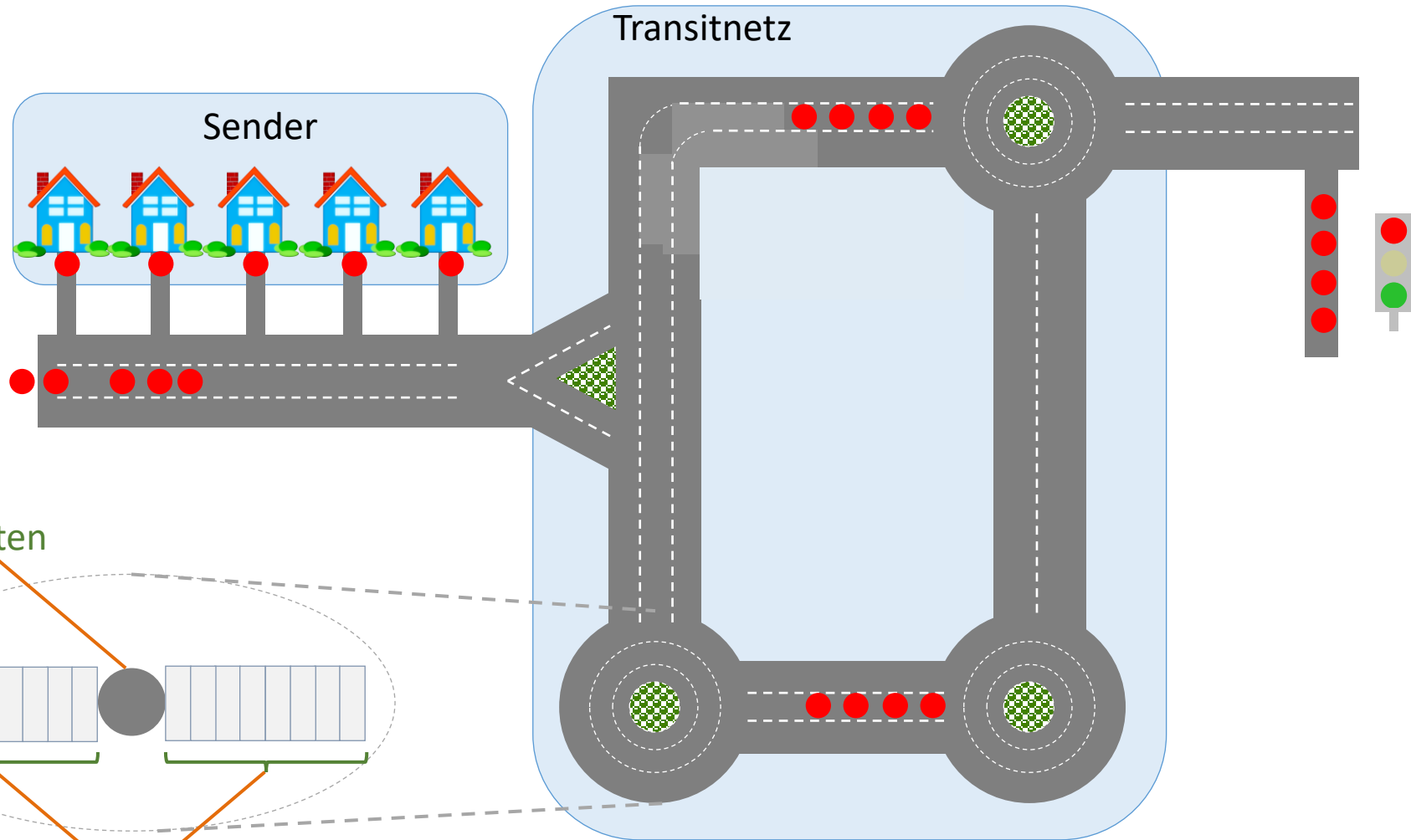
- Kommunikationsleistung ✓
 - Höherer Durchsatz
 - Bessere Auslastung
- Zuverlässigkeit
 - Flusskontrolle
 - Staukontrolle / Überlastkontrolle

Ziel: Höhere Leistung bei geringerer Fehlerrate

Flusskontrolle vs. Staukontrolle

- Flusskontrolle/Flusssteuerung
(engl. „flow control“)
 - Maßnahmen beim Sender
gegen Überlastung des Empfängers
- Staukontrolle/Überlastkontrolle
(engl. “congestion control”)
 - Maßnahmen gegen die Überlastung der Transitnetze bei gemeinsamer Nutzung durch mehrere individuelle Kommunikationsbeziehungen

Warteschlangen (Grafik)



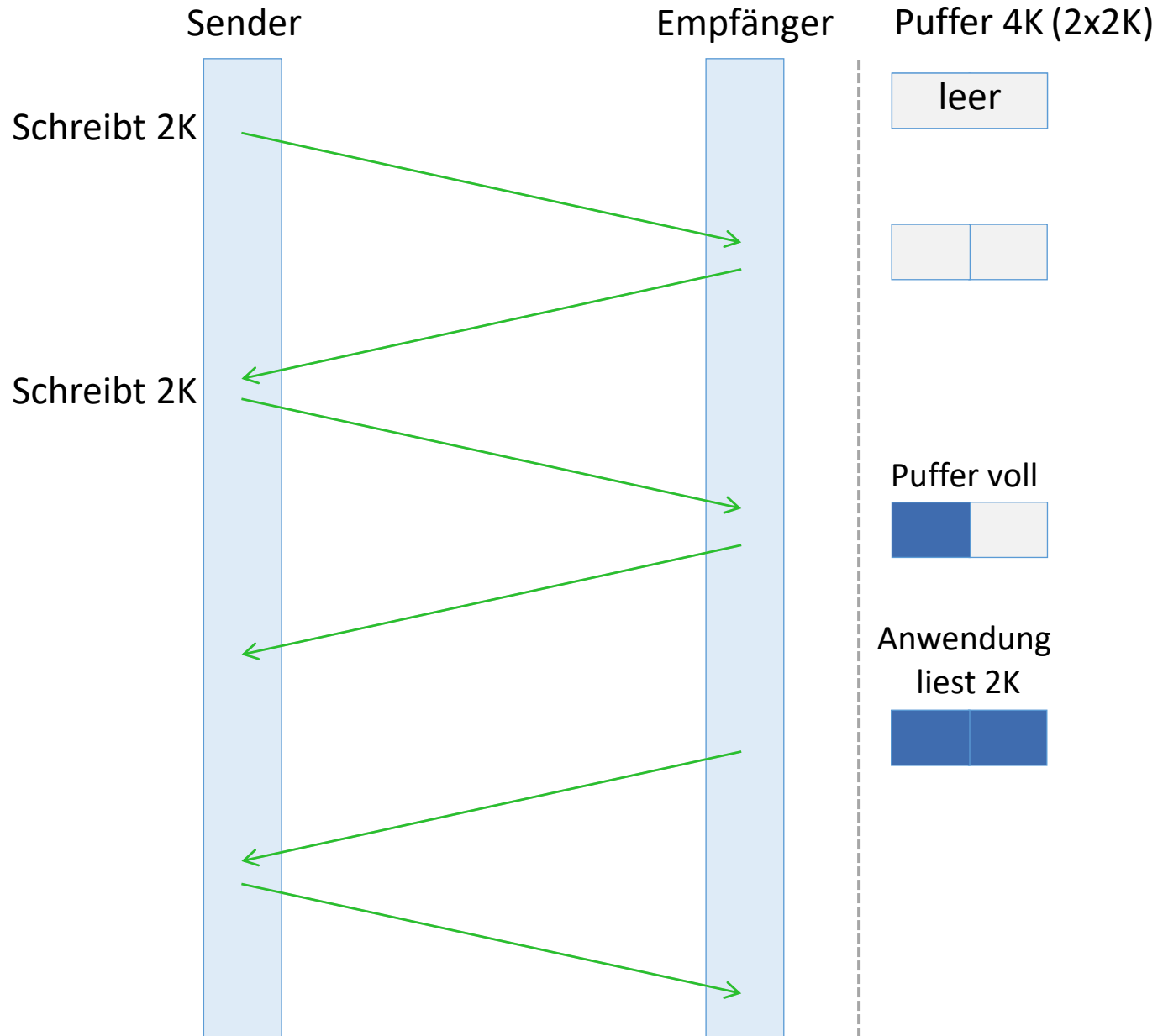
Knoten

Warteschlange

Lösungsansätze Flusskontrolle

- Stop and Wait–Technik → Oszillierendes Verhalten, schlecht bei großer RTD
- Fenstertechnik (mit festem oder dynamischem Fenster) → Problem bei Fensterverkleinerung
- Sender verlangt Pufferreservierung vor Sendung, Senden nach Bestätigung → zusätzliche Nachrichten und Verzögerung
- Empfänger stellt Credit über Allokationsnachrichten (z.B. TCP) → Verfahren nicht robust
- Zeitrasterabhängiges Senden

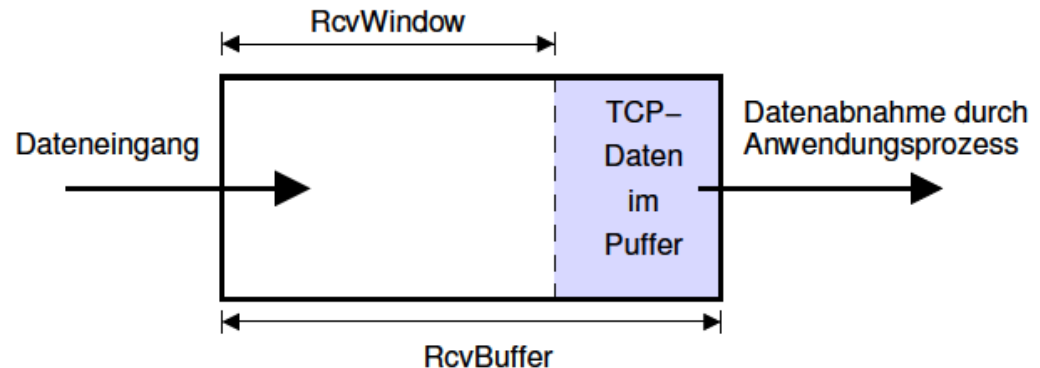
Flusskontrolle (1/2)



Flusskontrolle (2/2)

- Variablen des Senders:

- RcvWindow (w)
- LastByteSent (snn)
- LastByteAcked (snq)



- Variablen des Empfängers:

- RcvBuffer (w^*)
- LastByteRcvd (sne): angekommen, aber noch nicht gelesen (im Puffer)
- LastByteRead: bereits vom Anwendungsprozess gelesen

- $RcvWindow = RcvBuffer - (LastByteRcvd - LastByteRead)$

- $LastByteSent - LastByteAcked \leq RcvWindow$

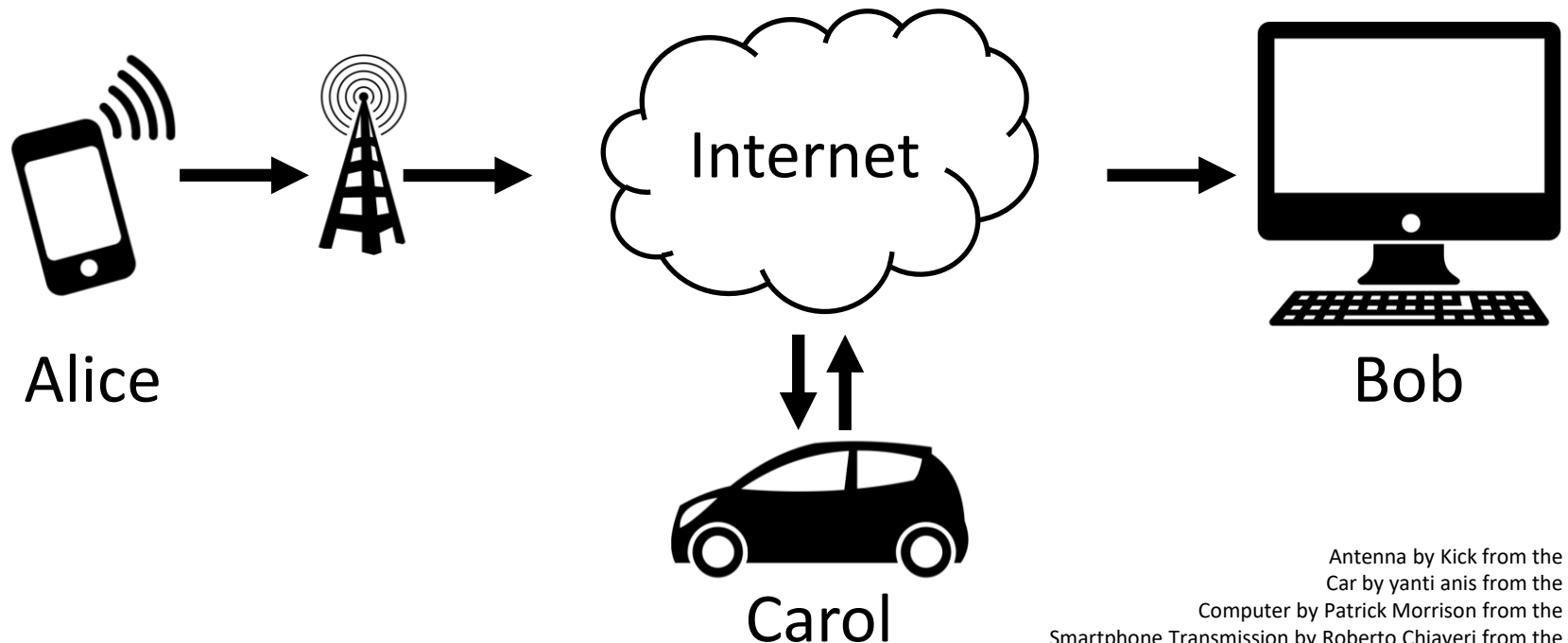
- Sammelquittungen über mehrere Segmente

Flusskontrolle vs. Staukontrolle

- Flusskontrolle/Flusssteuerung ✓
(engl. „flow control“)
 - Maßnahmen beim Sender
gegen Überlastung des Empfängers
- Staukontrolle/Überlastkontrolle
(engl. “congestion control”)
 - Maßnahmen gegen die Überlastung der Transitnetze bei gemeinsamer Nutzung durch mehrere individuelle Kommunikationsbeziehungen

Überlastung der Netze

Zu viele Transportinstanzen auf zu vielen Endsystemen speisen gleichzeitig zu viele Pakete ins Netz ein.



Antenna by Kick from the Noun Project
Car by yanti anis from the Noun Project
Computer by Patrick Morrison from the Noun Project
Smartphone Transmission by Roberto Chiaveri from the Noun Project

Überlastung der Netze

Zu viele Transportinstanzen auf zu vielen Endsystemen speisen gleichzeitig zu viele Pakete ins Netz ein.

➔ Überlastung der Router auf der Vermittlungsschicht

- Wie kann auf Transportschicht festgestellt werden, dass eine Überlastung der Transitnetze besteht?
- Wie kann man darauf reagieren?

Feststellen von Überlastung

- Durch Nutzung eines dedizierten Protokolls, mit dessen Hilfe überlastete Router den Endpunkten explizit eine Überlastung mitteilen
 - z.B.: XCP – eXplicit Congestion Protocol
- Durch indirekte Indikatoren
 - z.B. Segmentverlust

Lösungsansätze Staukontrolle

- virtuelle Kanäle: sicher, aber Puffervergeudung und Aufbauzeit
- Begrenzung des Flusses in das Netz
 - Begrenzung der Anzahl von Verbindungen pro Host/Prozess
 - Begrenzung der Senderate (Traffic Contract, Leaky Bucket)
- Sicherstellung des Abflusses aus dem Netz
 - garantierte Abnahmerate durch Empfänger
 - Reassembly im Host
- Konstante Last im Netz (Taxi-Verfahren, nicht optimal)
- Füllungsregelung pro Verbindung
- Im Internet: Verwerfen von Segmenten bei Überlast

Reaktives Verhalten auf Indikatoren

- Verlorenes Segment (Timeout, ACK-Duplikate)
impliziert „Stau“-Situation im Netz
 - Reaktion: Senderate verringern
- Angekommenes ACK impliziert guten Datenfluss.
 - Reaktion: Senderate erhöhen.
- Kontinuierliches Probieren: Senderate erhöhen bis
Segment verloren geht. Ab dann:
 - Reaktion: Senderate verringern

Überlastungskontrolle (bei TCP) in 3 Schritten

Senderate (ins Netz):

1. Langsamer Start, exponentielles Wachstum
(Slow Start)
2. Lineares Wachstum
(Überlastvermeidung = Congestion Avoidance)
3. Schnelle Erholung
(Fast Recovery)

Vorbereitung: Überlastungskontrolle

- TCP Instanzen benötigen folgende Variablen:
 - CongWindow (Congestion Window):
steuert die Einspeisung ins Netz wie folgt:
 $\text{LastByteSent} - \text{LastByteAcked} \leq \text{CongWindow}$
 - Effektives Fenster: $\min(\text{RcvWindow}, \text{CongWindow})$
 - MSS (maximum segment size):
maximale Datenmenge pro Segment.
 - Threshold: obere Schranke für das schnelle Wachstum des Überlastfensters (CongWindow).
 - RT (retransmission timer): nach Ablauf des Timer werden nicht bestätigte Segmente neu gesendet.

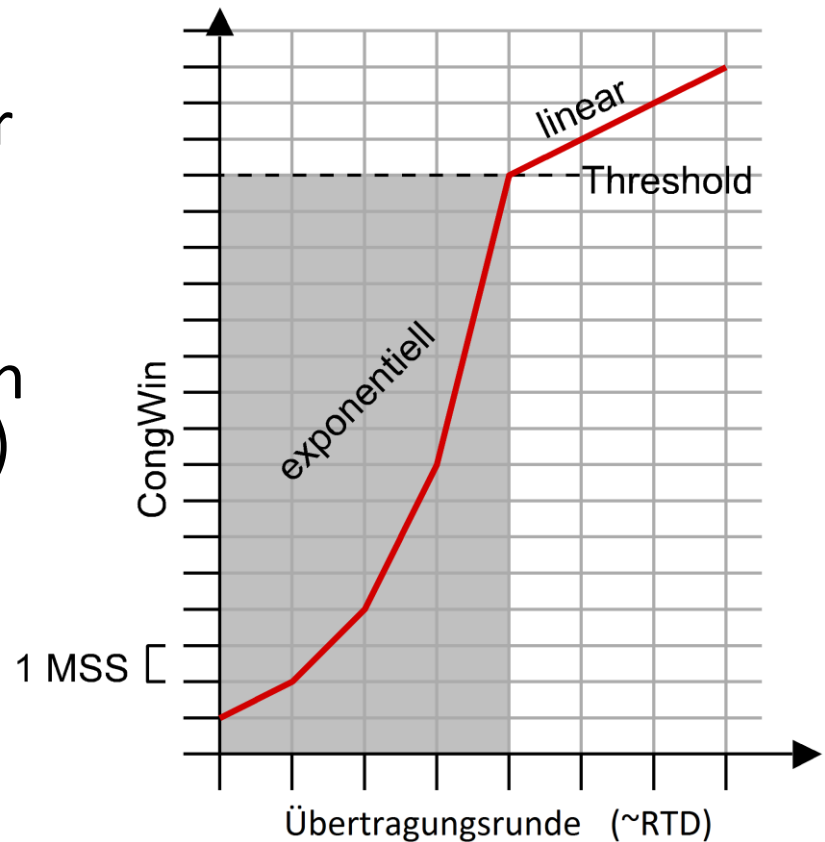
Senderate bei **TCP Tahoe** (bei Netz ohne Last) (1/3)

1. Slow Start:

- mit kleinem Überlastfenster (CongWin = 1 MSS).

Exponentielles Wachstum:

- Bis zum *Threshold*: CongWin verdoppeln (je ACK +1 MSS)
- Jede Runde werden jeweils doppelt so viele Segmente wie davor quittiert
- 1 Übertragungsrunde = ca. 1 RTD



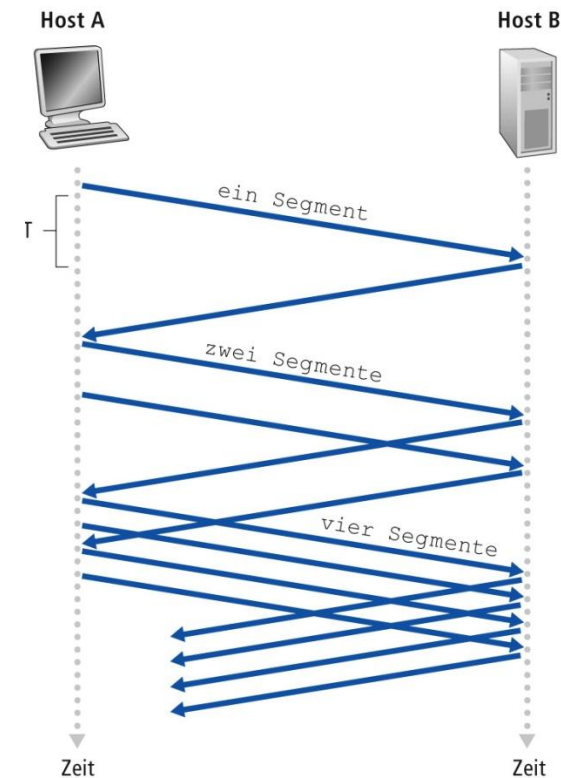
Senderate bei TCP Tahoe (bei Netz ohne Last) (1/3)

1. Slow Start:

- mit kleinem Überlastfenster (CongWin = 1 MSS).

Exponentielles Wachstum:

- Bis zum *Threshold*: CongWin verdoppeln (je ACK +1 MSS)
- Jede Runde werden jeweils doppelt so viele Segmente wie davor quittiert
- 1 Übertragungsrunde = ca. 1 RTD

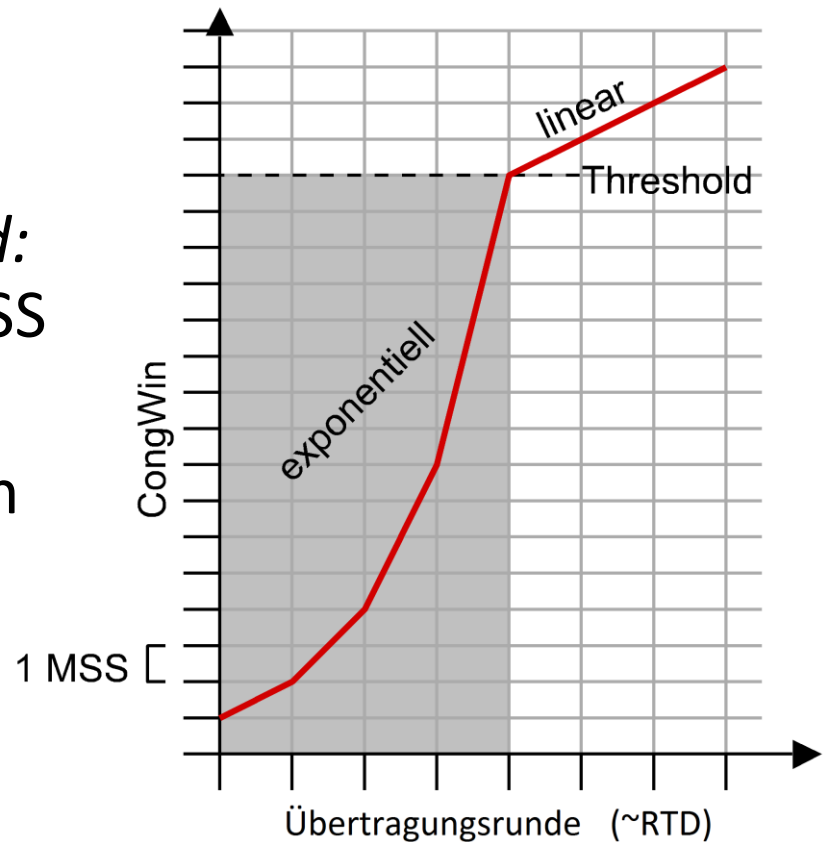


Senderate bei TCP Tahoe (bei Netz ohne Last) (2/3)

2. Congestion Avoidance

Lineares Wachstum:

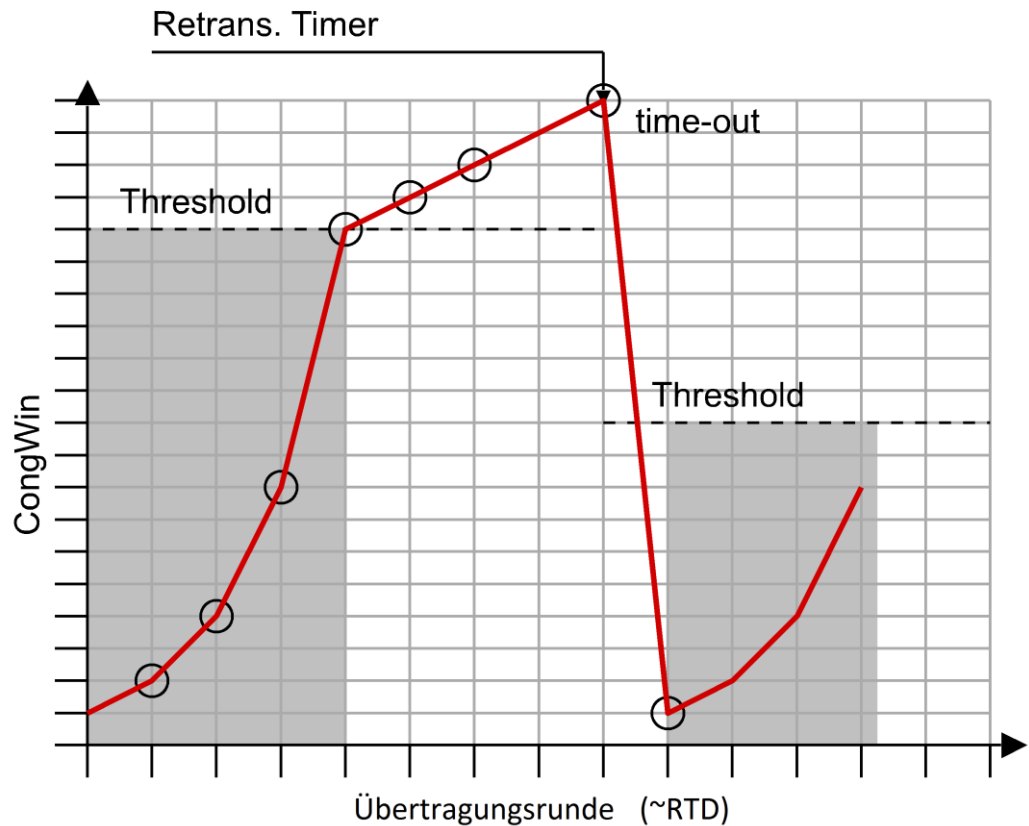
- Bei Erreichen von *Threshold*:
 $\text{CongWin} = \text{CongWin} + 1 \text{ MSS}$
je Runde
- Wiederhole so lange, bis ein
Segment verloren geht
(Timeout)



Senderate bei TCP Tahoe (**bei Netzüberlastung**) (3/3)

Bei Timeout (RT):
Reduzierung der
Senderate:

- Reduzierung des
Threshold =
CongWin/2
- Neustart mit Slow
Start:
CongWin = 1 MSS



Optimierung bei Verlust (RFC 5681)

- Grundlage:

Bei Segmentverlust

- ➔ alle darauf folgenden empfangene Segmente werden mit der gleichen Quittungsnummer quittiert
- ➔ **Quittungsduplikate** weisen frühzeitig (noch vor Timeout) auf Segmentverlust hin

- Mögliche Optimierung:

- Schnelle Neuübertragung (Fast Retransmit)
- Schnelle Wiederherstellung (Fast Recovery)

Optimierung bei Verlust (RFC 5681)

- **Schnelle Neuübertragung** (Fast Retransmit):
Sender wiederholt bei Empfang eines
Quittungsduplikats das folgende Segment
(noch vor Erreichen des Timeout).

Beispiel: Fast Retransmit

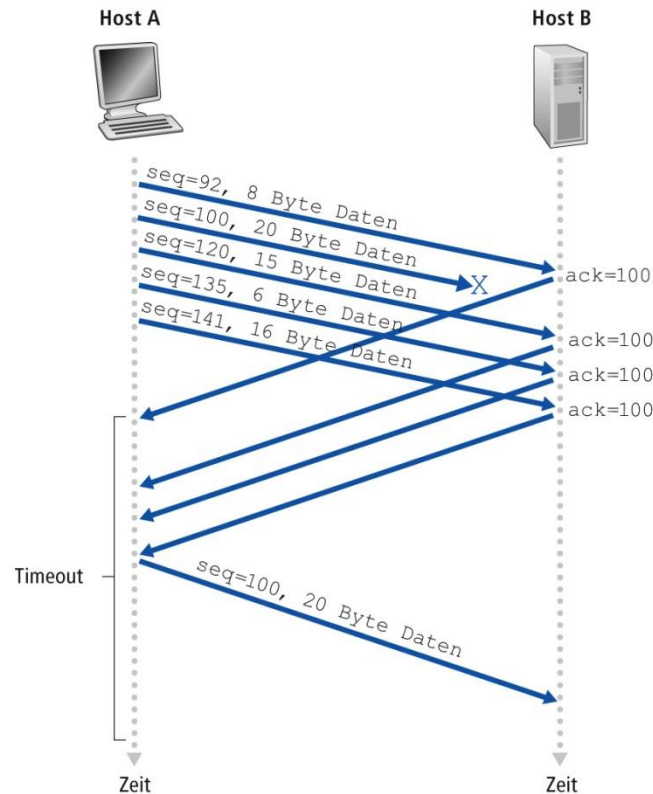


Abbildung 3.37: Fast Retransmit: erneute Übertragung des fehlenden Segments, bevor der Timer des Segments abläuft.

Optimierung bei Verlust (RFC 5681)

- **Schnelle Neuübertragung** (Fast Retransmit):
Sender wiederholt bei Empfang eines Quittungsduplikats das folgende Segment (noch vor Erreichen des Timeout).
 - **Schnelle Wiederherstellung** (Fast Recovery):
Sendeverhalten nach schneller Neuübertragung:
 - CongWin wird respektiert, bleibt aber unverändert
 - ab drittem Segment: Neuberechnung Threshold und CongWin (ohne Slow Start)
 - Pro empfangenem Quittungsduplikat wird ein Segment neuer Daten gesendet. (virtuelles CongWin)
- TCP Reno

Virtuelles CongWin

Pro empfangenem Quittungsduplikat wird ein Segment neuer Daten gesendet:

- Verlorenes Paket wurde nicht quittiert
→ CongWin verschiebt sich nicht weiter
- Um dennoch weiter senden zu können wird CongWin pro Quittungsduplikat um 1 erhöht
- Effektives CongWin bleibt unverändert

Nach der Schnellen Wiederherstellung wird CongWin wieder auf den ursprünglichen Wert zurückgesetzt.

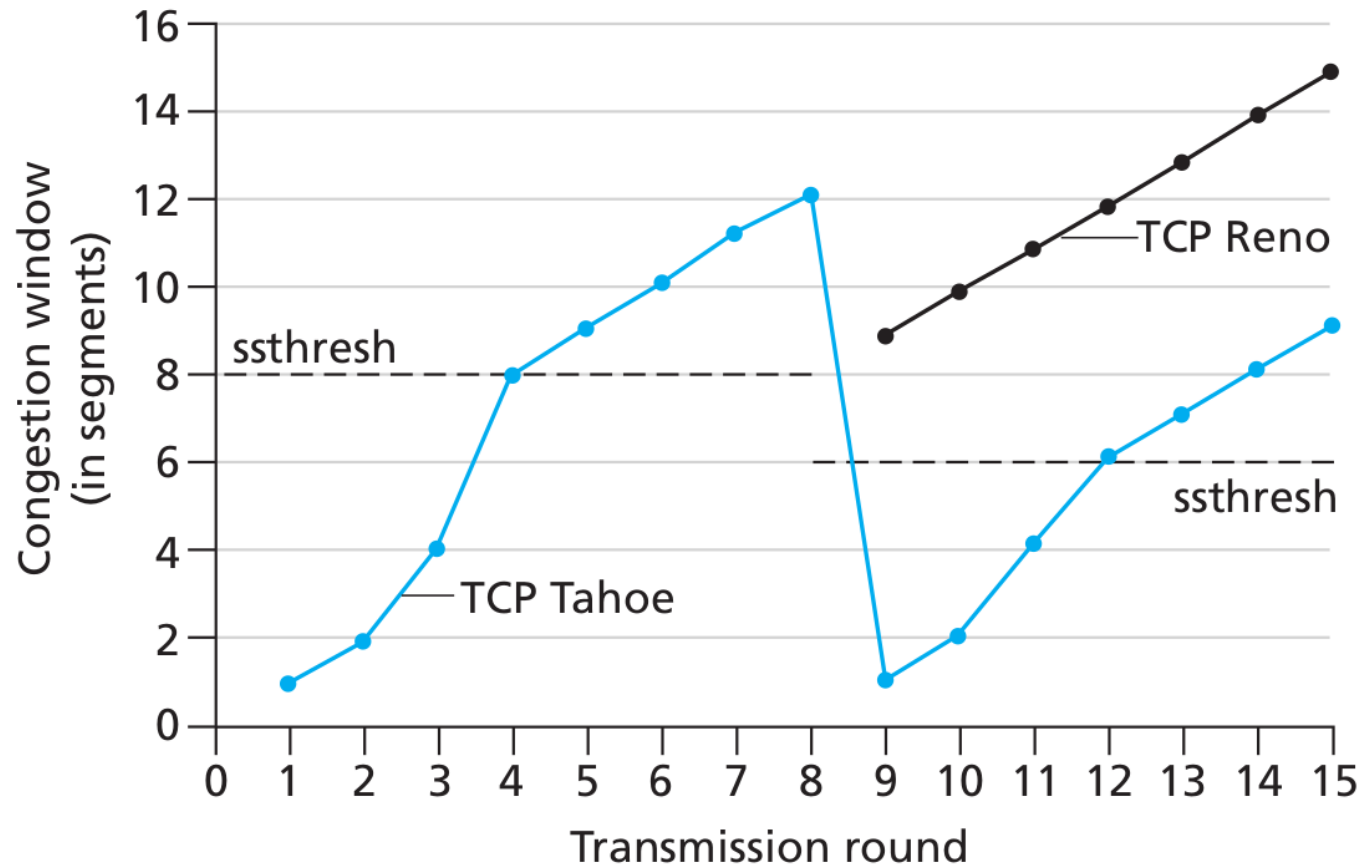
Senderate bei **TCP Reno** (bei Netzüberlastung)

Reduzierung des Threshold = $\text{CongWin}/2$
(identisch zu TCP Tahoe)

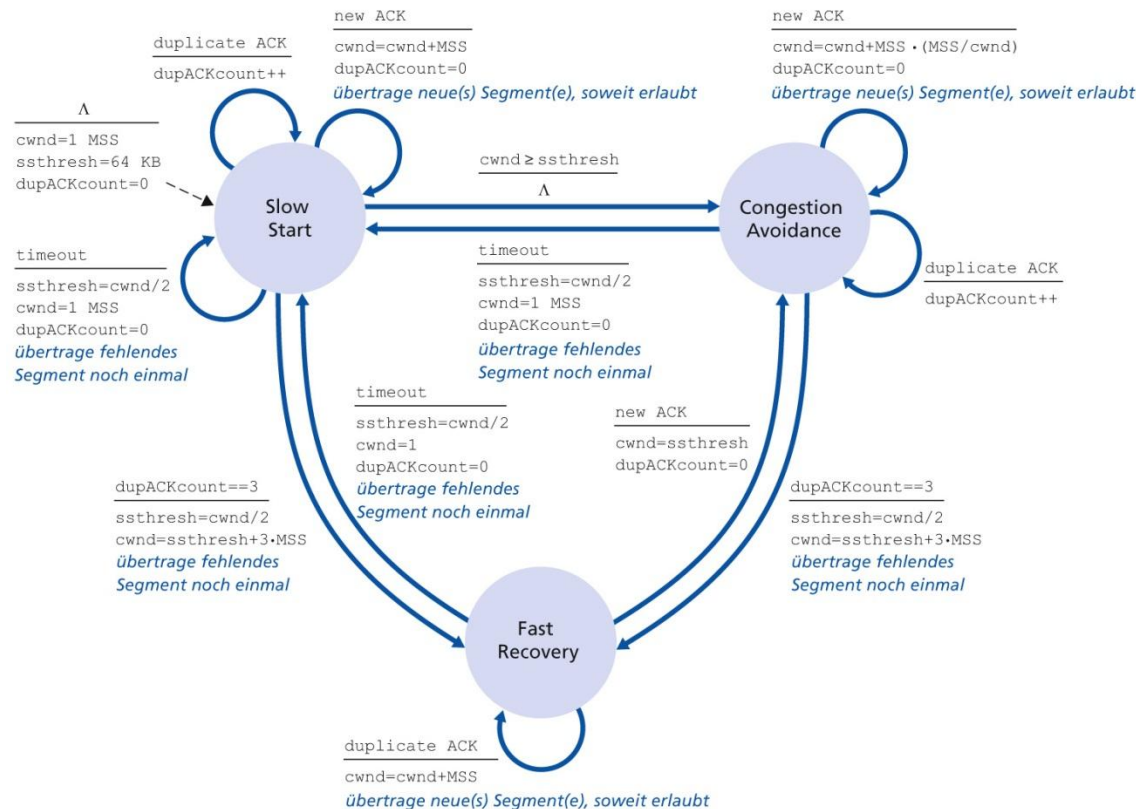
Je nach Fehlerart:

- Bei 3-fachen Quittungsduplikat:
CongWin = neuer Threshold
Weiter mit Congestion Avoidance
- Bei Timeout (identisch zu TCP Tahoe):
CongWin = 1 MSS
Neustart mit Slow Start

Staukontrolle Ablaufdiagramm



Staukontrolle Zustandsdiagramm



TCP Stauvermeidung (pragmatisch)

Additive Increase Multiple Decrease (AIMD) Prinzip.

- Ohne Slow Start, linearer Anstieg (Additiv Increase)
- Bei Paketverlust: Halbierung des CongWin um Faktor 2 (Multiple Decrease).

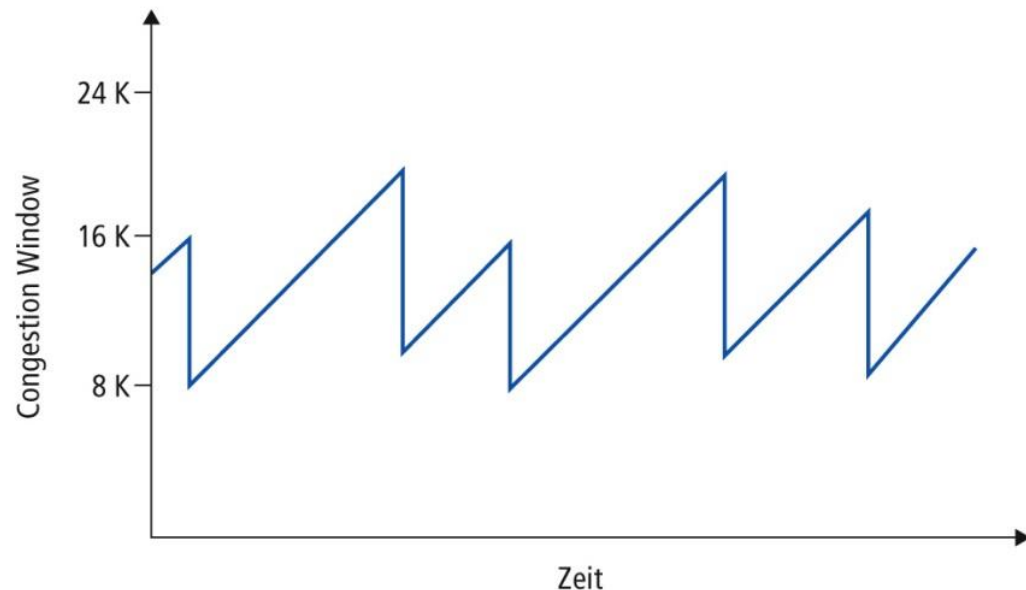


Abbildung 3.54: Überlastkontrolle: Additive-Increase, Multiplicative-Decrease.

Berechnung des Durchsatzes bei TCP

Gegeben:

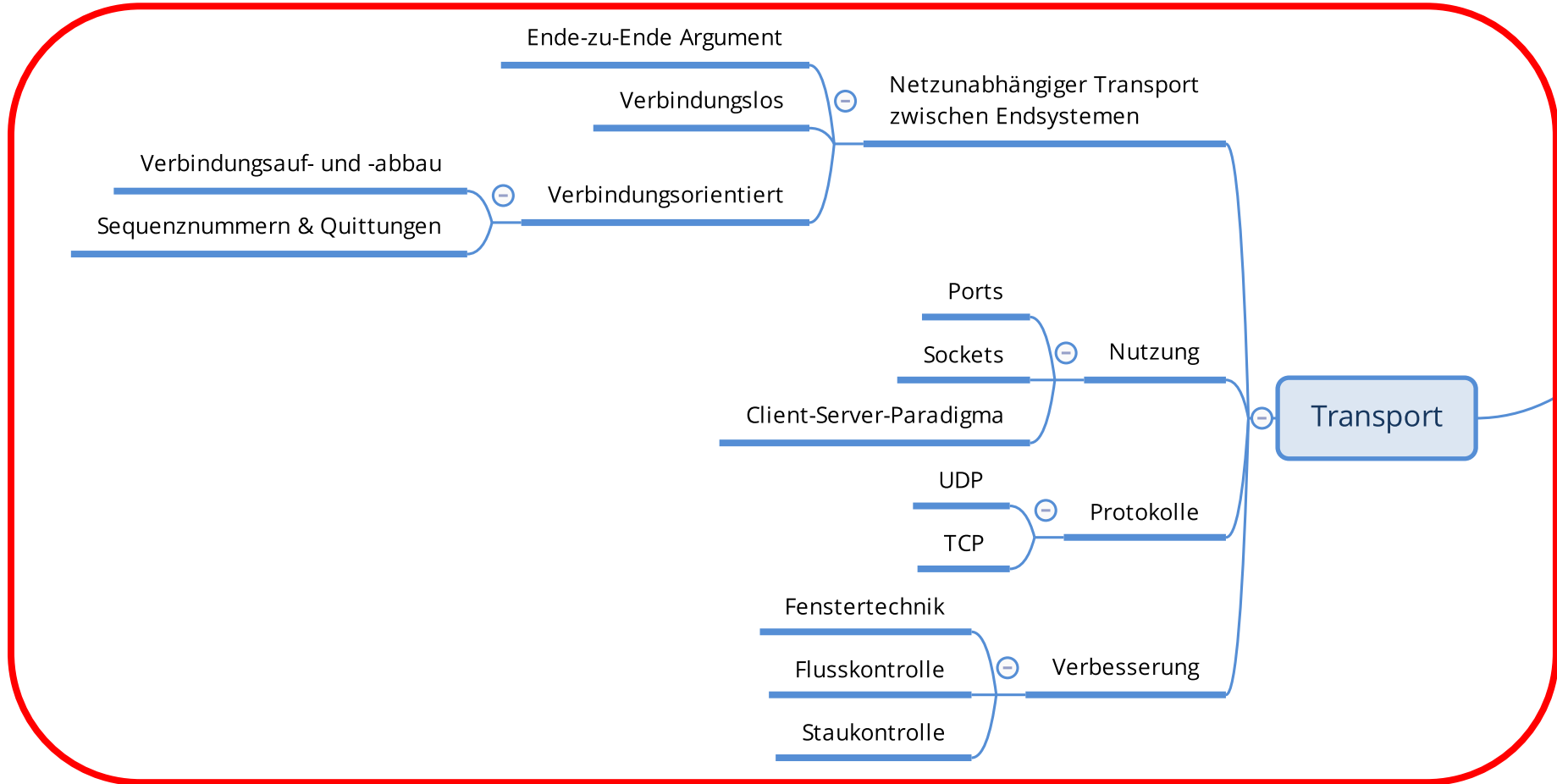
- w : aktueller Wert von CongWin
- W : Wert von CongWin bei Paketverlust
- RTD : Round Trip Delay

Annahme: W konstant über die Verbindung

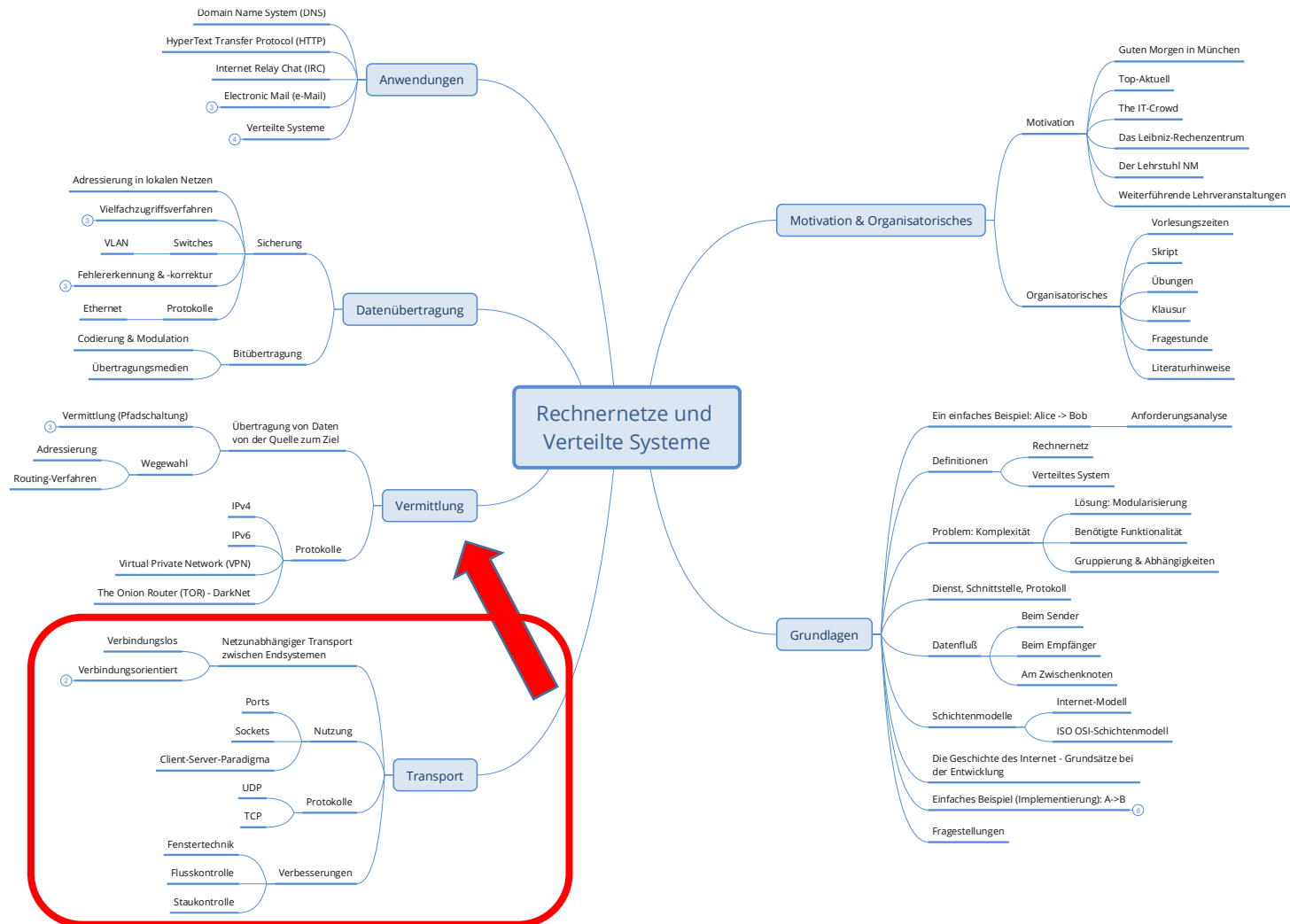
→ Durchsatz pendelt zwischen $\frac{W}{2 RTD}$ und $\frac{W}{RTD}$

→ Ungefährer Durchsatz bei TCP: $\frac{0,75 W}{RTD}$

Zusammenfassung

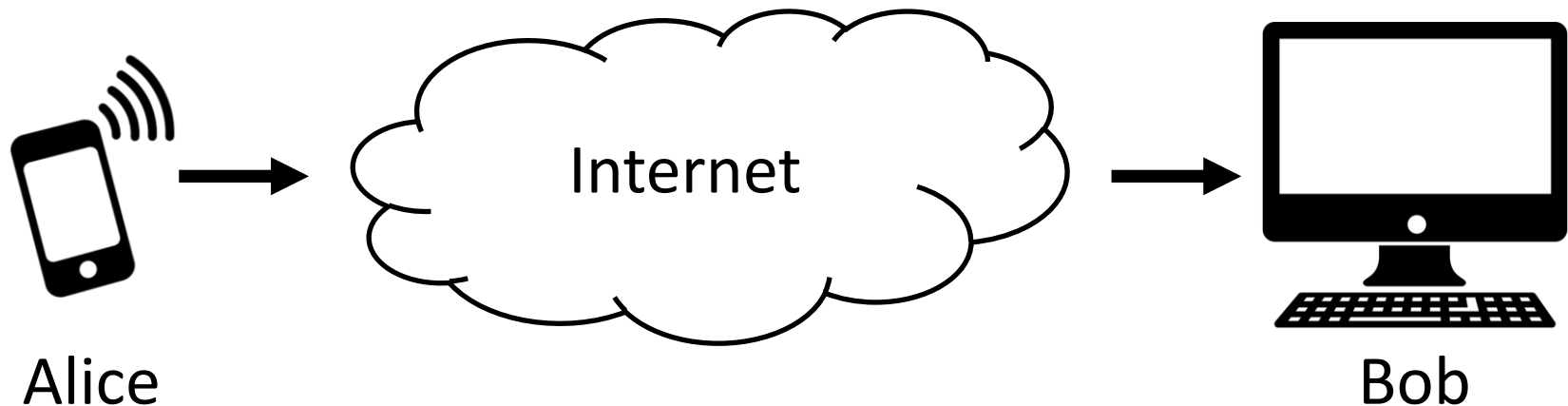


RNVS MindMap



Einfaches Beispiel

- Alice sendet eine Nachricht an Bob



Computer by Patrick Morrison from the Noun Project
Smartphone Transmission by Roberto Chiaveri from the Noun Project

Fragen (1/4)

1. Welche Parameter beeinflussen die Größe des Sequenznummernraumes?
2. Wie wird die Eindeutigkeit von Sequenznummern sichergestellt?
3. Wozu wird die Fenstertechnik eingesetzt?
4. Wie wirken sich zu kleine Fenster aus?
5. Wie wirken sich zu kleine Sequenznummernräume aus?

Fragen (2/4)

6. Warum ist für einen sicheren Verbindungsaufbau auf der Transportschicht ein Drei-Wege-Handschlag erforderlich?
7. Unter welchen Voraussetzungen genügt ein Zwei-Wege-Handschlag für einen Verbindungsaufbau?
8. Warum muss der Verbindungsabbau beidseitig geschehen?
9. Was ist der Unterschied zwischen Flusskontrolle und Staukontrolle?

Fragen (3/4)

10. Welche Anforderungen einer Anwendung können den Einsatz von UDP rechtfertigen?
11. Wie kann man in einer Anwendung, die UDP verwendet, einen zuverlässigen Datentransfer garantieren?
12. Alice sendet 2 TCP-Segmente (mit Sequenznummer 90 und 110) an Bob.
 - Wie viele Daten enthält das erste Segment? Wie lang ist das erste Segment insgesamt?
 - Wie lautet die Quittungsnummer der Bestätigung von Bob, wenn nur das erste Segment verloren geht?

Fragen (4/4)

- 13. Welche Mechanismen verwendet TCP für eine zuverlässige Ende-zu-Ende Verbindung?
- 14. Wie realisiert man „Multiplexen“ von Anwendungen auf der Transportschicht?
- 15. Wie funktioniert TCP Tahoe zur Staukontrolle?
- 16. Können Sende- und Empfangsfenster bei der Fenstertechnik unterschiedlich groß sein?